

操作系统·演进样本

操作系统从哪里开始

从一台没有操作系统的机器逐步演进

CPU Books

本样稿不预先罗列操作系统的组成，而是从一台只能取指、执行和访问内存的机器出发。每一章只引入解决当前问题所必需的新结构，并分别说明它的硬件模型、软件模型与抽象边界。

这册样稿怎样展开

本册不从“操作系统由哪些模块组成”开始。起点是一台可取指、执行和访问内存，但尚未拥有操作系统的机器。读者先亲手完成一次人工装入和运行，再让这种做法暴露问题。每一章继承上一章结束时的机器，只处理当时已经出现的一项具体困难。

第一章只问一项程序怎样在最小裸机上可靠运行；第二章处理人工装入过于繁琐的问题；第三章才让两项程序同时保留运行现场；第四章再处理程序不愿主动交还处理器的情形。尚未由问题要求的结构不会提前出现。

目录

这册样稿怎样展开	ii
第 1 章 第一项程序怎样在最小裸机上运行	1
1.1 先把一次运行落实为几项人工动作	1
1.2 从“求和”要求中辨认机器必须保留的事实	4
1.3 为什么既需要断电保存的位置，也需要运行期可写的位置	7
1.4 一个地址怎样在 RAM 中选中确切的字节	8
1.5 把八条助记符落实为内存中的三十二个字节	12
1.6 一个地址怎样到达唯一器件	15
1.7 时钟与复位怎样把随机上电状态收束到第一条取指	17
1.8 处理器怎样把指令字节变成状态变化	18
1.9 连接不仅传位，还必须约定何时有效	19
1.10 不同指令怎样越过各自的提交边界	21
1.11 处理器内部必须保存哪些运行事实	24
1.12 机器开始运行以前，RAM 中究竟要有什么	28
1.13 把具体的任务字节放入具体地址	31
1.14 启动程序如何判断“现在可以进入任务”	33
1.15 一次人工准备的完整记录	35
1.16 启动检查怎样对应人工记录	36
1.17 人工准备为何无法成为长期办法	37
1.18 从复位到任务入口：控制权怎样真正改变	38
1.19 第一项任务怎样逐步得到 21	40
1.20 把整次运行放到同一条时间线上	42
1.21 怎样证明描述的确是同一次运行	44
1.22 从循环不变量复算结果	45
1.23 用反事实检查准备协议的每一道边界	45
1.24 重复运行为什么不能只按一次复位	46
1.25 当前机器的责任边界	47
1.26 逐条指令 trace：不省略循环中的状态	48

第 2 章 为什么人工装入需要变成机器自己的工作	52
2.1 从人工写入转向运行中的字节接收	52
2.2 让外部任务进入机器：串行控制器	53
2.3 装入不是复制一遍就结束	54
2.4 从固件现场建立任务现场	57
2.5 把装入器写成可审计的状态机	59
2.6 把固件主循环连成完整控制程序	66
2.7 沿着处理器状态走完求和任务	69
2.8 正常路径的端到端时间线	71
2.9 错误路径揭示了当前机器缺少什么	72
2.10 为什么这仍然不是操作系统	74
2.11 为什么四轮结束时一定得到 21	75
第 3 章 第二项任务为什么需要常驻协调代码	83
3.1 直接从 A 跳到 B 会丢掉什么	83
3.2 现场保存在哪里	83
3.3 谁有资格保存现场	84
3.4 第一次运行和恢复运行为何不同	84
3.5 最小选择规则怎样形成	85
3.6 一次切换的状态账本	85
3.7 这段常驻代码已经是什么	85
第 4 章 任务不交还处理器时怎么办	87
4.1 常驻并不等于有机会执行	87
4.2 为什么选择可编程时钟事件	87
4.3 硬件保存的状态为什么通常不够	90
4.4 事件入口使用哪一份栈	90
4.5 怎样把事件帧并入 B 的任务记录	91
4.6 第一次出现可运行集合	91
4.7 “被唤醒”和“正在运行”尚未出现	92
4.8 一次强制切换的逐时刻状态	92
4.9 本章新增结构的准确边界	92
样稿停在这里	93

第 1 章

第一项程序怎样在最小裸机上运行

先做一件可以亲眼核对的事。机器断开运行，复位保持有效；操作人员把一段程序和四个整数写入指定的存储位置，然后释放复位。机器完成计算以后，另一个指定位置应出现整数 21。这一次运行不依赖文件、命令行、进程或系统调用，没有任何操作系统替人安排步骤。

真正的问题不是“求和怎么算”，而是桌面上的一组器件怎样共同兑现这项要求。接通电源不会自动产生第一条指令；一串存储字节也不会自己变成正在进行的计算。我们必须先认清当前机器有哪些部件、它们之间传递什么，再逐步建立地址、程序字节和处理器状态的含义。

1.1 先把一次运行落实为几项人工动作

桌面上只有一块接好电源的电路板和一台能够向存储单元写入字节的调试工具。操作人员希望运行一段求和程序，输入是

7, -2, 11, 5,

预期结果是 21。在没有操作系统的条件下，这项要求首先表现为四项具体动作：

1. 保持复位，使处理器暂时不能执行普通指令；
2. 把程序字节、四个输入整数和最初需要的栈内容写到约定位置；
3. 设置完成后释放复位，让处理器从固定起点开始；
4. 等待程序停止或返回，再读取结果位置中的四个字节。

这里的调试工具只在机器停止时帮助写入 RAM。它不参与程序运行，也不是程序可以调用的设备。采用这种安排不是因为它方便，而是因为它暴露了最朴素的方法：人在运行之前把每个必要字节摆好，机器只负责执行。第二章才会处理这种人工装入为何难以长期使用。

“程序”这个名称暂时只表示一段按顺序解释的指令字节。“一次运行”则表示处理器从约定起点开始，根据这些字节反复更新自身状态和存储内容，直到到达约定的结束位置。此时还没有程序身份、进程状态或资源管理对象。

1.1.1 为什么不能只有处理器和一片 RAM

假设机器只接处理器和 RAM。处理器内部的状态在刚上电时未必具有确定值，RAM 中的字节也会在断电后丢失。即使操作人员已经写好程序，仍有两个问题没有答案：处理器从哪个地址开始读取；写入工具撤走以后，谁保证每次释放复位都回到相同起点。

因此，运行机器还需要一小段断电后仍保留的启动字节。它们放在非易失启动存储中。复位释放时，处理器先从这段固定内容开始执行；固定内容检查本次人工装入的布局，准备寄存器和栈，最后才把指令位置交给 RAM 中的求和程序。后文把这段最先执行的软件称为启动程序或固件。

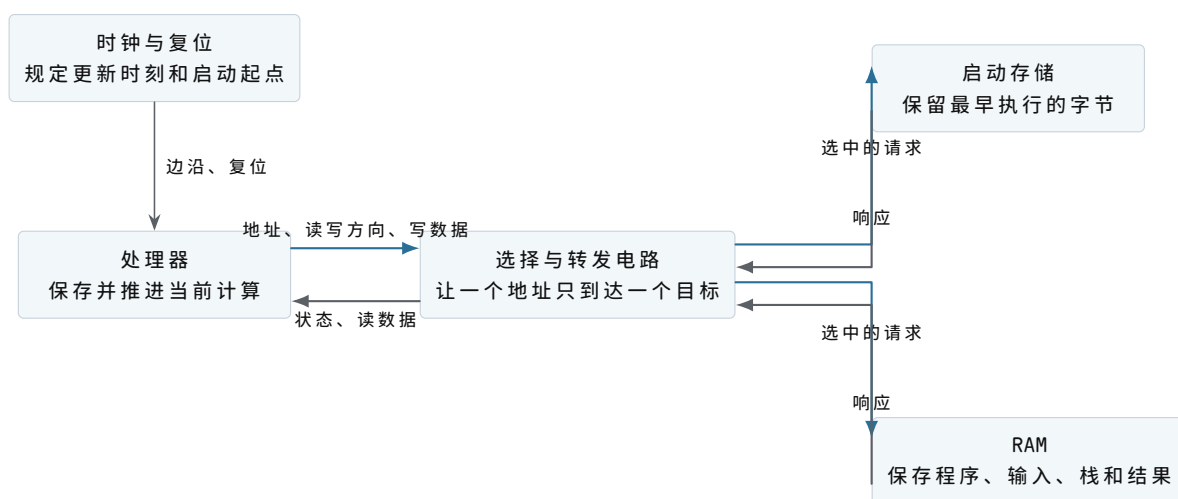
RAM 仍然不可替代。求和程序要保存输入、栈和最终结果，这些内容会在运行期间改变。启动存储负责“断电之后仍然存在”，RAM 负责“运行期间能够改写”，两者解决的是不同问题。

1.1.2 五类部件怎样形成一台可运行机器

处理器、启动存储和 RAM 还不能直接随意并联。处理器发出一个地址时，必须只有一个目标回答；目标返回较慢时，处理器还必须等待结果。机器因而使用一组选择和转发电路，把一次请求交给唯一器件，再把器件的响应送回处理器。后文在解释选择规则后，才把这组电路称为地址互连。

此外，寄存器更新需要共同的时序边界，刚上电的随机状态需要被收束到固定起点。时钟源提供周期性边沿，复位电路在电源尚未稳定时阻止普通执行，并在释放时给处理器规定初始状态。

至此，当前运行机器包含五类必要部件：



图示：图中只有当前真实存在的硬件。调试工具在复位期间预置 RAM，运行开始后退出；图中没有操作系统、调度器、系统调用或可编程计时器。

这幅图只提供方位，不代替后面的推导。处理器怎样表示下一条指令、请求中为何需要地址和方向、启动存储与 RAM 怎样占用不同范围、复位在哪个边沿释放，都还要用具体数值说明。

1.1.3 连接不是一条没有含义的双向线

处理器读取时，地址和读请求从处理器流向目标，数据从目标返回。处理器写入时，地址、写数据和允许更新的字节位置流向目标，目标返回成功或失败状态。时钟与复位则沿另一组连接送到需要按边沿更新的部件。不同方向承载不同责任，不能用一根无说明的线概括。

连接方向	运行时携带的内容	接收方必须完成的判断
处理器到选择电路	地址、读取或写入、宽度、写数据	哪个器件拥有该地址
选择电路到目标	已选中的请求和器件内部位置	请求是否合法，何时可以完成
目标到处理器	成功或失败、读取数据	当前指令能否提交结果
时钟与复位到处理器	周期边沿、复位是否有效	保持旧状态还是进入规定起点

表中尚未给信号命名，因为读者首先需要理解信息方向。等到讨论快慢不同的器件时，再引入“有效”和“就绪”等握手信号，并说明它们在哪个边沿共同成立。

1.1.4 当前程序将占用哪些位置

整体方位明确后，才需要给本次运行分配具体位置。教学处理器能够表达 32 位地址；每个地址指出一个字节。启动存储位于低地址，RAM 位于另一段地址。求和程序、输入、结果和栈在 RAM 中彼此分开：

地址或范围	人在释放复位前放入什么	运行期间由谁改变
0x00000000--0x0000ffff	固定启动程序	普通运行不能改写
0x00100000--0x0010001f	八条求和指令	本次程序不改写
0x00102000--0x0010200f	四个输入整数	本次程序只读取
0x00102020--0x00102023	初始清零的结果槽	求和程序写入 21
0x001effffc--0x001efffff	返回启动程序的地址	程序结束时读取

这些名称不是存储器内部的类型。RAM 只保存位；“指令”“输入”“结果”和“返回地址”来自处理器在不同阶段怎样使用同一组字节。地址表目前也只是本次人工约定，尚无硬件权限阻止程序写到别的区域。

1.1.5 把目标写成可以检查的启动契约

释放复位前，操作人员和机器之间需要一份最小约定。否则即使字节都已写入，启动程序也不知道入口、输入数量和结果位置。当前样机采用以下具体值：

约定项	本次取值	为什么必须明确
-----	------	---------

求和程序入口	0x00100000	决定第一次任务取指的位置
输入首地址	0x00102000	决定第一个整数从哪里读取
输入数量	4	决定循环何时结束
结果地址	0x00102020	让程序与观察者指向同一结果槽
初始栈位置	0x001efffc	保存程序结束后的控制去向
预期结果	21	提供可独立核对的外部事实

启动程序会把输入首地址、数量和结果地址放进处理器寄存器，并把栈指针指向预置返回地址。这里先说明约定的作用，寄存器和栈为何能够保存这些值将在下一节从计算中逐步得到。

1.1.6 什么时候才算完成了第一次运行

“机器开始动了”不是完成标准。对本章而言，成功至少要同时满足四项可检查事实：

- 1. 复位释放后，第一笔取指来自规定的启动位置；
- 2. 启动程序准备的入口、参数与栈恰好等于上表；
- 3. 求和程序逐条执行，结果槽最终保存字节 15 00 00 00；
- 4. 程序沿预置返回地址回到启动程序，启动程序停止继续改写本次结果。

第四项把“结果已经写入”和“本次控制流已经结束”分开。程序可能先写出 21，随后又因错误继续执行；只观察结果槽不足以证明整次运行正确。后文将分别给出结果写入和控制返回的状态快照。

本章尚未解决的问题。操作人员必须在每次运行前保持复位，逐区域写入程序、输入和栈，再检查地址是否一致。机器自身既不能从外部接收一份新程序，也不能判断一串到达的字节是否完整。先把这次人工运行讲清楚，章末再用这些具体成本推动下一次演进。

1.2 从“求和”要求中辨认机器必须保留的事实

整体连接已经给出了机器的方位，现在沿着其中的处理器继续追问。设想求和程序刚读完第一个整数 7，下一步准备读取 -2。如果此刻把机器中的全部状态抹去，再告诉它“继续”，它无法判断下一条指令在哪里，也不知道已经累加出的部分和、尚未读取的数组位置以及剩余元素个数。因此，“运行”不能只表示某段指令存在；运行意味着若干事实在相邻动作之间得以保留。

对这个求和循环，至少有五项事实不能丢失：

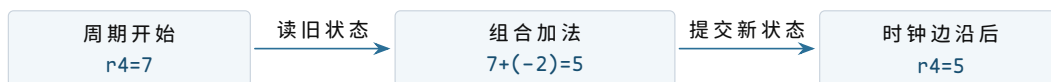
- 1. 下一条机器指令的地址；
- 2. 当前部分和；
- 3. 下一个输入元素的地址；

4. 尚未处理的元素个数；
5. 函数返回时应当回到位置。

这些事实的含义来自程序约定，电路只需要保存对应的位串。教学处理器把“下一条指令地址”单独保存在程序计数器 (program counter, PC) 中，把计算中的位串保存在通用寄存器中，把函数调用形成的返回地址保存在由栈指针 (stack pointer, SP) 定位的内存区域中。PC 和 SP 的名称描述用途，不表示里面的位具有特殊材料；它们仍是普通二进制状态。

1.2.1 状态与组合计算为什么必须分开

假设加法器的两个输入分别为 7 和 -2。只要输入稳定，加法器的输出就会稳定为 5。但加法器本身并不记得这个结果；输入变化后，输出随之变化。若下一条指令还要使用 5，就必须在规定时刻把它写入寄存器。寄存器在一个时钟边沿接收新值，在下一个边沿到来前保持该值，这使连续动作能够首尾相接。



图示：图中的加法结果在时钟边沿之前只是候选值；写入寄存器后，它才成为下一条指令可以读取的处理器状态。

“提交”在本章中表示一次处理器状态更新已经越过时钟边界。它不表示数据已经断电保存，也不表示整个任务完成。后续讨论存储和设备时，还会出现不同层次的完成边界。

1.2.2 给教学处理器规定一个足够小的软件视图

真实处理器的寄存器和指令很多，本章只保留运行该任务所需的部分。教学处理器有八个 32 位通用寄存器 `r0--r7`，另有 PC、SP 和零标志 `Z`。寄存器的用途由启动约定和任务代码共同决定：

状态	本次任务中的初始含义	执行期间怎样变化
PC	任务入口 <code>0x00100000</code>	顺序前进，分支和返回会改写它
SP	指向预置返回地址	调用时下降，返回时上升
<code>r0</code>	输入数组首地址 <code>0x00102000</code>	每轮增加 4，指向下一个整数
<code>r1</code>	元素个数 4	每轮减 1，为 0 时循环结束
<code>r2</code>	结果地址 <code>0x00102020</code>	本任务中保持不变
<code>r4</code>	部分和，初值 0	每轮加上一个数组元素
<code>r5</code>	当前读出的整数	每轮被下一次装入覆盖
<code>Z</code>	最近一次规定运算的结果是否为 0	供条件分支读取

这张表同时说明了一个重要边界：处理器不会识别“数组首地址”或“元素个数”。当启动程序把 `0x00102000` 写进 `r0` 时，硬件只得到一个 32 位位串。只有任务按

照约定把它用作装入指令的基址，它才表现为数组地址。

1.2.3 指令也必须以字节形式存在

处理器不能执行写在纸上的“求和”二字。它读取固定格式的机器指令位串，根据其中的操作码和操作数字段打开不同的数据通路。本章采用固定 4 字节的教学指令。第一个字节是操作码，第二、第三个字节通常给出寄存器编号，最后一个字节可表示小范围立即数或地址偏移。分支指令把后两个字节解释为相对位移。

指令形式	状态变化	本章中的用途
MOVI rd,imm	$rd \leftarrow imm$	把部分和设为 0
LOAD rd,[rb+d]	$rd \leftarrow MEM32[rb+d]$	从数组读取一个整数
STORE rs,[rb+d]	$MEM32[rb+d] \leftarrow rs$	写回最终结果
ADD rd,rs	$rd \leftarrow rd+rs$	更新部分和
ADDI rd,imm	$rd \leftarrow rd+imm$	移动指针或减少计数
BNEZ rs,target	$rs \neq 0$ 时改写 PC	控制循环
CALL target	压入返回地址并改写 PC	调用子程序
RET	从栈中取回 PC	返回调用者

这不是某个商业处理器的真实编码，而是一个字段完整、状态变化明确的教学接口。后文讨论的操作系统原理不依赖某个特定指令集；使用固定编码的意义在于，每次“取指”和“执行”都能对应到确切地址、确切字节和确切状态变化。

求和任务的核心指令如下。左侧地址按每条指令 4 字节递增：

```

0x00100000 MOVI  r4, 0
0x00100004 LOAD  r5, [r0+0]
0x00100008 ADD   r4, r5
0x0010000c ADDI  r0, 4
0x00100010 ADDI  r1, -1
0x00100014 BNEZ  r1, 0x00100004
0x00100018 STORE r4, [r2+0]
0x0010001c RET

```

第一条指令把 **r4** 置零。循环体每次读取一个 32 位整数，更新部分和，将输入指针移到下一个整数，再减少剩余数量。计数不为零时，PC 回到 **0x00100004**；第四轮后分支不成立，任务写出结果并执行 **RET**。

复算点。若 **r0=0x00102000**、**r1=4**，循环执行两轮后应有 **r0=0x00102008**、**r1=2**、**r4=5**。这里只计算程序语义；这些字节怎样进入内存、处理器怎样读到它们，仍未解决。

1.3 为什么既需要断电保存的位置，也需要运行期可写的位置

现已有一段机器指令，但还要说明复位后最早执行的字节为什么可靠。若所有字节都只放在 RAM 中，断电后内容消失；下一次通电时，PC 即使得到一个固定初值，也读不到可靠指令。机器因此需要一片非易失启动存储，保存最早执行的启动程序。

仅有启动存储仍不够。求和任务要更新栈、部分数据和装入后的任务区域，而本章把启动存储视为运行期只读。机器还需要 RAM，为会变化的字节提供位置。两类存储都能响应地址读取，但它们的保留时间和写入规则不同，不能用一句“内存”混在一起。

1.3.1 启动存储的三层含义

硬件模型。启动存储是一组按地址索引的非易失字节单元。输入是读地址和读请求，输出是相应字节及完成状态。上电期间的普通写请求不会改变它；如何烧写固件属于机器制造或维护阶段，本章不展开。

软件模型。启动存储占用物理地址 `0x00000000` 至 `0x0000ffff`。复位入口、人工装入检查程序和返回入口都位于其中。程序使用普通取指或读指令访问这段地址，但写入会得到错误响应。

抽象。启动软件可以依赖一段跨断电保留且复位后立即可读的字节序列。这个抽象只保证地址与字节，不提供文件名、目录或版本选择；这些含义若有需要，必须由固件格式另行建立。

1.3.2 RAM 的三层含义

硬件模型。RAM 是按地址索引的易失存储单元阵列。读请求返回旧值，写请求按照地址、写数据和字节使能更新选中单元。电源失效后，内容不再可靠。

软件模型。物理地址 `0x00100000--0x001ffffff` 可读写。任务代码、输入数据、栈和固件工作区都必须在这一范围内划出不重叠区域。RAM 不保存“代码”“整数”或“栈帧”标签，同一组字节的解释完全由访问它的指令和软件约定决定。

抽象。软件得到一片可变的、按字节寻址的工作空间。当前抽象不包含虚拟地址、访问权限和所有者；任何能够发出物理写请求的代码都可能改写其中任意位置。

1.3.3 四个整数在内存中究竟是什么

输入数组从 `0x00102000` 开始，每个元素占 4 字节。按低有效字节在前的次序，整数 7 存成 `07 00 00 00`，整数 -2 的 32 位补码为 `fe ff ff ff`。四个元素的实际布局如下：

地址范围	字节内容	按 32 位有符号整数解释
<code>0x00102000--03</code>	<code>07 00 00 00</code>	7
<code>0x00102004--07</code>	<code>fe ff ff ff</code>	-2
<code>0x00102008--0b</code>	<code>0b 00 00 00</code>	11

0x0010200c--0f	05 00 00 00	5
0x00102020--23	装入时清零	任务完成后应为 15 00 00 00

`LOAD r5,[r0+0]` 并不是“读取数组元素”这一高级动作。处理器先计算 `r0+0` 得到物理地址，再请求从该地址开始的 4 个字节，最后按规定字节序把它们组合成 32 位值写入 `r5`。数组是连续元素的约定，装入指令只看到地址和宽度。

1.3.4 存储容量与地址范围不是同一个概念

本样机的 RAM 容量为 1MiB，需要 2^{20} 个字节位置。处理器却能表达 2^{32} 个不同的物理地址。没有必要让每个地址都对应 RAM；地址还要留给启动存储和设备。地址是请求中的编号，容量是某个器件实际实现的单元数量。二者通过后面引入的地址互连建立对应关系。

到这里，机器已经有了保存启动字节和运行字节的器件，但处理器还没有连接到它们。下一步必须说明一次读写请求携带什么，以及多个器件如何保证只有一个作出响应。

1.4 一个地址怎样在 RAM 中选中确切的字节

前文把 RAM 画成一个方框，并规定它占用一段物理地址。这个方框还需要展开到足以解释“为什么地址加一会得到下一个字节”。本节不重复存储电路的门级实现，只说明控制一次访问所必需的内部结构。

1.4.1 容量来自地址数量与每个位置的宽度

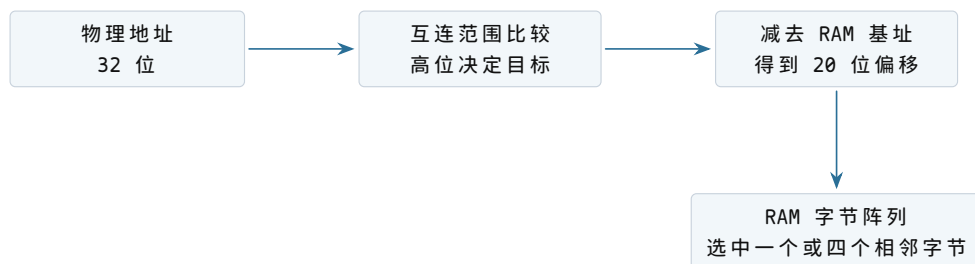
本章 RAM 范围为 `0x00100000--0x001ffffff`，共

$$0x001ffffff - 0x00100000 + 1 = 0x00100000$$

个字节，即 2^{20} 字节。RAM 内部不需要保存完整的高位地址。互连已经证明请求落在这个范围后，向 RAM 提供范围内偏移：

$$\text{offset} = \text{physical_address} - 0x00100000.$$

偏移的低 20 位足以选择 2^{20} 个字节位置。



图示：完整物理地址用于整机选择目标；目标内部只需使用足够区分自身位置的偏移位。

如果直接把 2^{20} 个字节画成一条线，硬件结构会显得不现实。实际阵列通常分成行、列和若干存储体。对本章的软件契约而言，这些组织方式可以不同，只要同一地址总返回同一组已提交字节，并遵守规定的响应次序。

1.4.2 为何一次四字节读取仍以字节地址命名

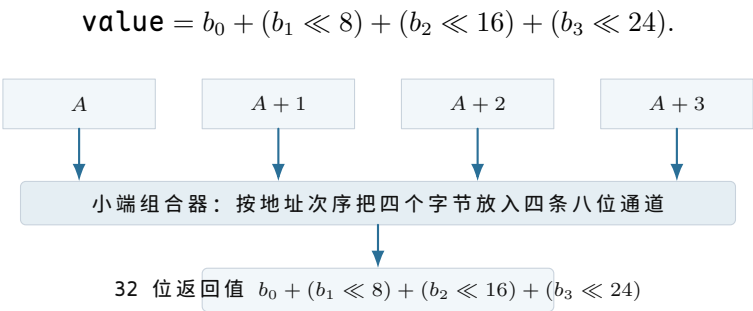
处理器的地址单位是字节。`LOAD` 宽度为四时，地址 A 表示读取 $A, A+1, A+2, A+3$ 四个相邻字节。地址没有自动除以四；只是 RAM 可以把低两位用来选择一个四字节组中的起点。

地址低两位	四字节访问是否对齐	本章处理方式
00	是	读取或写入同一个四字节组
01	否	返回 <code>ALIGNMENT</code> ，不改变任何字节
10	否	返回 <code>ALIGNMENT</code> ，不改变任何字节
11	否	返回 <code>ALIGNMENT</code> ，不改变任何字节

禁止未对齐访问不是数学必然。某些处理器会拆成两次阵列读取，再在内部拼接。本章选择拒绝，是为了让每条四字节事务只涉及一个对齐组，错误时也容易保证“全写或全不写”。程序因此必须让整数地址和 `SP` 都是四的倍数。

1.4.3 四条字节通道怎样组成一个 32 位值

可以把数据端口理解为四条八位通道。对齐地址 A 的低两位为零；第零通道连接 A ，第一通道连接 $A+1$ ，依此类推。读取时，小端组合器产生：



图示：字节顺序是处理器与存储接口之间的解释规则。RAM 单元只保存八个位，并不保存“最低有效字节”这个语义标签。

1.4.4 写入时为什么需要字节使能

写请求除了 32 位数据，还带四位 `byte_enable`。四字节 `STORE` 使用 `1111`；若以后增加单字节存储，地址低位和字节使能可只选择一条通道。例如在地址 $A+2$ 写入 `0x7f`，可能使用：

```
aligned_address = A
write_data      = 0x007f0000
byte_enable     = 0100
```

RAM 必须先同时检查范围、对齐规则和字节使能是否合法，再更新被选择的通道。若先更新第零通道、随后才发现第四个字节越界，软件将面对部分写入。本章接口把检查与更新分开，成功边沿一次提交全部被使能字节。

1.4.5 读取端口和写入端口是否能同时工作

当前机器只有一个处理器请求发起者，互连一次只接受一笔未完成事务。RAM 因而使用一个共享端口就足够：某个周期接受读或写，经过固定或可变延迟后给出一次响应。在响应出现以前，请求字段必须由发起者或 RAM 内部锁存。

```
ram_pending:
  valid
  operation
  offset
  width
  write_data
  byte_enable
```

这些字段解释了“RAM 正在忙”究竟保存什么。若只保存地址而忘记操作方向，响应时就无法判断应返回数据还是提交写入；若不保存字节使能，等待期间输入线变化会改变原请求含义。

内部阶段	已锁存的状态	对外接口状态
IDLE	无待处理请求	req_ready=1
CHECK	请求全部字段	暂不接受第二笔请求
READ	偏移与宽度	阵列选择相应字节
WRITE	偏移、数据、字节使能	成功边沿提交所选字节
RESPOND	数据或错误码	resp_valid=1

阶段名只是本章的一种实现。软件不能读取这些名称，只能观察到请求是否被接受、响应何时有效以及成功响应对应什么效果。把内部状态列出，是为了证明接口承诺有足够的物理状态可以实现，而不是把特定状态机强加给所有 RAM。

1.4.6 启动存储与 RAM 为什么不能合成同一种可写阵列

如果复位入口也位于普通可写 RAM，操作者准备任务时的一次错址就可能覆盖下一次启动所需的第一条指令。机器随后连检查错误记录的代码都无法取得。将启动程序放在普通运行不能写的存储中，保留了一个比任务内容更稳定的起点。

两种存储仍可共享读取请求格式：

性质	启动存储	RAM
复位后内容	制造或维护阶段已确定	本章不规定
普通读取	支持	支持
普通写入	返回 READ_ONLY	合法范围内提交

本章用途	检查、交接、返回入口	任务代码、数据、栈与工作区
------	------------	---------------

因此“都能返回字节”不等于“具有相同生命周期”。启动存储给出稳定起点，RAM 给出运行期可变状态；地址互连把同一种请求送到承担不同责任的目标。

1.4.7 物理地址表仍然不是内存保护表

范围比较只回答地址属于哪个器件。任务向 0x001ff000 写入时，互连会正确选择 RAM，RAM 也会正确更新准备记录；两个器件都履行了当前契约，却破坏了下一次启动依据。要拒绝这笔写入，判断还需要知道“当前请求者身份”和“该范围允许何种访问”。第一章尚未建立这两项输入，所以不能从地址表推断保护已经存在。

1.4.8 复位为什么不顺便清空全部 RAM

处理器复位只需设置少量控制寄存器，几个边沿内即可完成。RAM 有 2^{20} 个字节；若要求复位信号直接把每个存储单元同时改成零，就要为整个阵列增加清零通路，并规定清零期间普通访问如何处理。另一种实现是由启动程序逐地址写零，但这需要发出 2^{18} 笔四字节写，复位释放到首次检查之间会出现很长的执行阶段。

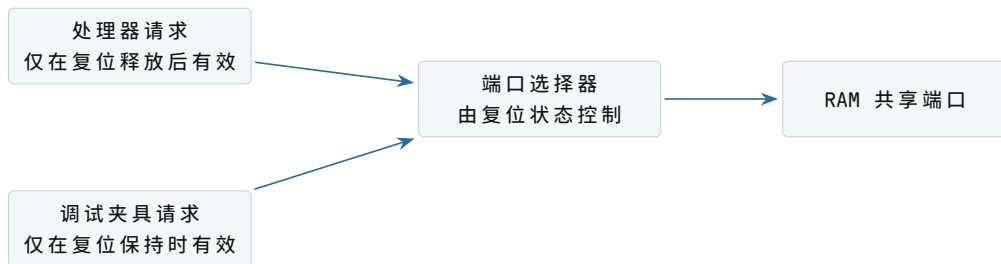
本章没有任何需求要求全部 RAM 为零。代码区、输入区、结果槽、返回栈字和准备记录都会由操作者明确写入；启动程序工作区只在首次写入后读取。其余字节保持未规定更简单，也迫使程序不依赖自己尚未建立的初值。

可选策略	获得的性质	付出的代价或仍有的限制
硬件全阵列清零	复位后每个 RAM 字节为零	阵列增加清零结构，复位时序更复杂
启动程序逐字清零	无需阵列专用清零线	启动时间随 RAM 容量增长
只初始化将要使用的区域	启动工作与本次需求成比例	未声明区域必须始终视为未规定

第三种策略形成一条重要纪律：任何代码在读取某个可写字节前，都要能指出本次运行中哪一步先写入了它。若回答只能追溯到“机器通常会给零”，该读取就没有建立在本章契约上。

1.4.9 调试夹具怎样写 RAM 而不与处理器争用端口

人工准备要求外部夹具能够访问 RAM，而正常运行时 RAM 端口属于处理器。两者若同时驱动地址线，电气上会冲突，协议上也无法判断哪笔请求先发生。本章利用复位状态设置一个简单选择器：



图示：选择器不是运行中的第二个请求发起者。复位状态保证同一时刻只有一侧能够产生有效请求，因此第一章仍是单发起者机器。

夹具写入期间，处理器被复位阻止，不能观察半成品；复位释放以后，选择器固定选择处理器，夹具即使改变自己的地址输入也不会触及 RAM。操作者若要再次修改内容，必须重新保持复位，先让处理器停止普通事务，再取得端口。

复位状态	处理器端口	夹具端口	RAM 接受者
保持有效	不发普通请求	可读写	夹具
正在释放的规定边沿	仍不接受新请求	结束最后一笔请求	无；完成所有权转换
已经释放	可发普通请求	被隔离	处理器

中间一行防止夹具最后一笔写尚未响应，处理器就开始第一笔取指。复位控制器只有在夹具报告空闲后才允许释放；因此 `prepared=1` 的写响应先于第一条启动指令。这个硬件次序与前文的人工提交顺序共同保证，启动程序不会读到正在变化的准备记录。

1.5 把八条助记符落实为内存中的三十二个字节

汇编形式便于人阅读，但处理器取回的是字节。若不把两者对应起来，“操作者已经把代码写入 RAM”仍然缺少可核对对象。下面为本章用到的操作码规定确切数值：

操作码	指令	四个字节的解释
0x10	MOVI	操作码，目标寄存器，16 位有符号立即数低字节、高字节
0x11	ADD	操作码，目标兼第一源寄存器，第二源寄存器，保留字节
0x12	ADDI	操作码，目标寄存器，16 位有符号立即数低字节、高字节
0x20	LOAD	操作码，目标寄存器，基址寄存器，8 位有符号偏移
0x21	STORE	操作码，源寄存器，基址寄存器，8 位有符号偏移
0x31	BNEZ	操作码，测试寄存器，16 位相对指令位移低字节、高字节

0x40	CALL	操作码，24 位相对字节位移
0x41	RET	操作码，三个必须为零的保留字节

寄存器编号直接使用 0x00--0x07 表示 r0--r7。16 位字段也采用低有效字节在前的顺序。分支位移以“从顺序后继开始相差多少条 4 字节指令”计数，因此循环分支从 0x00100014 的顺序后继 0x00100018 回到 0x00100004，相差 -5 条指令。16 位补码 -5 为 0xfffb，在内存中写成 fb ff。

1.5.1 任务代码的地址、字节和含义逐行对应

起始地址	四个实际字节	译码结果
0x00100000	10 04 00 00	MOVI r4,0
0x00100004	20 05 00 00	LOAD r5,[r0+0]
0x00100008	11 04 05 00	ADD r4,r4,r5
0x0010000c	12 00 04 00	ADDI r0,4
0x00100010	12 01 ff ff	ADDI r1,-1
0x00100014	31 01 fb ff	BNEZ r1,-5
0x00100018	21 04 02 00	STORE r4,[r2+0]
0x0010001c	41 00 00 00	RET

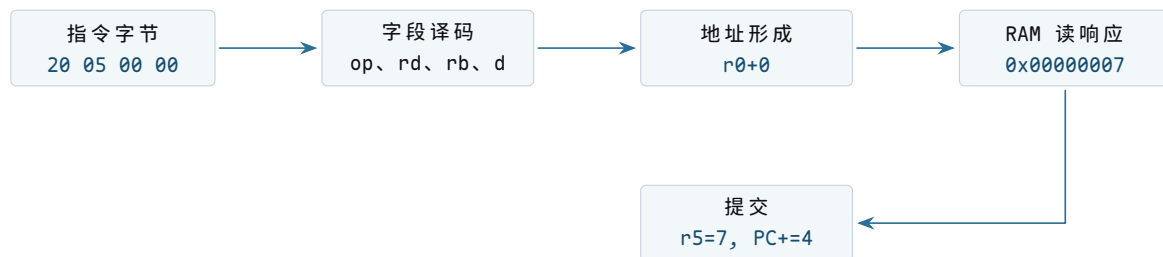
这张表让三个容易混淆的地址分开了。指令地址是 PC 的值；装入和存储指令形成的数据地址来自寄存器与偏移；编码中的寄存器号只是选择处理器内部状态，不是内存地址。例如地址 0x00100004 保存的第二个字节 05 表示目标寄存器 r5，并不表示要访问物理地址 5。

1.5.2 完整译码一条装入指令

当 PC 为 0x00100004 时，处理器取回 20 05 00 00。译码器按固定字段作出以下决定：

- 1. 第一个字节 0x20 选择 LOAD 语义；
- 2. 第二个字节 0x05 选择写回目标 r5；
- 3. 第三个字节 0x00 选择基址 r0；
- 4. 第四个字节 0x00 符号扩展为偏移 0；
- 5. 读取 r0=0x00102000，形成有效地址 0x00102000；
- 6. 发起 4 字节读事务；
- 7. 响应成功后把返回值写入 r5，PC 增加 4。

其中第 5 步只产生地址，第 6 步才与外部存储交互，第 7 步才提交寄存器。若事务返回错误，r5 必须保持旧值，普通顺序 PC 也不能提交。否则软件会看到一条“既失败又部分完成”的装入指令。



图示：译码、地址形成、外部读取和状态提交是连续但不同的阶段。任何阶段失败，都不能假装后继阶段已经发生。

1.5.3 负数为什么没有负号字节

-2 的 32 位补码由 $2^{32} - 2$ 得到，即 `0xfffffffffe`。低有效字节先存，所以 RAM 中依次是 `fe ff ff ff`。装入指令不附带“有符号”标签；它把 32 个位原样放入 `r5`。加法器对补码位串执行模 2^{32} 加法，得到与有符号加法一致的低 32 位。

第一轮后 `r4=0x00000007`。第二轮执行加法：

$$0x00000007 + 0xfffffffffe = 0x00000005 \pmod{2^{32}}.$$

最高位之外的进位被丢弃，寄存器得到 5。只有当上层要判断溢出或比较大小时，才需要区分有符号解释与无符号解释。本次输入之和未超出 32 位有符号范围。

1.5.4 对齐要求来自一次取回的组织方式

教学处理器要求 4 字节指令地址和 4 字节数据地址均为 4 的倍数。因此 `0x00100004` 和 `0x0010200c` 合法，而 `0x00102002` 上的 4 字节读取返回 `ALIGNMENT`。这个约束简化了取指和 RAM 字节通道的组合。

对齐不是“地址看起来整齐”的排版规则。若处理器允许非对齐读取，它可能要发出两笔事务并拼接结果；若中间一笔失败，还要规定整条指令是否提交。不同体系结构可以选择不同接口，但软件必须遵守当前机器的明确契约。

1.5.5 保留字段为什么也必须检查

`RET` 的后三个字节目前没有语义，规范要求它们为零。译码器若忽略任意值，未来给这些字段增加新含义时，旧映像可能被解释成另一种指令。要求发送端置零、处理器拒绝非零保留字段，可以使格式扩展保持明确。

复算点。若循环分支的两个位移字节误写为 `fc ff`，符号扩展结果为 -4 。顺序后继 `0x00100018` 加上 -4×4 得到 `0x00100008`，循环会跳过 `LOAD`，反复累加旧 `r5`。校验和能检测传输改变；若映像本来就这样编码，只有执行语义检查或测试才能发现逻辑错误。

1.5.6 同一片 RAM 为什么可以同时放指令、数据和栈

RAM 只根据地址返回字节。处理器在取指阶段把返回的四个字节送到指令译码器；装入阶段把返回字节送到通用寄存器；`RET` 则把返回字节送到 PC。目标位置没有自带类型，访问路径决定解释方式。

访问时的 PC/指令	读取的 RAM 地址	返回字节的解释
PC 取指	0x00100008	ADD r4,r5 的编码
LOAD	0x00102004	32 位输入整数 -2
RET	0x001efffc	新 PC 0x00000300

这也意味着错误地址可能改变解释。若 PC 被破坏为 0x00102000，处理器会尝试把 07 00 00 00 当作指令，而不会得到“这是数组”的提示。若存储指令写到代码区域，后续取指会看到修改后的字节。当前机器没有执行权限和只读页，所有保护都只是合作约定。

1.6 一个地址怎样到达唯一器件

若把处理器的地址线同时接到启动存储和 RAM，而不给出选择规则，两个器件可能同时驱动返回数据，处理器也无法判断结果来自哪里。机器需要一个位于请求发起者和目标器件之间的地址互连。它读取地址的高位，与预先接好的范围比较，只允许匹配的目标接收本次请求。

1.6.1 一次事务包含哪些信息

处理器访问外部状态时发出一份请求。教学互连中的请求至少含有以下字段：

request:	
address	32-bit physical address
operation	READ or WRITE
width	1, 2, or 4 bytes
write_data	valid only for WRITE
byte_enable	which byte lanes may change

目标完成后返回：

response:	
status	OK, UNMAPPED, ALIGNMENT, or READ_ONLY
read_data	valid only when a READ completes with OK

请求和响应可能由并行导线、片上网络分组或多周期握手承载。本书把从请求被接受到响应返回的这段交互称为总线事务（bus transaction）。这里的“总线”描述软件可观察的传送约定，不要求真实电路采用某个特定工业协议。

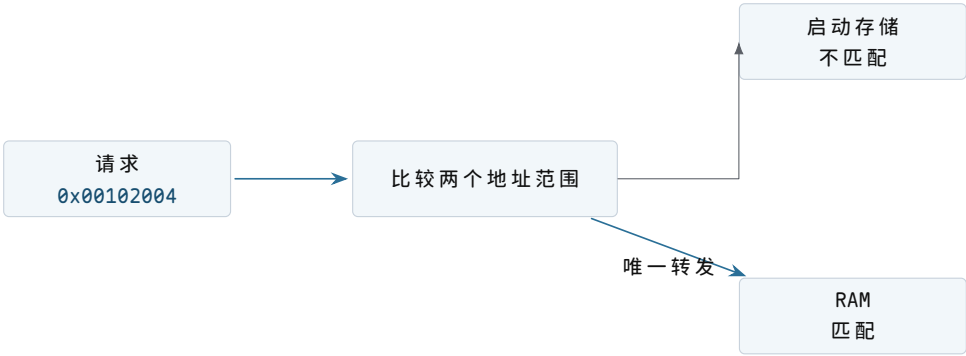
宽度字段不能省略。同一地址可用于读取一个字节或四个字节，目标必须知道哪些字节参与本次操作。写请求还需要字节使能，避免更新相邻位置。读请求中的写数据没有意义，写请求返回的读数据也没有意义；明确这些有效条件可以防止把任意电平误当成软件结果。

1.6.2 地址互连怎样进行选择

本样机采用一张固定物理地址表：

物理地址范围	被选中的目标	当前用途
0x00000000--0x0000ffff	启动存储	启动程序与复位入口
0x00100000--0x001fffff	RAM	人工装入的程序、数据、栈与结果
其余地址	无目标	返回 UNMAPPED

例如地址 0x00102004 落在 RAM 范围内。互连从 RAM 起始地址中减去 0x00100000，得到器件内部偏移 0x00002004，再把请求转发给 RAM。启动存储在这次事务中保持不响应。若地址为 0x20000000，没有范围匹配，互连直接返回 UNMAPPED，不能让处理器无限等待。



图示：互连的关键保证不是“所有器件都连在一起”，而是一个合法地址在一次事务中至多选择一个目标。灰色连线表示物理上可达但本次未被选中。

1.6.3 地址互连的三层含义

硬件模型。互连保存或固化一组地址范围，接收请求字段，产生目标选择信号，再把唯一目标的响应送回发起者。没有匹配或出现重叠匹配时，它产生错误响应。

软件模型。程序看到统一的物理地址空间。相同的 LOAD/STORE 指令访问不同范围时会到达不同器件；程序必须遵守每个范围允许的宽度、方向和对齐规则。

抽象。软件可以用一个物理地址唯一指出当前机器中的一个可寻址位置。这个能力不包含虚拟内存、名称、访问权限或并发仲裁；它只回答“这次请求交给谁”。

1.6.4 一次 RAM 读取的完整时序

以第一轮循环读取整数 7 为例。执行 LOAD r5,[r0+0] 时，r0=0x00102000。处理器先形成地址，再等待目标完成：

- 1. 处理器计算有效地址 0x00102000+0；
- 2. 它发出 4 字节读请求，并在请求未完成时保留该条指令的上下文；
- 3. 互连选择 RAM，把地址换算为器件内部偏移 0x00002000；

- 4. RAM 读取连续四个字节 07 00 00 00;
- 5. RAM 返回 OK 和读数据，互连把响应送回处理器;
- 6. 处理器把组合后的 32 位值 0x00000007 写入 r5;
- 7. 只有写回发生后，PC 才按该指令的完成规则进入下一位置。

若 RAM 需要多个时钟才能响应，处理器不能提前把 r5 改成未知值，也不能把这条装入当作已经完成。教学处理器在等待期间暂停该指令的提交。更复杂处理器可以执行其他独立工作，但最终仍必须使软件观察到与规定顺序一致的结果；本章不需要展开内部优化。

1.7 时钟与复位怎样把随机上电状态收束到第一条取指

存储和互连已经接好，处理器仍需要一个共同的状态更新节拍，并且需要在上电后得到确定的起点。电源刚稳定时，各寄存器中的位不应被假定为零。复位电路必须主动把少量关键状态设置为规定值，其中最重要的是 PC 和控制状态。

1.7.1 时钟不是软件计时器

时钟源周期性地产生边沿，时序电路在边沿处提交新状态。它使“先读取旧寄存器、再写入新寄存器”的顺序有了物理边界，但普通软件此时没有可读的时钟计数器。知道电路有时钟，不等于能够请求“十毫秒后通知我”。可编程计时器要在后续确有需求时作为新器件加入。

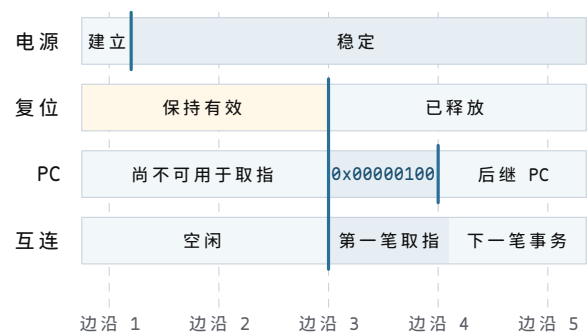
硬件模型。时钟源输出周期边沿；复位电路在电源和时钟尚未稳定时保持复位有效，稳定后按规定边沿释放。处理器在复位有效期间不提交普通指令状态。

软件模型。复位释放后，PC 被设为 0x00000100，SP 和通用寄存器的值除非另有规定，否则视为未定义。固件不能在初始化前读取这些寄存器并假定它们为零。

抽象。启动代码可以依赖每次复位都从同一入口开始，并且已提交的指令状态在时钟边界上遵守体系结构规则。它不能依赖真实频率恒定，也不能由边沿数推断日期或调度时刻。

1.7.2 从电源稳定到第一条指令完成

下面的时间线把“开机”拆成可观察事件。纵向虚线表示时钟边沿，横向各行表示不同部件：



图示：电源稳定并不立即开始取指。复位先保持若干边沿，释放后才把规定的 PC 值用于第一笔事务；第一条指令完成后，PC 才进入后继值。

第一条指令位于启动存储，而不是 RAM 中的任务入口。原因很直接：RAM 此时尚未装入任务，输入参数和栈也未建立。复位 PC 若直接指向 `0x00100000`，处理器会把上电后的不确定 RAM 字节当作指令，机器行为无法预测。

1.8 处理器怎样把指令字节变成状态变化

处理器是当前唯一能够主动发起取指和数据事务的部件。它内部至少要保留 PC、SP、通用寄存器、标志和当前指令的执行控制状态。这里不展开门级数据通路，只说明后续软件真正能依赖的边界。

硬件模型。处理器根据 PC 发出取指请求，译码返回的操作码与字段，读取旧寄存器，执行算术或形成访存地址，并在指令完成时提交目标寄存器、内存效果和下一 PC。访存错误会阻止普通写回，并把失败位置与原因留在处理器内部的停止记录中。

软件模型。软件使用本章定义的寄存器和指令。除分支、调用、返回和显式跳转外，PC 每完成一条固定长指令增加 4。调用约定规定哪些寄存器传参、SP 如何移动以及返回值放在哪里，这些规则不是电路自动理解的类型系统。

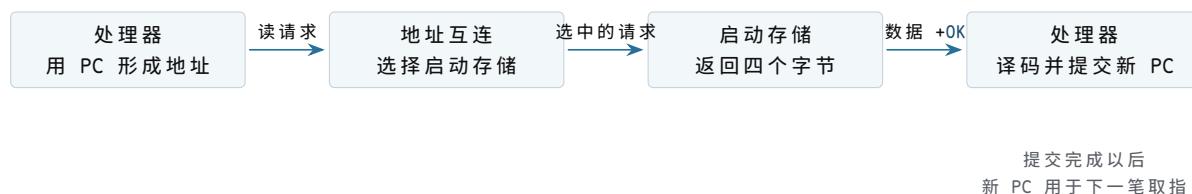
抽象。软件得到一条按指令集规则推进的执行流：每条完成的指令都有确定的前置状态和后置状态。当前处理器不隔离任务，不保存多个执行现场，也不会因为另一个任务等待而主动停止当前任务。

1.8.1 第一条取指逐字段走读

复位释放后，PC 为 `0x00000100`。处理器发出的请求为：

```
address      = 0x00000100
operation    = READ
width       = 4
write_data   = ignored
byte_enable  = 1111
```

互连发现地址属于启动存储。这四个字节就是启动程序的第一条普通指令；本章样机把它规定为读取人工准备记录中提交标志的 `LOAD`。启动存储返回数据及 `OK` 后，处理器完成译码并等待相应的数据读取；只有该装入指令成功，目标寄存器和顺序后继 `0x00000104` 才一同提交。这里不需要一条尚未解释的特殊“启动跳转”：复位 PC 本身就指向启动程序入口。



图示：取指不是处理器内部凭空取得指令。它是一笔带地址的读取事务；只有响应成功并完成译码后，处理器才提交新的 PC。

1.8.2 顺序 PC、分支 PC 与返回 PC

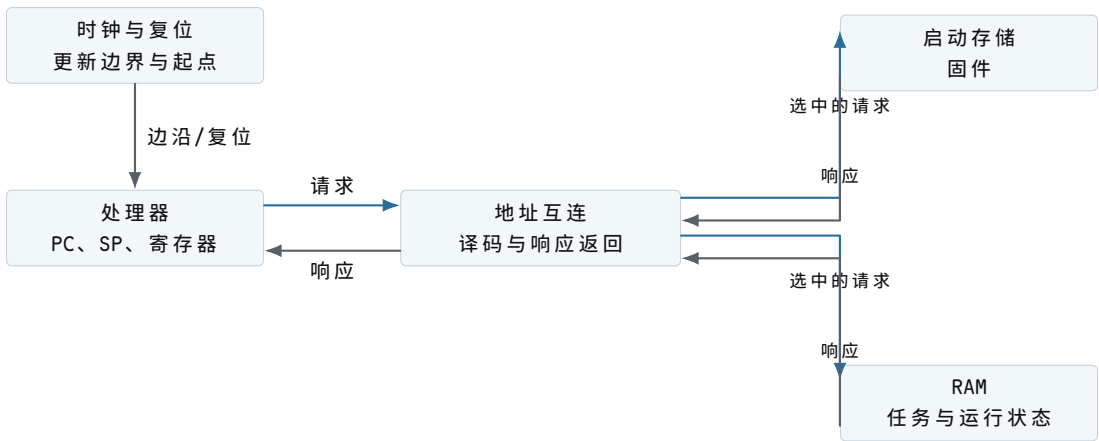
求和任务中出现三类下一 PC：

- 1. 普通指令完成后，下一 PC 为当前 PC 加 4；
- 2. BNEZ 根据 r1 的值，在循环入口和顺序后继之间选择；
- 3. RET 从 SP 指向的内存中读取返回地址，再把该值写入 PC。

三者最终都只是决定 PC 的新值，但所依赖的状态不同。普通前进只依赖当前 PC；条件分支还依赖寄存器或标志；返回还要成功完成一次栈内存读取。把它们都写成“跳到下一条”会隐藏错误来源。

1.8.3 把已经解释的连接重新合在一起

章首先给出的整体图用于确定方位。现在，每个方框和箭头都已有明确职责，可以用同一幅结构复核一次请求怎样往返。机器仍然只包含时钟与复位、处理器、地址互连、启动存储和 RAM。



图示：此时机器已经能从固定启动程序取指并访问 RAM。地址互连只负责按地址送达，不负责判断 RAM 中的字节是否构成一份完整程序。

这幅图说明了当前能力的上限。复位能找到启动程序，启动程序能读写 RAM；但本章仍需由操作人员在复位期间预置求和程序。机器运行后没有接收新程序的通道，也没有谁替操作人员检查一串外部字节是否完整。

1.9 连接不仅传位，还必须约定何时有效

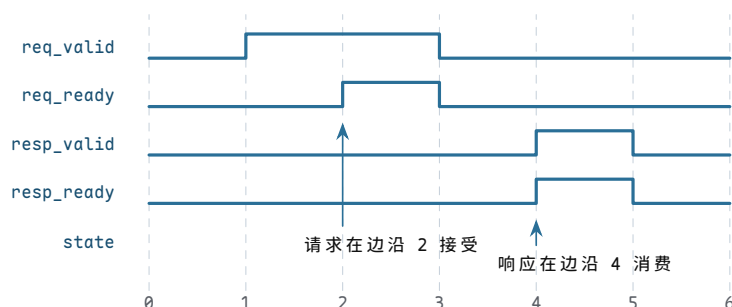
地址线、数据线和方向线即使全部接通，发起者仍要知道目标何时看到了请求，目标也要知道处理器何时接收了响应。若启动存储一周周期返回而 RAM 三周期返回，固定等待一个周期会让处理器过早采样 RAM。教学互连因此使用请求/响应握手。

1.9.1 请求通道的四个时序事实

处理器把请求字段放稳后置 req_valid=1。互连只有在能够接收时置 req_ready=1。某个时钟边沿同时看到二者为 1，才表示请求已被接受。在此之前，处理器必须保持地址、方向、宽度和写数据不变。

信号	驱动者	为 1 时的含义
req_valid	处理器	请求字段当前有效
req_ready	互连	本边沿可以接受这份请求
resp_valid	互连	响应状态和读数据当前有效
resp_ready	处理器	本边沿可以消费响应

响应通道采用同样规则。`resp_valid` 与 `resp_ready` 在同一边沿为 1，响应才被消费。目标可以在请求接受后的任意较晚周期给出响应，处理器无需预先知道固定延迟。



图示：处理器从周期 1 起保持请求，直到边沿 2 完成握手；目标稍后给出响应，并在边沿 4 被消费。有效信号单独为 1 都不表示传送已经发生。

1.9.2 为什么有效和就绪不能合成一根线

若只有一根“完成”线，无法区分“发起者现在提供有效请求”和“接收者现在有容量接受”。当互连忙于前一笔事务时，处理器必须保留请求；当处理器暂时不能接收响应时，互连也必须保留响应。两端各自表达有效与就绪，才能在不同速度部件之间建立无歧义边界。

当前处理器每次只允许一笔未完成事务，因此不需要请求编号。若以后允许多笔并发且响应可能乱序，就必须增加事务标识，并保存每笔请求对应的目标寄存器和 PC。那是性能优化引出的新状态，不属于最小机器。

1.9.3 写请求中的数据何时可以撤掉

处理器从置 `req_valid` 开始，就要同时保持 `write_data` 和 `byte_enable`。只有请求握手边沿之后，才可撤掉这些字段。请求被互连接受并不等于目标写入已经完成；处理器还要等待响应握手，才能提交存储指令的顺序后继。

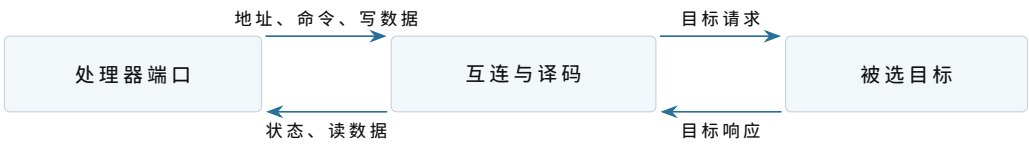
这形成两个边界：

1. 请求接受：互连已取得一份稳定请求，处理器可以释放请求通道；
2. 操作完成：目标已经执行或拒绝操作，响应被处理器消费。

在简单零等待器件上，两者可能落在相邻甚至同一周期，但软件模型仍不应把它们定义成同一个概念。

1.9.4 物理连接中的方向与扇出

教学图用箭头表达信息方向。地址、操作类型和写数据从处理器流向互连；读数据和响应状态从目标流回处理器；目标选择信号从互连流向某个器件。时钟可以扇出到多个时序部件，复位也可送达需要确定初态的控制状态。



图示：双向交互由两组单向信息构成。把一条线画成无说明的双向箭头，会隐藏谁必须保持字段以及哪个边沿完成传送。

1.9.5 为什么地址不能由所有目标共同解释

目标器件只需识别自己的内部偏移，不必都实现完整 32 位范围比较。互连先完成全局选择，再把低位偏移交给目标。这减少重复逻辑，也把地址冲突集中在一处检查。

例如 RAM 收到内部偏移 0x00002004，无需知道全局地址曾是 0x00102004。但软件错误报告应保留原始全局地址，因为程序使用的是物理地址表，不是器件内部偏移。

1.9.6 复位连接不应清空所有 RAM

处理器复位要确定 PC 和控制状态，互连复位要清除未完成事务。RAM 内容是否在复位时清零是另一件事。多数 RAM 不会因处理器复位自动擦除，启动程序必须检查或初始化依赖初值的区域。

这解释了为什么启动程序不能仅凭复位后看到的旧 RAM 字节就判断人工准备已经完成。复位只重建控制起点，不证明操作人员写完了全部区域。下一节会增加一份很小的启动说明，让启动程序在移交处理器前检查本次人工装入是否完整。

1.10 不同指令怎样越过各自的提交边界

“处理器执行一条指令”不能只画成一个方框。算术指令、装入、存储、分支和返回依赖的外部状态不同，失败时允许发生的效果也不同。教学处理器按指令类别规定提交规则。

指令类别	完成前读取	候选效果	提交条件
寄存器算术	源寄存器、立即数	目标寄存器与标志的新值	当前指令无译码错误
装入	基址寄存器、RAM 响应	目标寄存器的新值	读响应为 OK
存储	基址、源寄存器	RAM 中若干字节的新值	写响应为 OK

条件分支	测试寄存器、位移	顺序 PC 或目标 PC	目标满足对齐约束
RET	SP、栈读响应	新 PC 与增加后的 SP	栈读成功且返回 PC 合法

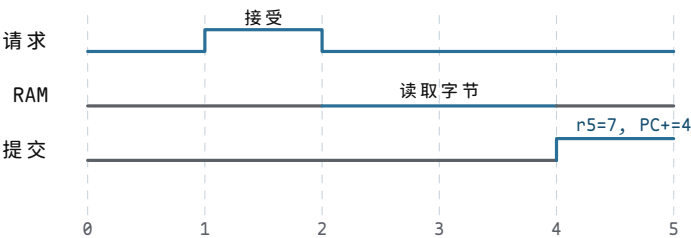
算术指令的结果在写回前只是处理器内部候选值。装入多了一笔外部读，不能在响应之前写目标寄存器。存储的效果发生在 RAM 一侧；处理器收到成功响应后才能继续，但不能先改 RAM、随后又向软件报告该指令未执行。教学模型要求目标对每笔写要么完整接受指定字节，要么返回错误且不改变任何字节。

1.10.1 装入等待期间保存什么

假设 RAM 需要三个周期完成读取。处理器至少要保留下列临时事实：

```
pending_load:
  instruction_pc = 0x00100004
  destination   = r5
  address       = 0x00102000
  width         = 4
  next_pc       = 0x00100008
```

它们不是软件可直接读取的通用寄存器，而是处理器内部执行状态。若等待时把 PC 提前提交到 0x00100008，又允许下一条指令读取旧 r5，程序会观察到错误顺序。最小处理器因此在响应返回前不开始下一条指令。



图示：请求被接受不等于装入已提交。RAM 完成读取后，处理器才同时更新目标寄存器和顺序 PC。

1.10.2 存储的提交点位于目标响应

STORE r4,[r2+0] 的处理器请求包含地址、宽度和数据。RAM 检查地址及字节使能，在接受写入的边界更新四个字节并返回 OK。若地址未对齐，它返回 ALIGNMENT 且四个字节全部保持旧值。

对软件而言，存储指令完成后，同一处理器随后读取该地址必须看到新值。当前机器只有一个请求发起者，没有缓存和其他处理器，所以不需要讨论多核可见顺序。这个简化条件应明确写出，不能把单核结论直接推广到并发机器。

1.10.3 分支先计算两个候选，再提交一个

BNEZ 的顺序后继为 PC+4，目标为

$$PC+4 + 4 \times \text{signextend}(\text{disp16}).$$

处理器读取测试寄存器，在两个候选中选择一个写入 PC。未被选择的地址不会产生取指事务。对循环分支，disp16=0xffffb 表示 -5，目标计算为

$$0x00100018 - 20 = 0x00100004.$$

若位移计算溢出 32 位地址或目标未对齐，教学处理器不提交普通 PC，而是记录失败原因并停止。真实体系结构可能规定模地址运算或不同的受控转移行为；本书此处只依赖“错误不会静默成为另一条合法顺序指令”这一教学契约。

1.10.4 CALL 与 RET 为什么同时改变内存和 PC

CALL target 需要保留顺序后继，以便子程序结束后继续。教学处理器按以下顺序定义调用：

- 1. 计算返回地址 PC+4；
- 2. 计算新栈指针 SP-4，检查是否对齐；
- 3. 向 MEM32[SP-4] 写返回地址；
- 4. 写响应成功后提交 SP<-SP-4 和 PC<-target。

若第 3 步失败，SP 和 PC 都保持调用前的体系结构值。RET 进行相反方向的动作：读取 MEM32[SP]，成功后提交 PC<-value 和 SP<-SP+4。调用与返回因此不是纯粹的 PC 跳转，它们还维护一条位于 RAM 的控制流历史。

状态	调用前	CALL 成功后
PC	0x00100100	子程序入口 0x00100200
SP	0x001efffc	0x001efff8
栈字 0x001efff8	不属于有效栈	返回地址 0x00100104

当前求和核心没有调用子程序，但启动协议仍预置一个返回地址，使最外层 RET 能把控制权交回固件。最外层返回与普通函数返回使用相同处理器规则，区别只在栈中字节由谁建立。

1.10.5 错误响应怎样留下可检查的停止状态

如果处理器遇到未定义操作码后仍继续前进，后面的现象会掩盖最初原因；如果它只停住而不记录位置，操作者又无法区分“正在等待”和“已经失败”。因此当前样机给处理器增加四个仅供调试夹具读取的锁存字段：

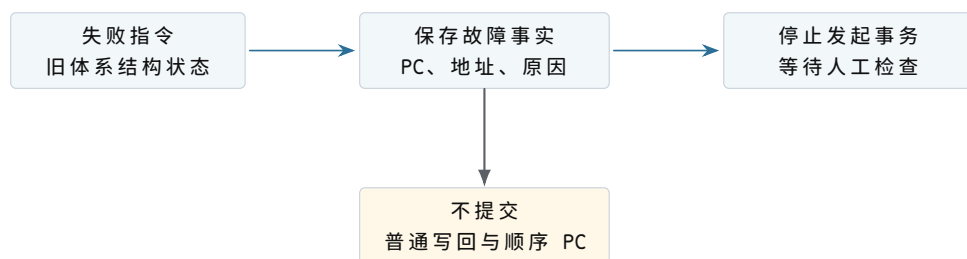
fault_pc	address of the instruction that could not complete
----------	--

```

fault_address  external address, if the instruction issued one
fault_cause    INVALID_OPCODE, UNMAPPED, ALIGNMENT, or READ_ONLY
halted        1 after a fault record has been committed

```

错误发生时，处理器先锁存前三项事实，再把 `halted` 置一。失败指令的普通目标寄存器、顺序 PC 和目标内存效果均不提交；处理器也不再发出新的普通取指请求。操作者可在保持复位前通过调试夹具读取这些字段，修正人工装入的内容，然后重新准备 RAM 并复位。



图示：停止记录只保存足以说明失败位置的最小事实。它不会保存一份可在稍后恢复的完整运行现场，也不会自动选择另一段程序继续执行。

1.10.6 无效操作码与未映射地址属于不同层次

若取回的第一个指令字节不是已定义操作码，错误在处理器译码阶段产生。此时 `fault_cause` 取值为 `INVALID_OPCODE`，没有数据地址。若合法 `LOAD` 形成的地址没有目标，处理器已经完成译码，互连返回 `UNMAPPED`，此时 `fault_address` 有效。区分二者可以判断是代码字节本身无法解释，还是合法指令访问了错误位置。

1.10.7 为什么停止记录还不是受控入口

当前硬件不会因为错误而把 PC 改到另一段程序，也没有保存全部通用寄存器，更没有从失败位置恢复的协议。它做的事情只有“冻结普通推进并留下原因”。后续确实需要机器自动转向一段处理程序时，才会进一步推导入口地址、专用栈、现场保存和返回规则；不能把这里的停止锁存器提前称为完整的异常处理机制。

1.11 处理器内部必须保存哪些运行事实

寄存器表只列出了程序能够直接命名的状态。处理器在一条指令尚未完成时，还要记住取到的指令、等待的事务和候选结果。否则外部响应经过数个周期回来时，它无法知道响应属于哪条指令，也无法在失败时保留旧状态。

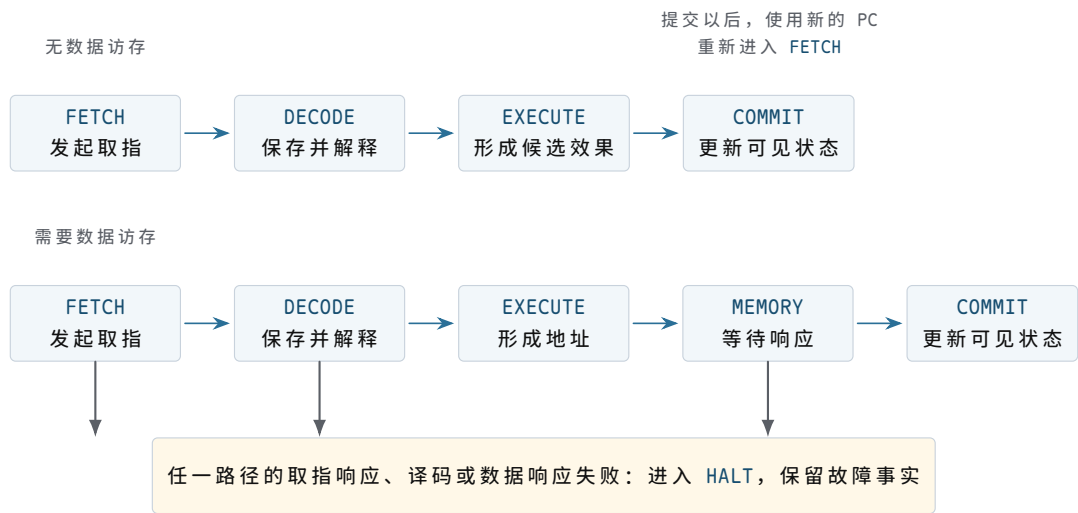
1.11.1 可见状态与暂存状态先分开

类别	具体字段	生命周期
程序可见	PC、SP、 <code>r0--r7</code> 、算术标志	从一条已提交指令延续到下一条

当前指令	<code>instruction_pc</code> 、四个编码字节、译码字段	从取指响应到提交或失败
访存等待	操作、地址、宽度、数据、目标寄存器、后继 PC	从数据请求接受到数据响应
候选效果	新寄存器值、新 PC、新 SP	只在当前指令成功时写入可见状态
停止记录	故障 PC、地址、原因、 <code>halted</code>	从失败提交保持到复位

“切换到下一条指令”准确地说，是清除当前指令与候选字段，保留刚提交的程序可见状态，再用新的 PC 发起取指。它不是把处理器全部清零。

1.11.2 最小顺序处理器怎样分阶段推进



图示：图中一次只有一条指令占据这些阶段。后续若讨论流水线，必须允许多条指令同时处于不同阶段，并重新定义提交顺序；第一章不预设该结构。

1.11.3 `FETCH` 保存当前 PC 的副本

发起取指前，处理器执行：

```
instruction_pc = PC
request.address = PC
request.operation = READ
request.width = 4
```

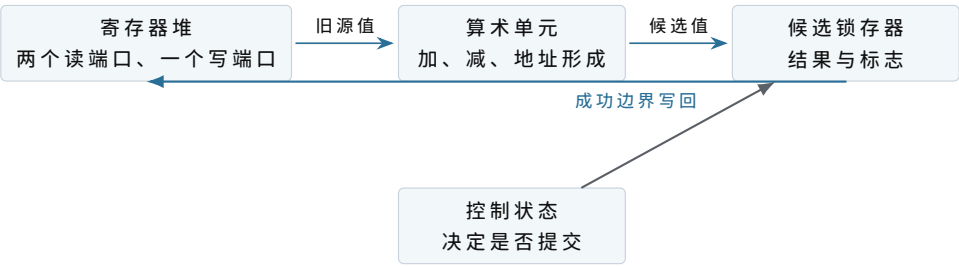
即使后续已有候选顺序 PC，故障报告也使用 `instruction_pc`。取指响应回来后，四个字节进入指令寄存器。只有此时译码器才稳定看到操作码和字段。若启动存储需要两个周期，处理器在等待期间不能用输入端口上暂时出现的其他数值进行译码。

1.11.4 DECODE 读取的是旧寄存器值

以 `ADD r4,r4,r5` 为例，译码字段选择两个读端口：

```
source_a = registers[4]
source_b = registers[5]
destination = 4
```

寄存器堆读出当前已提交的 `r4` 和 `r5`。运算结果先进入 `candidate_value`；在 `COMMIT` 以前，寄存器堆中的 `r4` 不变。这样，同一条指令的两个源操作数都来自统一的前置状态。



图示：候选锁寄存器把计算与提交隔开。发生错误时，控制状态可以丢弃候选值而不改动程序可见寄存器。

1.11.5 不同指令实际使用哪些内部字段

指令	执行期间额外保存	成功时提交
MOVI	目标号、符号扩展后的立即数、顺序 PC	目标寄存器与 PC
ADD	两个旧源值、目标号、候选和	目标寄存器、标志与 PC
LOAD	有效地址、宽度、目标号、顺序 PC	返回值写目标寄存器，更新 PC
STORE	有效地址、宽度、源数据、字节使能	RAM 效果已经成功后更新 PC
BNEZ	测试值、顺序 PC、分支目标	二选一的新 PC
RET	旧 SP、栈地址、候选返回 PC	新 PC 与 <code>SP+4</code>

表中没有“当前任务编号”。第一章的处理器不知道任务身份；同一组字段先执行启动程序，后执行求和程序。软件约定改变了这些位的含义，硬件保存结构没有随之更换。

1.11.6 一次 `LOAD` 的内部状态逐周期变化

假设 RAM 三个周期后响应，且请求在第 2 个边沿被接受：

边沿后	控制阶段	保持的内部事实	程序可见状态
0	FETCH	<code>instruction_pc=00100004</code>	PC 仍为 <code>00100004</code>
1	DECODE	目标 <code>r5</code> 、基址 <code>r0</code>	全部不变
2	MEMORY	地址 <code>00102000</code> 、宽度 4、后继 PC	全部不变
3	MEMORY	同一待处理记录	全部不变
4	MEMORY	收到数据 7，形成候选	全部不变
5	COMMIT	待处理记录即将清除	<code>r5=7, PC=00100008</code>
6	FETCH	新 <code>instruction_pc=00100008</code>	保持边沿 5 的值

若边沿 4 收到 `UNMAPPED`，边沿 5 改为提交停止记录，`r5` 和 PC 保持边沿 0 以前的值。响应延迟只增加中间等待行，不改变成功提交后的状态。

1.11.7 一次 `STORE` 的候选效果为什么不在处理器里

装入的候选值最终写寄存器；存储的目标位于 RAM。处理器能保存地址和数据，却不能仅凭“请求已经送出”认定 RAM 已改变。RAM 的成功响应说明目标侧已经在自己的提交边界接受了全部字节，处理器随后只需更新 PC。

观察位置	请求接受后、响应前	成功响应后
处理器	保存地址、数据、字节使能与后继 PC	清除待处理记录，提交后继 PC
RAM	保存待检查的写请求	四字节已经作为一组更新
程序可见	当前 <code>STORE</code> 尚未完成	后续同地址读取必须看到新值

1.11.8 PC 为何不能在发请求时提前增加

若发起 `LOAD` 时就把 PC 改到下一条，等待中发生故障时会出现两个问题：停止记录不再自然指向失败指令；复位以外的恢复机制将不知道该重试哪一条。保留旧 PC，把 `PC+4` 只放在候选字段中，可以让“当前指令”在整个等待期保持稳定。

某些真实处理器会在内部预测和提前取后续指令，但仍需要一个按程序顺序提交的边界，使软件观察结果等价于规定模型。第一章直接采用顺序实现，不让优化掩盖基本状态关系。

1.11.9 停止状态为何仍需要时钟边沿提交

组合电路检测到错误的瞬间，`fault_cause` 可能随输入变化。处理器在规定边沿同时锁存 PC、地址和原因，再置 `halted=1`。从下一周期开始，控制状态保持 `HALT`，不再发请求。这样操作者读到的四个字段来自同一次失败，而不是不同时刻的混合。

```

if response.error:
    next_fault_pc      = instruction_pc
    next_fault_address = pending.address
    next_fault_cause   = response.error
    next_halted       = 1
    commit no ordinary destination

```

1.11.10 交接时改变的是同一套寄存器，不是复制出第二套

启动程序进入任务时，处理器没有把旧 `r0--r7` 自动保存到隐藏区域。它们被任务入口值直接覆盖。启动程序能够在任务返回后继续，是因为返回入口不依赖交接前那些临时寄存器，长期需要的准备编号已经写在 RAM 工作区。

交接对象	本章实际动作	没有发生的动作
通用寄存器	写成任务参数和清零值	没有保存启动程序完整寄存器副本
SP	从启动栈值改为任务栈顶	没有同时保留两个活动 SP
PC	从交接指令改为任务入口	没有“任务模式”位
RAM	保留两段栈和工作区字节	没有自动禁止任务访问启动区

这一事实为后续多项程序问题留下准确起点：若程序 A 暂停后还要从原位置继续，就必须在覆盖寄存器前把它的 PC、SP 和仍有用的寄存器值保存到某处。第一章没有这种需要，所以也不提前引入保存现场结构。

1.12 机器开始运行以前，RAM 中究竟要有什么

到这里，处理器已经能够取指，启动存储和 RAM 也已经接入同一组地址。可是“接好了 RAM”并不等于“RAM 里已经有程序”。上电后的 RAM 字节没有本章可依赖的初值；即使某种器件偶然保留了上次断电前的电荷，也不能据此判断哪些字节属于这一次运行。

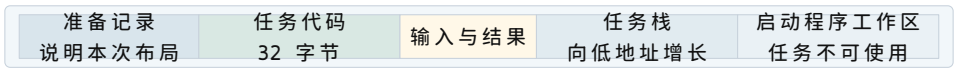
本章暂不增加任何自动输入设备。操作者使用机器外部的调试夹具，在复位保持有效时直接写 RAM。夹具可以选中一个物理地址并写入一个字节，也可以读回一个字节核对。它不属于正在运行的机器，处理器也不能通过指令调用它。这样的准备方式十分笨拙，却能把最基本的问题暴露得很清楚：在处理器取得第一条任务指令之前，必须先建立哪些事实？

1.12.1 只写入指令字节为什么还不够

这次要运行的程序把四个 32 位有符号整数相加。若只把八条静态指令写入 `0x00100000`，程序仍无法确定下列内容：

- 四个输入整数放在何处，按什么字节顺序解释；
- 应当处理几个元素，结果应写到哪个地址；
- 栈可以使用哪一段 RAM，最初的 SP 应取什么值；
- 最后一条 `RET` 应当把 PC 带回哪里；
- 这些区域是否已经完整写入，而不是只写完一半。

前四项决定程序如何运行，最后一项决定机器是否可以开始运行。把它们混成一句“程序已装入”会丢失最重要的边界。为此，我们先把一次运行所需的 RAM 分成五个区域。



图示：图中横向宽度表示逻辑分区，不按真实容量比例绘制。每个区域仍由普通 RAM 字节组成；分区首先是一份准备约定，并不是 RAM 芯片自动提供的保护。

1.12.2 先把一次运行写成可核对的记录

操作者和启动程序需要对同一组地址达成一致。若这些数值只记在操作者的纸上，机器无法核对；若启动程序把数值全部写死，换一项程序就必须重做启动存储。于是我们在 RAM 中保留一小段固定位置，用字段描述本次准备结果。暂且称它为准备记录，地址从 `0x001ff000` 开始。

偏移与字段	本次取值	解释方式	启动前必须证明的条件
<code>+0x00 magic</code>	<code>RUN1</code>	四个固定字节	格式与启动程序约定一致
<code>+0x04 generation</code>	<code>17</code>	无符号整数	与操作者本次准备编号一致
<code>+0x08 code_base</code>	<code>0x00100000</code>	物理首地址	四字节对齐且位于 RAM
<code>+0x0c code_bytes</code>	<code>32</code>	字节数	非零、四的倍数、末地址不回绕
<code>+0x10 data_base</code>	<code>0x00102000</code>	输入首地址	四字节对齐且位于 RAM
<code>+0x14 data_bytes</code>	<code>16</code>	输入字节数	恰好容纳四个 32 位整数
<code>+0x18 result_base</code>	<code>0x00102020</code>	结果首地址	可写、对齐且不覆盖输入
<code>+0x1c stack_low</code>	<code>0x001e0000</code>	栈下界	低于 <code>stack_top</code>
<code>+0x20 stack_top</code>	<code>0x001f0000</code>	栈上界	对齐且不越过 RAM 末端
<code>+0x24 entry</code>	<code>0x00100000</code>	首条任务指令	落在代码区且四字节对齐

+0x28 code_sum	代码校验和	32 位无符号和	与实际 32 个代码字节相符
+0x2c data_sum	数据校验和	32 位无符号和	与实际 16 个输入字节相符
+0x30 prepared	1	提交标志	必须最后写入

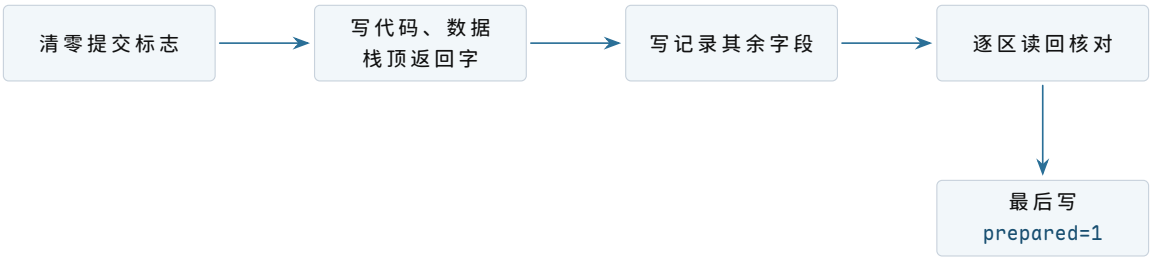
记录没有让程序自动进入 RAM。它只是把“操作者声称自己做完了什么”变成机器可读取的字节。启动程序仍要逐项验证这些声称。尤其不能看见 `prepared=1` 就直接跳转，因为旧 RAM 里可能恰好留着一个一。

1.12.3 为什么提交标志必须最后写

假设操作者先写 `prepared=1`，随后写到第 19 个代码字节时连接松动。启动程序若只看提交标志，就会把后半段旧字节当作新程序。相反，若提交标志最后写入，那么任意中断点都属于下面两种状态之一：

观察时刻	<code>prepared</code>	启动程序可以作出的判断
准备尚未完成	0	不得进入任务，不必猜测哪些区域已写完
全部写入并读回后	1	可以继续核对格式、边界与校验和

这里出现了本书第一次明确的提交顺序：先建立所有被依赖的内容，最后写一个代表“整组内容已经准备完成”的字段。这个字段不保证内容正确，却把“尚未完成”与“可以开始核对”分开了。



图示：顺序由操作者和调试夹具保证。机器运行以后才会读取提交标志，因此不存在处理器与夹具同时修改这些字节的情形。

1.12.4 代数检查为什么要在访问以前完成

判断一个区域是否落在 RAM 中，不能只检查首地址。若代码首地址为 `base`、长度为 `size`，最后一个有效字节是

$$\text{last} = \text{base} + \text{size} - 1.$$

直接用 32 位加法计算这个式子可能回绕。例如 `base=0xffffffff0`、`size=0x40` 会得到一个很小的结果，看起来反而位于低地址。启动程序因此采用不会先发生回绕

的比较：

$$\text{size} > 0, \quad \text{base} \leq \text{RAM_END}, \quad \text{size} - 1 \leq \text{RAM_END} - \text{base}.$$

同一方法用于代码、输入和结果槽。两个半开区间 $[a, a+n)$ 与 $[b, b+m)$ 不重叠，当且仅当

$$a + n \leq b \quad \text{或} \quad b + m \leq a,$$

但端点加法仍需先做防回绕检查。对本次布局，启动程序应证明：

- 代码区 $[0x00100000, 0x00100020)$ 与输入区分离；
- 输入区 $[0x00102000, 0x00102010)$ 不覆盖结果槽；
- 栈区 $[0x001e0000, 0x001f0000)$ 不覆盖准备记录；
- 入口位于代码区，初始 SP 位于栈上界并满足四字节对齐。

这些检查并不创造内存保护。任务开始后仍能形成任意物理地址。它们只证明启动程序自己不会把一个显然自相矛盾的布局当作有效布局。

1.12.5 校验和能证明什么，不能证明什么

操作者把代码区每个字节按无符号数相加，只保留低 32 位，得到 `code_sum`；数据区同理。启动程序重新读取对应范围并复算。若结果不同，至少有一个字节、长度或地址与准备记录不一致。

简单加法校验和可能让两处相反方向的变化互相抵消，所以它不能抵抗蓄意修改，也不能证明程序含义正确。本章使用它只有一个有限目的：让最常见的漏写、错址和连接不稳在跳转前暴露。后续若需要对抗恶意内容，必须引入更强的完整性与身份机制，不能更换一个术语就假定安全性已经获得。

1.13 把具体的任务字节放入具体地址

现在可以给出这项任务的完整静态布局。指令均为四字节定长；表中“作用”是对字节编码的解释，真正存入 RAM 的仍是机器码。

物理地址	指令	成功提交后的作用
0x00100000	MOVI r4,0	把累加器清零，PC 前进四字节
0x00100004	LOAD r5,[r0]	从当前输入地址读一个 32 位整数
0x00100008	ADD r4,r4,r5	把本轮整数加入累加器
0x0010000c	ADDI r0,r0,4	输入指针指向下一个整数
0x00100010	ADDI r1,r1,-1	剩余元素数减一
0x00100014	BNEZ r1,-5	若尚有元素，回到 0x00100004
0x00100018	STORE r4,[r2]	把最终和写入结果槽
0x0010001c	RET	从栈顶读取返回 PC

输入按小端字节序写入。数字 7, -2, 11, 5 的 32 位表示分别是 0x00000007、0xfffffffffe、0x0000000b 和 0x00000005。因此前八个输入字节是：

address	+0	+1	+2	+3
0x00102000	07	00	00	00
0x00102004	fe	ff	ff	ff

读取 0x00102004 的四个字节时，处理器按

$$fe + (ff \ll 8) + (ff \ll 16) + (ff \ll 24) = 0xfffffffffe$$

重组。加法单元只处理 32 位位型；把该位型解释为有符号数时才得到 -2。这一区分解释了为什么 RAM 不需要知道“这里存的是负整数”。

1.13.1 结果槽为何先写成已知的哨兵值

操作者在 0x00102020 写入 0xdeadbeef，而不是预先写零。运行后若仍读到哨兵值，说明结果存储没有成功提交；若预先写零而正确结果也恰好为零，就无法凭该地址区分“程序算出零”和“程序根本没有写”。

观察到的结果槽	可确定的事实	尚不能单独确定的事实
0xdeadbeef	预期结果写入尚未发生	程序未开始、运行中或已失败
0x00000015	某次写入留下十进制 21	是否由预期代码在本次运行写入
其他值	发生过非预期写入或计算	错误来自输入、代码、布局还是硬件

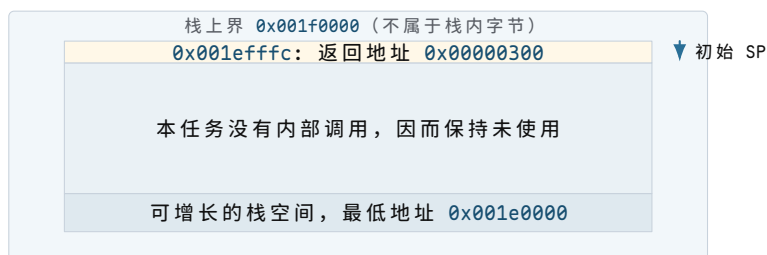
因此还需要准备编号和处理器停止状态共同判断。单个内存值是证据的一部分，不是完整运行历史。

1.13.2 最外层返回地址为何也要由操作者建立

任务最后执行 RET。按照已经定义的指令规则，它从 MEM32[SP] 读新 PC，随后把 SP 增加四。若初始 SP 直接等于栈上界 0x001f0000，第一次读取就落在栈区之外。启动前应当在最高的有效栈字写入返回地址：

```
MEM32[0x001efffc] = 0x00000300
initial SP         = 0x001efffc
```

0x00000300 位于启动存储，其中的代码只把“任务已经合作返回”写入启动程序工作区，然后进入一个不再修改任务结果的停止循环。这里没有第二项任务，也没有调度决策；返回只把控制权交还给本次运行的固定启动程序。



图示：采用半开区间后，上界本身不是有效地址；把返回字放在 `stack_top-4`，恰好满足四字节读取。

1.14 启动程序如何判断“现在可以进入任务”

复位释放后，启动程序从 `0x00000100` 开始。它不负责把任务从外部搬进 RAM，因为这项工作已经由操作者完成；它只把不可信的 RAM 内容逐步缩小为一组可以采用的初值。

1.14.1 检查顺序不是任意排列

启动程序采用下列顺序：

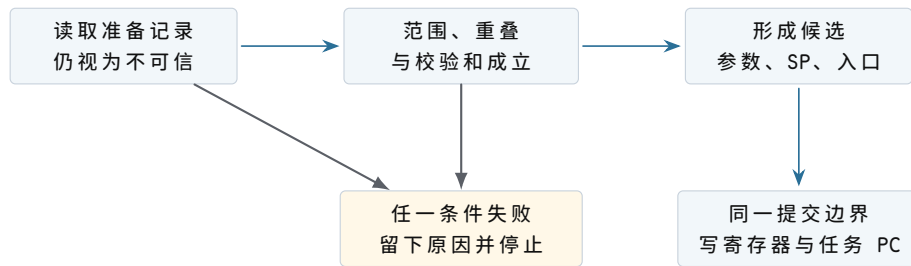
- 1. 读取 `prepared`。若不是一，进入“未准备”停止状态；
- 2. 核对 `magic` 和准备编号，排除旧格式与旧批次；
- 3. 检查所有地址的对齐、范围、长度与互不重叠关系；
- 4. 重新计算代码和输入校验和；
- 5. 检查返回地址位于启动存储规定的返回入口；
- 6. 把参数和栈值写成候选寄存器状态；
- 7. 最后一次重新读取 `prepared`，仍为一才提交任务入口 PC。

之所以先检查短字段，再扫描代码和输入，是因为明显无效的记录不值得触碰它声称的任意地址。之所以在跳转前重读提交标志，是为了明确提交所依据的仍是同一份准备结果。本章规定操作者在释放复位后不再使用夹具写 RAM，所以不会出现真正并发修改；第二次读取仍能让协议边界清晰，并为后续自动输入留下可比较的结构。

1.14.2 候选寄存器状态具体包含哪些值

寄存器	任务入口值	该值从何而来
PC	<code>0x00100000</code>	准备记录的 <code>entry</code> ，已证明落在代码区
SP	<code>0x001efffc</code>	<code>stack_top-4</code> ，该处已有返回地址
r0	<code>0x00102000</code>	输入区首地址
r1	4	<code>data_bytes/4</code>
r2	<code>0x00102020</code>	结果槽首地址
r3	17	本次准备编号，供返回后核对
r4--r7	0	消除上电旧值对本任务的影响

启动程序自己的临时变量位于专用工作区，不能继续依赖将要交给任务的通用寄存器。建立入口状态时，它先在内部或工作区形成候选值；任何检查失败都不提交任务 PC。PC 是最后改变的状态，因为一旦 PC 指向任务入口，下一笔取指就不再属于启动程序。



图示：“形成候选值”和“让任务开始运行”是两个阶段。只有最后的 PC 提交才改变取指所属的代码区域。

1.14.3 六种准备错误分别停在哪里

错误	最早能发现它的检查	为什么不能继续
提交标志仍为零	第一步	内容可能尚未全部写完
准备编号仍为 16	格式字段检查	RAM 中可能是上一次运行留下的记录
代码长度为 <code>0xffffffff0</code>	范围检查	末地址计算可能回绕，不能扫描
入口为 <code>0x00102000</code>	入口检查	它落在输入而不是代码区
代码校验和不符	完整性检查	至少一个被执行的字节与清单不一致
返回地址为奇数	栈字检查	<code>RET</code> 将产生未对齐取指地址

当前机器没有屏幕，也没有串行输出。启动程序把失败阶段和一个辅助数值写到工作区，然后执行停止指令。操作者通过夹具读取：

```

boot_status:
  phase   = PREPARED, FORMAT, RANGE, CHECKSUM, STACK, or ENTERED
  detail  = field offset or failing address
  run_id  = generation copied from the record
  
```

这仍是人工诊断。它的意义在于让每次失败对应一个已经说明的边界，而不是让机器“没反应”成为唯一现象。

1.14.4 这一阶段真正形成了什么

此时可以谨慎地把“可运行的一项程序”描述为四部分的组合：

指令字节 + 初始数据 + 初始处理器状态 + 开始条件。

缺少其中任何一项，物理 RAM 里即使存在某些正确字节，也不足以得到一次可重复运行。准备记录把前三项的位置写明，最后写入的提交标志给出开始核对的条件；启动程序完成验证并提交 PC，才给出真正的开始事件。

这一结构还不是文件，也不是进程映像。它没有名称查找、持久保存、权限、动态分配或多个运行实例。现在给它更大的概念名只会把尚未解决的问题藏起来。下

一节将沿着这份具体布局，观察第一条任务指令怎样开始、最后一条又怎样返回。

1.15 一次人工准备的完整记录

现在把准备过程按地址和顺序展开。假设调试夹具每次最多写四个字节，并能在每笔写后读回。操作者为本次运行选择编号 17。

1.15.1 第一阶段：先宣告当前内容不可运行

处理器保持复位。夹具向 0x001ff030 写入四字节零，再读回确认。即使 RAM 中其余区域仍是上一次运行的完整内容，启动程序此刻也不得采用它。

```
write32 0x001ff030 0x00000000
read32  0x001ff030 -> 0x00000000
```

若这一步读回失败，不能继续写代码后再“看看是否能运行”。失败已经说明夹具、互连旁路或 RAM 写入路径不可靠，应在机器仍保持复位时诊断。

1.15.2 第二阶段：写入三十二个代码字节

地址	写入的四个字节	读回后解释
0x00100000	10 04 00 00	清零 r4
0x00100004	20 05 00 00	从 r0 指向处装入 r5
0x00100008	11 04 05 00	r4 <- r4+r5
0x0010000c	12 00 04 00	r0 <- r0+4
0x00100010	12 01 ff ff	r1 <- r1-1
0x00100014	31 01 fb ff	非零时回退五条指令
0x00100018	21 04 02 00	把 r4 写到 r2 指向处
0x0010001c	41 00 00 00	从任务栈返回

夹具按地址递增写入并读回。读回比较以字节为单位，而不是把四字节先解释成宿主机器的整数，从而避免夹具所在计算机与教学处理器字节序不同造成误判。

1.15.3 第三阶段：写输入、结果哨兵和返回栈字

地址	32 位写入值	用途
0x00102000	0x00000007	第一个输入 7
0x00102004	0xfffffffffe	第二个输入 -2
0x00102008	0x0000000b	第三个输入 11
0x0010200c	0x00000005	第四个输入 5
0x00102020	0xdeadbeef	尚未产生结果的哨兵
0x001efffc	0x00000300	最外层 RET 的返回地址

结果槽与输入末端之间保留十六字节空隙不是硬件要求，而是为了让本例中的错一位地址更容易观察。栈下方大部分空间暂未写入，因为任务没有内部调用；不能因此假定那些字节为零。

1.15.4 第四阶段：写准备记录但仍不提交

```
0x001ff000 magic          = bytes "RUN1"
0x001ff004 generation     = 17
0x001ff008 code_base      = 0x00100000
0x001ff00c code_bytes     = 32
0x001ff010 data_base      = 0x00102000
0x001ff014 data_bytes     = 16
0x001ff018 result_base    = 0x00102020
0x001ff01c stack_low      = 0x001e0000
0x001ff020 stack_top      = 0x001f0000
0x001ff024 entry          = 0x00100000
0x001ff028 code_sum       = computed checksum
0x001ff02c data_sum       = computed checksum
0x001ff030 prepared       = 0
```

操作者重新从记录读取代码首地址与长度，再按记录指定范围读回代码并复算校验和，而不是只拿手边原始清单计算。这样可以同时发现“字节写对但记录地址写错”的情况。数据区同理。

1.15.5 第五阶段：最后四字节才改变准备结论

全部读回一致后，夹具执行唯一的提交写：

```
write32 0x001ff030 0x00000001
read32  0x001ff030 -> 0x00000001
```

随后操作者停止使用夹具写 RAM，再释放复位。若在提交后又修补一个代码字节，原校验和与“提交后内容不变”的前提同时失效，必须重新清零标志并完整核对。

1.16 启动检查怎样对应人工记录

启动程序的检查顺序可以用一张状态表精确表示。每个失败状态都保留准备编号、阶段和一个辅助值。

阶段	成功条件	失败时 detail 保存什么
READY	prepared=1	实际读到的标志
FORMAT	magic=RUN1	第一个不匹配的字节偏移
RANGE	所有区域范围合法且互不冲突	失败字段偏移或区域对
CODE	代码校验和相等	重新计算所得值

DATA	输入校验和相等	重新计算所得值
STACK	栈顶字为合法返回地址	实际返回字
ENTER	再次读取提交标志仍为—	第二次读到的值

1.16.1 成功检查的一组关键快照

快照	当前 PC 范围	已证明的内容	尚未证明的内容
S_0	启动入口	复位起点确定	RAM 是否属于本批次
S_1	记录检查	标志、格式、编号正确	区域内容完整性
S_2	范围检查	地址和长度关系合法	实际字节是否匹配
S_3	校验循环	代码、输入校验和匹配	任务算法是否正确
S_4	交接准备	参数、栈和入口候选完整	任务是否最终返回
S_5	任务入口	入口状态已经提交	结果尚未产生

这组快照避免把不同层次的证明混合。校验和匹配只说明扫描到的字节与记录一致，不说明机器码一定实现求和；算法正确性要由译码表、指令 trace 和循环不变量说明。

1.16.2 一次校验失败的完整状态

假设地址 0x0010000a 的字节应为 05，实际读回为 04。准备记录中的 code_sum 仍按正确代码计算。启动程序扫描 32 字节后得到不同总和，于是写：

```
boot_status.phase = CODE
boot_status.detail = recomputed_sum
boot_status.run_id = 17
processor PC      = checksum_failure_stop
```

它不把 PC 改为 0x00100000，也不保留一部分任务参数作为有效入口状态。操作者保持或重新拉起复位，清零提交标志，修复字节，再重新扫描整个代码区。只重算出错位置之后的部分不足以证明此前字节未被同时改变。

1.17 人工准备为何无法成为长期办法

本例只有 32 个代码字节、16 个输入字节和一份短记录，操作者尚可逐项核对。若程序扩大到 64 KiB，以每笔四字节写入计算，需要 16384 笔写和同量级读回；任意一次地址抄错都可能迫使整段重新核对。更换输入还要重复编辑记录和准备编号。

问题不在于操作者“不够仔细”，而在于机器没有承担可机械重复的搬运工作。已

经存在的启动程序能读取 RAM、计算校验和、建立入口状态，却没有获得外部字节。下一章增加的第一个部件因此应当只解决“怎样让启动程序可靠地接收一串字节”，而不同时引入第二项任务或定时器。

1.18 从复位到任务入口：控制权怎样真正改变

RAM 已由操作者完整准备，但处理器尚处于复位状态。现在释放复位。为了避免把“启动”写成无法检查的瞬间，本节沿时间顺序记录谁发起事务、哪些状态只是候选、哪些状态已经提交。

1.18.1 复位只建立最少的处理器起点

复位电路规定：

```
PC      = 0x00000100
halted  = 0
control = FETCH
SP, r0 ... r7 = unspecified
```

通用寄存器仍是不确定值，所以启动程序不能先使用旧 SP 压栈，也不能假定某个寄存器已经为零。启动程序开头使用处理器内部复位时可用的少量暂存状态，先把自己的工作栈设为 0x001fd000，随后才执行普通调用。这个栈属于启动程序工作区，与任务栈分离。

状态	复位刚释放	启动程序初始化后
PC	0x00000100	位于记录检查代码中
SP	未规定	0x001fd000
r0--r7	未规定	启动程序按自身约定使用
任务参数	尚不存在	仍未提交，只保存在候选区

这里的“未规定”不是随机数发生器，而是软件不得依赖的值。某次实验恰好观察到零，不会把它升级为体系结构承诺。

1.18.2 第一条装入为何分成两笔读取

位于 0x00000100 的第一条指令要读取准备记录的提交标志。处理器先按 PC 取回指令字，再执行该指令形成的数据地址 0x001ff030。两笔事务不可混为一笔：

事务	地址	目标	返回内容
取指	0x00000100	启动存储	LOAD 的四个编码字节
数据读	0x001ff030	RAM	提交标志 1

第一笔读取回答“要执行什么动作”，第二笔读取才回答“提交标志是什么”。只有第二笔响应成功后，目标寄存器和顺序 PC 才提交。若准备记录地址未映射，处理器保留 fault_pc=0x00000100，而不是错误地报告下一条指令。



图示：PC 在数据读取等待期间仍指向产生该读取的指令。这使失败位置具有唯一含义。

1.18.3 验证完成以前，任务寄存器仍不能被看作有效

启动程序可能暂时把 `entry` 读入某个寄存器，但这不表示它已经接受入口。只有格式、范围、重叠、校验和和栈字全部通过后，候选区才包含一组相互一致的值。

```
candidate:
  task_pc = 0x00100000
  task_sp = 0x001efffc
  r0      = 0x00102000
  r1      = 4
  r2      = 0x00102020
  r3      = 17
  r4..r7  = 0
```

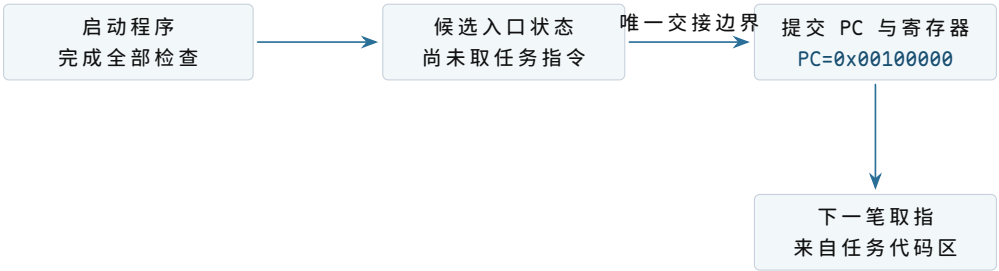
若代码校验失败，启动程序丢弃整个候选区并停止，而不是保留已经读出的三个参数、只修补其中一个。这样，“任务入口状态”要么完整提交，要么完全没有出现。

1.18.4 交接边界上到底保存了什么、改变了什么

验证结束时，启动程序先把自己的继续地址 `0x00000300` 放入任务栈顶，再写任务通用寄存器和 SP，最后提交任务 PC。紧邻提交边界的两份快照如下。

状态	交接前最后一刻	交接后第一刻
PC	启动程序交接指令地址	0x00100000
SP	启动程序工作栈	0x001efffc
r0	启动程序临时值	0x00102000
r1	启动程序临时值	4
r2	启动程序临时值	0x00102020
r3	启动程序临时值	17
r4--r7	启动程序临时值	全部为零
启动程序工作栈	保留检查过程的帧	原样留在 RAM，但任务不应使用

处理器并没有一个名为“任务”的硬件开关。它只在同一时钟提交边界写入这些寄存器，并让下一笔取指使用新的 PC。我们把边界前后的两段执行分别称为启动程序与任务，是因为它们的代码地址、数据约定和责任不同。



图示：交接不是把代码“搬进处理器”，而是改变处理器下一次取指所使用的地址，并按约定建立这段代码将读取的寄存器和栈。

1.19 第一项任务怎样逐步得到 21

任务入口处的静态指令已经给出。现在不再用“循环执行四次”一句带过，而是从第一次取指开始，区分指令地址、数据地址和提交后的状态。

1.19.1 入口指令只清零累加器

第一笔任务取指访问 `0x00100000`。互连根据地址选择 RAM，返回 `MOVI r4,0` 的编码。处理器译码后形成两个候选：

```
candidate r4 = 0x00000000
candidate PC = 0x00100004
```

这条指令不访问数据 RAM。提交时同时写 `r4` 和 `PC`。其他寄存器保持入口值，尤其 `r0` 仍指向第一个输入，`r1` 仍为四。

1.19.2 第一轮装入为何同时出现两个地址

`PC` 为 `0x00100004` 时，取指地址是 `0x00100004`；译码得到 `LOAD r5,[r0]` 后，读取旧 `r0=0x00102000`，形成数据地址 `0x00102000`。前者指向“装入动作的编码”，后者指向“被装入的整数”。

阶段	保存的事实	尚未允许发生的效果
取指完成	当前 <code>PC</code> 、操作码、目标寄存器号	不得写 <code>r5</code>
数据请求被接受	地址、宽度、目标 <code>r5</code> 、后继 <code>PC</code>	不得开始下一条指令
RAM 返回 <code>0x00000007</code>	候选 <code>r5=7</code>	响应成功前仍不提交
装入提交	<code>r5=7, PC=0x00100008</code>	无其他寄存器变化

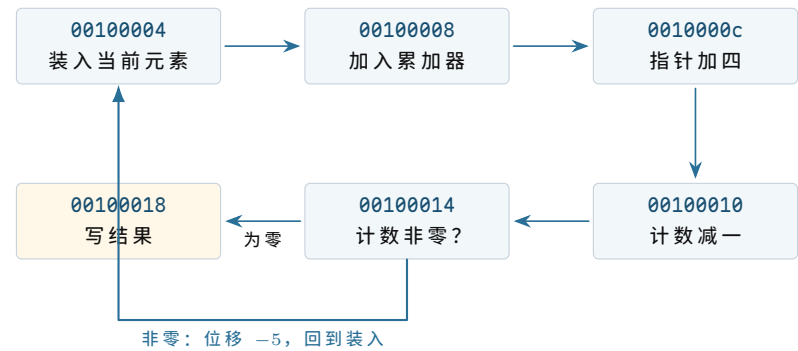
随后 `ADD r4,r4,r5` 读取旧值 0 与 7，候选结果为 7，提交后 `r4=7`。再后两条指令分别令输入指针增加四、剩余计数减少一。

1.19.3 分支位移怎样回到正确的指令

分支位于 `0x00100014`。它的顺序后继是 `0x00100018`，编码中的有符号位移为 -5 个指令字，即 -20 字节：

$$0x00100018 + (-20) = 0x00100004.$$

第一轮结束时 `r1=3`，所以采用目标地址。处理器不会重新执行清零指令；循环入口刻意放在装入处，累加器由此保留上一轮结果。



图示：回边指向装入指令而不是初始化指令。地址计算使“保留累加器”成为可以核对的控制流事实。

1.19.4 四轮以后哪些值应当精确相等

轮次结束	本轮元素	r4	r1	r0
第一轮	7	7	3	0x00102004
第二轮	-2	5	2	0x00102008
第三轮	11	16	1	0x0010200c
第四轮	5	21	0	0x00102010

第四轮分支不采用回边，PC 顺序前进到 `0x00100018`。注意 `r0=0x00102010` 指向输入区末端的第一个字节，不再指向某个有效元素。这是常见的半开区间表示：已处理范围为 `[0x00102000,0x00102010)`。

1.19.5 结果写入何时才算完成

`STORE r4,[r2]` 形成地址 `0x00102020`、数据 `0x00000015` 和四个有效字节。小端写入后的 RAM 为：

```
0x00102020 = 15
0x00102021 = 00
0x00102022 = 00
0x00102023 = 00
```

RAM 在检查地址和字节使能后，于自己的写提交边界替换四个字节并返回 `OK`。处理器收到成功响应后才把 PC 改为 `0x0010001c`。若连接在响应前失败，处理器不能假定结果已写；本章 RAM 契约则保证目标要么接受全部四字节，要么一个也不改变。

1.19.6 RET 怎样把控制权交回启动程序

执行 RET 前：

```
PC          = 0x0010001c
SP          = 0x001efffc
MEM32[0x001efffc] = 0x00000300
```

处理器读取栈顶，验证返回地址四字节对齐且位于可取指范围，然后形成 `new_PC=0x00000300` 与 `new_SP=0x001f0000`。两者在同一指令提交边界更新。下一笔取指重新落入启动存储。



图示：返回地址不是处理器猜出的“调用者”。它是启动前写入栈中的四个具体字节。

1.20 把整次运行放到同一条时间线上

现在可以从操作者开始准备一直排到结果被读回。表中的“可见状态”只列已经提交的事实。

阶段	主动者	关键动作	阶段结束时可依赖的事实
T ₀	操作者	保持复位，清零提交标志	处理器不发起普通事务
T ₁	调试夹具	写代码、输入、哨兵值和返回字	各区域存在，但尚未宣告完整
T ₂	调试夹具	写记录、读回、最后提交	<code>prepared=1</code> 且读回一致
T ₃	复位电路	在稳定边沿释放复位	<code>PC=0x00000100</code>
T ₄	启动程序	检查记录、范围、校验和与栈	候选入口状态完整
T ₅	启动程序	提交参数、SP 与任务 PC	下一笔取指属于任务
T ₆	任务	四轮装入与加法	<code>r4=21, r1=0</code>
T ₇	任务	写结果并执行 RET	结果槽为 21，PC 回到启动存储
T ₈	启动程序	写返回状态并停止修改结果	操作者可以稳定读回

“程序运行结束”在这台机器上不是一个物理引脚。我们把它定义为三个可共同核对的条件：

1. 启动程序工作区的 `boot_status.phase` 为 `RETURNED`；
2. 其中记录的准备编号仍为 17；
3. 结果槽不再是哨兵，并且本例值为 `0x00000015`。

仅满足第二项不能证明任务运行过；仅满足第三项不能排除旧 RAM；仅看到 PC 位于启动存储，也可能是验证尚未结束。组合条件把“本次运行、合作返回、结果可读”联系起来。

1.20.1 如果任务永远不执行 `RET`

把分支位移改成让程序永远回到自身。任务一旦进入该循环，时钟仍不断推动取指与分支，启动程序却没有机会继续执行。当前机器没有能在任务运行中主动改变 PC 的其他部件，也没有第二份处理器状态。

操作者只能重新拉起整机复位。复位会丢弃正在运行的寄存器值，并从 `0x00000100` 重新开始。若 RAM 仍保留原内容，启动程序还必须借助新的准备编号或人工清除提交标志，避免误把旧批次当作一次新运行。

1.20.2 如果任务写错了地址

任务中的 `STORE` 若把 `r2` 改为代码地址，RAM 会按普通写请求接受，因为当前互连只按地址选择目标，不检查“这段字节原来被当作代码”。本次最后一次取指已经完成时，错误写入甚至可能不影响本次结果，却会破坏下一次运行。

这说明启动前的区域不重叠检查只约束初始布局，不能约束运行中的每笔访问。要让某些地址在任务运行时不可写，后续必须增加会参与每笔事务判定的新机制，并说明判定依据从何而来。本章尚未获得这种能力。

1.20.3 如果处理器记录了故障

无效操作码、未映射地址、未对齐访问或向启动存储写入都会使普通效果不提交。处理器锁存 `fault_pc`、`fault_address` 与 `fault_cause`，再把 `halted` 置一。此后时钟仍存在，但取指状态机保持停止。

操作者读取记录后能定位最初失败的指令，却不能让任务从那里继续。复位清除 `halted` 的同时也重新建立 PC，所有未另行保存的寄存器进度都会丢失。故障记录因此属于整机诊断，不属于任务恢复。

到这里能够完整描述的运行。操作者在复位期间建立代码、数据、栈和准备记录；启动程序只验证这些既有字节并提交一组入口状态；处理器随后顺序执行一项任务，任务把 21 写入结果槽并合作返回。任何阶段失败都停止或依赖整机复位，没有后台程序、第二项任务、自动输入设备或运行中强制转移。

1.21 怎样证明描述的确是同一次运行

结果槽中出现 21 很有吸引力，但它不是充分证据。RAM 可能保留了上一次的 21，操作者也可能提前把该值写入。要判断这一次运行，必须把准备、交接、执行和返回四个阶段的证据连成一条不矛盾的链。

1.21.1 为每个阶段选择最小而不同的证据

要证明的事件	使用的证据	单独使用时的不足
本批次已准备完整	<code>prepared=1</code> 、编号 17、两个校验和	只能说明启动前字节通过检查
任务入口已经提交	<code>phase=ENTERED</code> 、记录的入口 PC	不能说明任务走到循环末尾
结果存储已经发生	哨兵被替换为 <code>0x00000015</code>	不能单独排除旧结果或其他写入者
任务已经合作返回	<code>phase=RETURNED</code> 、编号 17	不能替代对结果数值的检查
没有处理器故障	<code>halted=0</code>	不能证明程序语义正确

启动程序在进入任务前把 `phase` 写为 `ENTERED`，返回入口把它改为 `RETURNED`。两次写入都同时复制准备编号。因为当前只有一个处理器和一个运行程序，不存在另一个合法写入者；即便如此，仍要把状态字和结果槽共同检查。

1.21.2 提交次序给证据建立先后关系

我们可以从已定义的提交边界推出：

1. `phase=ENTERED` 的写响应成功后，启动程序才提交任务入口 PC；
2. 任务四轮循环结束后，才可能到达结果 `STORE`；
3. 结果写响应成功后，任务才取到 `RET`；
4. `RET` 成功提交后，返回入口才可能写 `phase=RETURNED`。

于是同一编号下的 `RETURNED` 蕴含控制流已经越过结果存储的成功响应。但它仍不蕴含结果一定是 21，因为错误的算法也可以合作返回。最终数值要由指令 `trace` 和输入字节独立复算。



图示：箭头来自具体指令和提交规则，不是状态名称本身带来的保证。若将来出现并发写入者或缓存，这条证据链需要重新说明。

1.22 从循环不变量复算结果

逐条 trace 可以展示每条动态指令；另一个互补方法是找出每轮都保持的关系。设输入数组为 $a_0, \dots, a_3$ ，刚要开始第 k 轮时：

$$r0 = \text{data_base} + 4k,$$

$$r1 = 4 - k,$$

$$r4 = \sum_{i=0}^{k-1} a_i.$$

当 $k=0$ 时，入口初始化给出 $r0=\text{data_base}$ 、 $r1=4$ 和 $r4=0$ ，关系成立。

假设第 k 轮开始时关系成立。装入得到 a_k ，加法令累加器成为 $\sum_{i=0}^k a_i$ ，指针增加四，计数减少一。因此下一轮开始时：

$$r0 = \text{data_base} + 4(k+1), \quad r1 = 4 - (k+1),$$

不变量继续成立。当第四轮结束时 $r1=0$ ，分支退出，累加器为

$$7 + (-2) + 11 + 5 = 21.$$

这个推导依赖三项此前已具体建立的事实： $r1$ 初值确为四、每个元素恰为四字节、回边指向装入而不指向清零。若任何一项变化，证明必须随之修改。

1.22.1 32 位加法是否会溢出

本例的所有部分和都位于有符号 32 位范围，故位型加法与数学整数加法得到相同解释。一般输入可能溢出。本章处理器的 ADD 保留低 32 位，不自动停止：

$$0x7fffffff + 1 = 0x80000000.$$

若把结果解释为有符号数，它是 -2147483648 。因此“求和”在当前指令契约中准确含义是 32 位模加法；本例恰好无需区分。需要报告算术溢出的程序必须检查标志或使用更宽表示，不能从自然语言“相加”推断硬件会代劳。

1.23 用反事实检查准备协议的每一道边界

好的协议不仅要说明成功路径，还应能回答“少做一步会怎样”。下面每个实验只改变一个准备动作。

1.23.1 没有先清零旧的提交标志

RAM 中仍有编号 16 的完整记录，操作者直接开始覆盖代码，尚未写完就释放复位。若启动程序只检查 `prepared=1`，它可能扫描一半新、一半旧的代码。加入准备编号和校验和后，最迟在完整性检查处停止；而正确操作在第一步清零标志，可以更早明确“尚未完成”。

1.23.2 只检查代码首地址，不检查长度

令 `code_base=0x001ffff0`、`code_bytes=32`。首地址位于 RAM，但区域末端越过 `0x001fffff`。若启动程序逐字节复算校验和，第十七个字节已经没有目标，互连

返回 `UNMAPPED`。预先做不回绕的范围检查能在任何扫描前拒绝该记录，并把原因定位到字段关系，而不是一次偶然的访存失败。

1.23.3 代码与输入区域重叠

若 `data_base=0x00100010`，后两个输入整数会覆盖计数更新和分支指令。两个区域各自都位于 RAM，两个校验和也可能分别按被覆盖后的内容计算成功；只有跨区域重叠检查能指出同一批字节被赋予了互不相容的角色。

1.23.4 先改变 PC，再写完任务寄存器

假设交接代码先提交 `PC=0x00100000`，下一周期才准备 `r0`。第一条任务指令清零 `r4` 后，第二条立即读取旧 `r0`，可能访问任意地址。入口 PC 与参数必须属于同一个交接提交边界，或者硬件必须保证提交后才开始取指。本章采用前者。

1.23.5 结果写入成功，但返回栈字损坏

四轮计算和结果 `STORE` 均可成功，随后 `RET` 读到未对齐地址并使处理器停止。此时结果槽为 21，`phase` 仍为 `ENTERED`，故障原因为返回目标非法。这个例子说明“结果已产生”与“任务已合作返回”是两个不同事件；状态设计需要同时容纳二者。

单点变化	预期停止位置	由哪项状态区分
提交标志未清零	编号或校验检查	<code>phase</code> 与 <code>detail</code>
代码区域越过 RAM	范围检查	失败字段偏移
代码与输入重叠	区域关系检查	两个区域编号
入口寄存器非原子建立	任务早期访存	<code>fault_pc</code> 与数据地址
返回栈字损坏	<code>RET</code>	结果已变、状态仍为 <code>ENTERED</code>

1.24 重复运行为什么不能只按一次复位

第一次任务返回后，RAM 至少有三处变化：结果槽已从哨兵变为 21，准备状态变为 `RETURNED`，任务也可能改写自己的栈。若操作者只复位处理器，启动程序看到的仍是编号 17 和旧结果。

本章规定每次运行采用新的准备编号，并重新执行以下动作：

1. 保持复位，先把 `prepared` 清零；
2. 写入新的编号，恢复输入、结果哨兵和初始栈字；
3. 必要时重写代码并读回全部声明区域；
4. 写入新的校验和，最后重新提交；
5. 释放复位。

即使代码没有变化，也不能省略恢复数据与结果槽，因为程序语义可能依赖初

值。若希望快速重复运行而不重写不变代码，需要进一步区分只读模板与每次运行的可变状态，并由机器自己完成复制；这已经是后续演进，不应暗含在本章的人工步骤中。

1.24.1 复位、重新准备和重新运行是三个不同动作

动作	确定改变的状态	不保证改变的状态
复位	PC、控制状态、halted	RAM 中的代码、输入、结果与记录
重新准备	声明的 RAM 区域与准备编号	处理器当前 PC 和内部执行状态
重新运行	从固定入口推进并产生新效果	断电后的保存、其他机器上的结果

所以正确顺序是先保持复位，再准备，最后释放复位。运行中直接让夹具改代码，会产生一份本章没有定义的交错历史。

1.25 当前机器的责任边界

经过完整走读，我们可以把每个部件承担的责任写得足够窄。

部件或参与者	它负责保证	它不负责保证
操作者与夹具	复位期间按顺序写入并读回声明字节	运行中自动提供新程序
启动存储	复位后提供不变的检查与返回指令	保存任务运行产生的结果
RAM	对成功事务保存和返回相应字节	理解代码、输入、栈等含义
地址互连	每个地址选择唯一目标并返回错误	阻止任务访问别的 RAM 区域
处理器	按指令提交规则更新 PC、寄存器与访存效果	保证任务返回或算法正确
启动程序	验证人工记录并建立一次入口状态	并发运行、自动装入或故障恢复
任务	按约定读取参数、写结果并合作返回	保护启动程序工作区

这张表没有额外创造一个新层次。它只是防止把某个部件的局部承诺扩大成整机承诺。例如 RAM 能够正确保存写入字节，不等于任务不能写错地址；启动程序检查入口合法，也不等于任务运行中只能在代码区取指。

1.25.1 首次运行已经得到的三个重要分界

第一，字节存在与可以开始执行不同。代码、数据、栈和入口状态共同通过检查以后，PC 的提交才建立开始事件。

第二，候选效果与已提交效果不同。装入等待响应时不能提前写目标寄存器，存储失败时不能留下半个新整数，交接失败时不能让任务看到半套参数。

第三，结果出现与控制权返回不同。结果写入发生在 `STORE` 提交，控制权返回发生在 `RET` 提交；状态记录让两者可以分别观察。

这些分界会在后面的机器演进中反复出现，但后续章节必须从新的具体问题重新推导它们的用途，而不能只复述三个口号。

1.25.2 当前机器仍然不能做的事情

- 操作者必须逐字节或逐块写 RAM，程序越大，准备时间越长；
- 每次换程序都要人工计算地址、校验和、栈字和记录字段；
- 运行中的任务独占处理器，除故障停止和整机复位外，没有强制收回途径；
- 所有 RAM 地址对任务可见，初始分区没有运行期保护；
- RAM 断电后不提供本章可依赖的长期保存；
- 机器一次只有一组任务寄存器，没有第二个可暂停后继续的现场。

下一步最直接的困难不是同时运行很多任务，而是人工装入本身。假设程序由几万字组成，继续要求操作者借助夹具写入每个地址，既慢又容易把地址、长度或字节顺序抄错。要让机器从一条外部字节通道自动接收同样的代码与数据，就必须新增接收硬件，并让启动程序承担接收、校验和最后交接。这个问题将单独构成下一章。

1.26 逐条指令 trace：不省略循环中的状态

前面的四轮表只保留了每轮结果。下面把任务从入口到返回的 23 条动态指令全部列出。为了控制表格宽度，地址省略共同的前导 `0x00`；“执行后”表示该指令成功提交后的体系结构状态，未列出的寄存器和内存保持不变。

序号	指令地址	执行动作	提交后的关键状态
1	100000	MOVI r4,0	r4=0, PC=100004
2	100004	从 102000 装入 7	r5=7, PC=100008
3	100008	0+7	r4=7, PC=10000c
4	10000c	输入指针加 4	r0=102004
5	100010	计数减 1	r1=3
6	100014	r1!=0, 采用分支	PC=100004
7	100004	从 102004 装入 -2	r5=-2
8	100008	7+(-2)	r4=5

9	10000c	输入指针加 4	r0=102008
10	100010	计数减 1	r1=2
11	100014	r1!=0，采用分支	PC=100004
12	100004	从 102008 装入 11	r5=11
13	100008	5 + 11	r4=16
14	10000c	输入指针加 4	r0=10200c
15	100010	计数减 1	r1=1
16	100014	r1!=0，采用分支	PC=100004
17	100004	从 10200c 装入 5	r5=5
18	100008	16 + 5	r4=21
19	10000c	输入指针加 4	r0=102010
20	100010	计数减 1	r1=0
21	100014	r1==0，不分支	PC=100018
22	100018	向 102020 存储 21	MEM32[102020]=21
23	10001c	读取栈顶并返回	PC=000300, SP=1f0000

动态指令数可以由结构独立复算。初始化执行一次；循环体包含装入、加法、指针更新、计数更新和分支，共 5 条，执行四轮得到 20 条；最后存储和返回各一次，所以总数为 $1 + 20 + 2 = 23$ 。算式与逐行编号一致，说明循环次数和退出路径没有遗漏。

这类自我校验正是使用具体程序的价值。抽象文字很难暴露“循环到底有几条动态指令”这样的小错误，而带编号的 trace 会立即产生矛盾。

1.26.1 事务数与指令数不是同一个数量

每条指令至少需要一次取指事务，共 23 笔。四条动态 LOAD 再产生 4 笔数据读，STORE 产生 1 笔数据写，RET 产生 1 笔栈读。因此不考虑启动前的人工准备，任务总共发出：

$$23 + 4 + 1 + 1 = 29$$

笔外部事务。

算术指令也会读取寄存器，但寄存器堆位于处理器内部，不经过地址互连。把所有“读”都称为内存事务会高估外部访问；反过来只数显式 LOAD/STORE，又会漏掉取指和返回栈读取。

1.26.2 若 RAM 响应时间不同，程序结果是否改变

设启动存储读取需 2 个周期，RAM 读取需 3 个周期，RAM 写入需 2 个周期。教学处理器在外部事务期间等待，因此总运行周期会增加，但只要每笔事务最终按同一顺序成功，23 条动态指令提交后的寄存器和内存结果不变。

这就是功能抽象与性能事实的分界：软件依赖读取返回相应地址的字节，不应依赖每笔读取恰好几个周期；若程序需要计时，必须另有可读计时器契约。

1.26.3 三组反例：改变一个初值会发生什么

1.26.4 元素个数为零

当前代码先执行一次 `LOAD`，然后才减少计数并判断。因此若启动时 `r1=0`，它仍会读取一个元素，随后计数变为 `0xffffffff`，分支继续，最终很可能越过数组。这说明启动契约必须要求 `r1>0`，或者任务代码要在第一次装入前增加零长度检查。

硬件不会替程序选择哪一种语义。若任务格式允许空数组，正确代码应先判断计数；若格式规定至少一个元素，启动程序应验证输入。约束放在哪里，会决定错误由哪个层次报告。

1.26.5 结果地址与输入重叠

若启动程序把 `r2` 错设为 `0x00102008`，最终存储会覆盖第三个输入整数 11。由于写发生在所有读取之后，本次结果仍是 21；但任务返回后输入数据已改变。若算法在写结果后还要重读数组，行为就会变化。

“这一次碰巧正确”不能证明布局合法。装入契约若要求输入保持不变，就应让结果槽与输入范围不重叠，并在建立参数时检查。

1.26.6 数组末尾越过 RAM

假设 `r0=0x001ffffc`、`r1=2`。第一轮能读取 RAM 最后 4 字节；指针随后变为 `0x00200000`，第二轮地址不在任何目标范围，互连返回 `UNMAPPED`。检查数组范围时应验证：

$$\text{count} \leq \frac{\text{RAM_END} - \text{array_base}}{4},$$

并在乘法前避免 `count*4` 溢出。

1.26.7 把器件承诺重新放回整机

现在所有交互已经走读完成，可以进行一次横向复核：

器件	保存或处理的事实	程序可依赖的接口	当前整机得到的承诺
时钟与复位	边沿、复位有效状态	固定复位 PC	确定起点与状态提交节拍
处理器	PC、SP、寄存器、译码状态	指令集与调用约定	可顺序推进的执行流
启动存储	非易失字节阵列	只读物理范围	复位后立即存在的固件字节
RAM	易失可写单元	可读写物理范围	运行期可变字节空间

地址互连	地址比较和响应路由	物理地址表与错误码	地址唯一选择目标
------	-----------	-----------	----------

最右列刻意使用有限承诺。RAM 不承诺断电保存，互连不承诺权限，处理器也不承诺当前任务最终返回。准确列出这些限制，是为了让下一次增加硬件或程序时能够指出它究竟补上了哪一项缺口。

1.26.8 章末自检：能否独立重建这次运行

读完本章后，应当能够不借助“系统自动完成”这样的句子回答以下问题。

- 1. 复位释放后，为什么第一笔取指访问启动存储而不是任务 RAM？
- 2. `LOAD r5,[r0+0]` 的指令地址、数据地址和目标寄存器号分别是什么？
- 3. 为什么最后一个人工写入字节完成不等于任务已经可以执行？
- 4. 哪些字段共同证明入口地址合法？哪个算术检查必须防止 32 位回绕？
- 5. 启动程序为什么要分别建立任务 SP 和自己的 SP？
- 6. PC 在什么确切时刻从启动程序范围进入任务范围？此前谁控制取指次序？
- 7. 结果写入 RAM、任务返回、操作者稳定读回分别由什么事件完成？
- 8. 如果任务进入无限循环，当前机器中的哪个器件能够强制改变它的 PC？

前七问都能沿本章给出的地址、字段和事务找到具体答案。第八问的答案是“没有”。普通时钟只推动当前指令流，当前也没有能够请求控制转移的外部部件。这一空缺不是概念定义，而是机器在无限循环反例中的可观察事实。

第一章得到的机器。它能从确定复位入口启动，验证一项由操作者预先写入 RAM 的程序，为该程序建立参数和栈，逐条执行到结果写回，并在程序合作时返回启动代码。它尚不能自动接收程序。下一章只增加一条外部字节通道，由“人工准备为何不可持续”推导接收、校验与交接程序。

为什么人工装入需要变成机器自己的工作

继承的机器。操作人员能在复位期间把一项程序、输入和初始栈写入 RAM；释放复位后，启动程序建立寄存器并移交处理器。程序可以计算并合作返回，但更换程序仍要停机并借助外部写入工具。

现在只改变一个条件：机器已经装入机箱，运行期间不能再由操作人员逐地址改写 RAM。外部设备仍然能够按顺序送来字节，但机器必须自己接收、验证并放置这些字节。为此，本章才增加串行控制器和装入程序；它们不负责同时保留两项运行现场，也不能在程序失控时收回处理器。

2.1 从人工写入转向运行中的字节接收

上一章的操作人员可以在复位期间逐地址写 RAM。机器一旦离开实验台，这种方式就出现了具体成本：每次更换程序都要停机、连接调试工具、重新写入多个区域，并由人保证最后一个字节已经完成。机器自身无法重复这套过程。

一种选择是把永远不变的程序直接固化在启动存储中。这样不再需要每次装入，但更换程序要重新写非易失器件；输入参数和可写栈仍要另行准备。本章需要的是“保持固定启动程序不变，运行期间接收另一份可替换程序”，所以必须给运行中的处理器增加一个能够取得外部字节的接口。

2.1.1 为什么不能收到一条指令就立刻执行

外部连接按时间送来字节，处理器执行程序却会分支回到先前地址。若机器收到四个指令字节就立即执行，循环第二次取指时，先前字节已经从顺序数据流中消失。除非外部端同时承担一片可随机访问的指令存储，否则流式执行无法支持本书的求和循环。

另一个问题是完整性。前几条指令到达后便开始执行，后续传输若中断或校验失败，已经执行的指令可能改写 RAM。此时不能再把整次接收当作“从未发生”。因此，基本方案先把全部程序写入 RAM，验证通过以后才改变处理器入口。

方案	省去的步骤	失去的能力或边界
程序固化在启动存储	每次传输与装入	不能方便替换程序
收到一条执行一条	完整 RAM 映像	不能可靠分支回旧指令

接收与执行重叠	等待完整传输的时间	不能在产生效果前完成整体验证
先接收、验证、再移交	无	需要接收状态、RAM 区域和明确提交点

最后一行没有宣称适用于所有机器。它只是给本章提供一条可检查的基本路径：外部字节先成为完整映像，处理器随后一次性从启动程序切到新入口。串行控制器只负责把线上位流整理成字节；映像格式、范围检查和最终移交仍由启动程序负责。

2.2 让外部任务进入机器：串行控制器

可以把求和任务永久写进启动存储，但那样每换一次任务都要重新烧写固件。本章为机器增加一个最小串行控制器。外部发送端在一根接收线上按时间发送位，控制器把位组合成字节，再把字节暴露为处理器可以寻址的状态。发送方向用于固件报告接受、拒绝和最终结果。

2.2.1 新器件接在哪里

串行控制器有两类连接。第一类是机器外部的收发信号；第二类是接入现有地址互连的配置与数据端口。处理器不直接观察线上的每一位，而是通过内存映射寄存器读取已经组装好的字节。



图示：串行线与处理器请求不是同一种事件。控制器先在外围时间域接收位，再把完整字节保存在内部槽中；处理器随后通过互连读取这个槽。

2.2.2 串行控制器内部保存什么

硬件模型。接收移位寄存器按串行时序收集位，形成一个字节后写入 `rx_data` 并置 `rx_ready=1`。处理器读取 `RXDATA` 后清除 `rx_ready`。若旧字节尚未读取而新字节到达，控制器置 `rx_overrun=1`。发送侧在 `tx_ready=1` 时接受一个字节，移出期间清零 `tx_ready`，发送完毕后重新置位。

软件模型。地址互连为控制器分配三个 32 位寄存器：

地址与名称	访问方式	软件可见语义
0x40000000 STATUS	只读	bit0 为 <code>RX_READY</code> ，bit1 为 <code>RX_OVERRUN</code> ，bit2 为 <code>TX_READY</code>
0x40000004 RXDATA	只读	返回接收字节，并消费当前接收槽
0x40000008 TXDATA	只写	在 <code>TX_READY=1</code> 时提交一个待发送字节

固件读取字节时必须先检查状态：

```
receive_byte:
    status = read32(0x40000000)
    if status has RX_OVERRUN: reject current transfer
    if status lacks RX_READY: repeat
    return read8(0x40000004)
```

抽象。上层软件得到一个按到达次序读取、按提交次序发送的字节通道。它不保证任务边界、长度、校验或重传；这些规则要由固件和外部发送端建立。它也不保证处理器在做其他计算时仍能及时取走字节，单槽接收的速率上限由轮询速度决定。

2.2.3 为什么不能只约定“发完为止”

串行字节本身没有结束标记。若发送端静默，固件无法区分“任务已经发完”和“下一个字节还没到”。因此，传输必须先提供长度。长度又可能因误码变成巨大数值，固件还需要固定标识和校验，才能避免把随机字节当作可执行映像。

本章使用 32 字节的教学头部：

偏移	字段	为什么必须存在
+0x00	<code>magic</code>	区分任务传输和随机输入，本章固定为 <code>MTK1</code>
+0x04	<code>header_bytes</code>	使接收者知道载荷从哪里开始
+0x08	<code>image_bytes</code>	决定接收终点并用于范围检查
+0x0c	<code>load_address</code>	指定载荷写入 RAM 的起始位置
+0x10	<code>entry_offset</code>	在载荷内部指出第一条任务指令
+0x14	<code>zero_bytes</code>	指定载荷后需清零的可写区域
+0x18	<code>stack_bytes</code>	说明任务要求的最小栈空间
+0x1c	<code>checksum</code>	检测头部之外的载荷是否损坏

这只是固件与发送端的装入协议，不是文件系统中的文件格式。它没有文件名、目录、权限、时间戳或持久对象身份。给它增加这些名称并不会让机器自动获得文件系统。

2.3 装入不是复制一遍就结束

固件收到头部后，不能立即按其中地址写 RAM。一个恶意或损坏的 `image_bytes` 可能在加法中溢出，使表面上的终点变成较小地址；载荷可能覆盖固件工作区或任务栈；入口偏移也可能指到载荷之外。所有会影响写入范围和最终 PC 的字段都必须先验证。

2.3.1 先确定本次内存所有权

本章采用以下 RAM 布局：

物理范围	启动期间的所有者	用途
0x00100000--0x0017ffff	待装入任务	映像、零填充区和任务数据
0x00180000--0x001effff	待装入任务	向下增长的任务栈候选区
0x001f0000--0x001ffff	固件	接收状态、校验状态和固件栈

“所有者”此时只是固件遵守的软件约定，并非硬件权限。固件在移交前不会主动把任务载荷写进自己的工作区；任务开始后却仍可用普通存储指令覆盖那里。正是这种缺乏强制边界的事实，决定了本章还不能称为有了操作系统保护。

2.3.2 范围检查必须考虑整数溢出

设

$$L = \text{load_address}, \quad N = \text{image_bytes}, \quad Z = \text{zero_bytes}.$$

若直接计算 $L + N + Z$ ，32 位加法可能回绕。稳妥检查先确认每个长度不超过允许区间，再用减法形式判断：

$$N \leq U - L, \quad Z \leq U - (L + N),$$

其中 U 是任务载入区的开区间上界 0x00180000。在使用 $U - L$ 前，还要确认 $L < U$ 且 L 不低于 0x00100000。

入口地址

$$E = L + \text{entry_offset}$$

必须落在已接收的映像字节中，并满足 4 字节指令对齐。栈大小必须能在候选栈区内安排，且栈区不能与载荷及零填充区重叠。

```
validate_header(h):
    require h.magic == MTK1
    require h.header_bytes == 32
    require TASK_LOAD_LOW <= h.load_address < TASK_LOAD_HIGH
    require h.image_bytes <= TASK_LOAD_HIGH - h.load_address
    image_end = h.load_address + h.image_bytes
    require h.zero_bytes <= TASK_LOAD_HIGH - image_end
    require h.entry_offset < h.image_bytes
    require (h.load_address + h.entry_offset) is 4-byte aligned
    require MIN_STACK <= h.stack_bytes <= STACK_REGION_SIZE
```

验证顺序是协议的一部分。只有前一项证明相应运算安全，后一项才可以使用计算结果。把所有条件压成一个长布尔表达式，会掩盖哪一步可能先发生溢出。

2.3.3 逐字节接收时谁拥有哪些状态

头部通过后，固件建立一个装入记录：

```
load_record:
    destination_start
    destination_next
    payload_remaining
    checksum_so_far
    expected_checksum
    entry_address
    zero_start
    zero_bytes
    stack_low
    stack_top
    state = RECEIVING
```

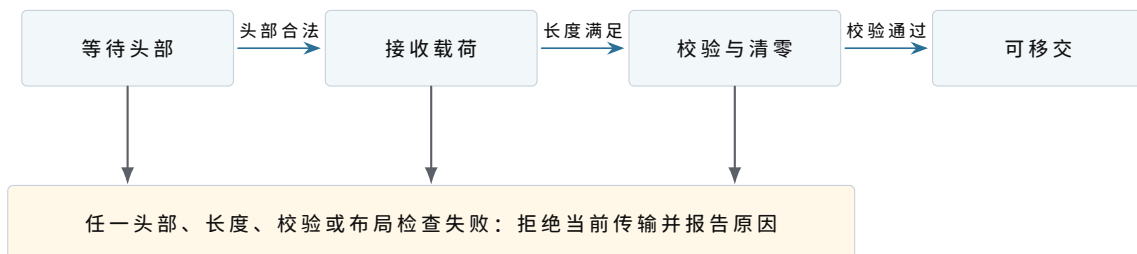
`destination_next` 指出下一个字节写到哪里；`payload_remaining` 防止接收循环依赖外部静默；`checksum_so_far` 只覆盖已经接受的载荷。每消费一个 `RXDATA` 字节，固件按固定顺序更新这些字段：

1. 从控制器消费一个字节；
2. 把字节写到 `destination_next`；
3. 将该字节并入校验状态；
4. 递增 `destination_next`；
5. 递减 `payload_remaining`。

若在第 2 步之后复位，RAM 中可能留下部分载荷，但复位会重新从启动存储执行，装入记录也会重建。固件不能仅凭 RAM 中“看起来像代码”的字节跳转，必须重新完成协议验证。

2.3.4 接收完成、验证完成与可执行不是一个时刻

最后一个字节写入 RAM 时，只能说明传输长度已经满足。固件还要比较最终校验值、清零 `zero_bytes` 指定的区域、建立输入数组和初始栈。为避免把半成品当作任务，装入记录按以下状态推进：



图示：“载荷到齐”只关闭接收阶段；只有校验、零填充和布局建立全部成功，装入记录才进入“可移交”。任何失败都不能改变 PC 到任务入口。

本章的提交点不是写入最后一个字节，而是稍后把处理器 PC 改为已经验证的入口地址。在此之前，即使 RAM 中已有完整任务，处理器仍执行固件。

2.3.5 用具体数字走完一次装入

求和任务头部给出：

```
magic          = MTK1
header_bytes   = 32
image_bytes    = 0x00000300
load_address   = 0x00100000
entry_offset   = 0x00000000
zero_bytes     = 0x00000100
stack_bytes    = 0x00004000
checksum       = expected value
```

载荷占用 `0x00100000--0x001002ff`，零填充区占用 `0x00100300--0x001003ff`，都低于任务载入区上界。任务栈使用 `0x001ec000--0x001effff`，与映像不重叠。入口地址就是 `0x00100000`。

输入数组是任务数据的一部分，位于 `0x00102000`，不在上述 `0x300` 字节载荷范围内会产生矛盾。为使布局一致，本次实际映像应覆盖到输入区之后，因此把 `image_bytes` 修正为 `0x00002100`。这项复算刻意展示了字段之间必须相互核对：若只逐项抄表，装入器会接受一个宣称包含输入数据、实际长度却到不了输入地址的映像。

修正后的范围为：

范围	装入后的内容	来源
<code>0x00100000--0x0010001f</code>	求和核心指令	载荷
<code>0x00100020--0x00101fff</code>	其他任务代码与常量	载荷
<code>0x00102000--0x0010200f</code>	四个输入整数	载荷
<code>0x00102020--0x00102023</code>	结果槽，初值为零	载荷
<code>0x00102100--0x001021ff</code>	未初始化区	固件清零
<code>0x001ec000--0x001effff</code>	任务栈	固件保留并建立初始内容

外部发送端不应发送含糊的“差不多大小”。装入协议中的长度必须覆盖实际传输的每个字节，固件也只能在校验过的范围内解释入口和数据。

2.4 从固件现场建立任务现场

任务字节可执行后，处理器仍保存固件的 PC、SP 和寄存器。直接把 PC 改为任务入口会让任务继承任意寄存器值，并继续使用固件栈。任务第一次访问参数或执行 `RET` 时就会得到错误结果。固件必须先构造一份明确的初始现场。

2.4.1 初始寄存器约定

本次启动约定如下：

状态	移交值	原因
r0	0x00102000	输入数组首地址
r1	4	数组元素个数
r2	0x00102020	结果槽地址
r3--r7	0	消除对固件残留值的依赖
SP	0x001efffc	指向预置的固件返回地址
PC	0x00100000	最后一步才写入的任务入口

SP 不是直接设为 0x001f0000。固件先在 0x001efffc--0x001effff 写入 32 位返回地址 0x00000300，再让 SP 指向这个字。任务最终执行 RET 时，处理器读取 MEM32[SP] 作为新 PC，并把 SP 增加 4，于是控制流回到固件返回入口。



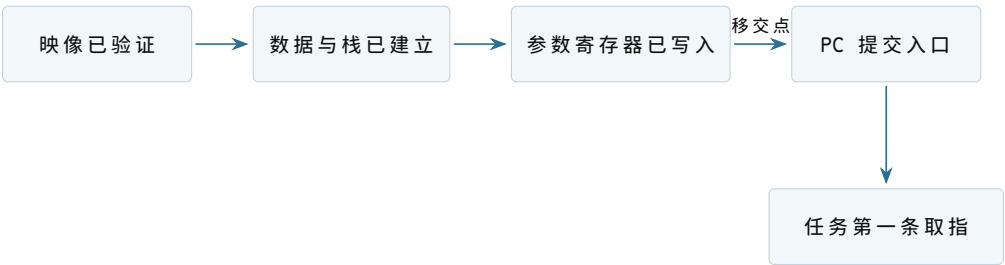
图示：向下增长表示压入新内容时 SP 减小。栈区上界本身是开区间；预置返回地址占用上界下方最后 4 字节。

2.4.2 为什么入口地址必须最后提交

建立初始现场的安全顺序是：

- 1. 确认装入记录处于“可移交”；
- 2. 停止接收本次映像，避免后续字节继续改写任务区；
- 3. 写入输入数据、结果初值和零填充区；
- 4. 建立任务栈并写入固件返回地址；
- 5. 设置通用寄存器和 SP；
- 6. 最后把 PC 改为任务入口。

前五步仍在固件指令流中执行。第六步一旦提交，下一次取指来自任务区域，固件就不能继续执行“还有一步初始化”。因此 PC 的改变是控制权移交的提交边界。



图示：“字节已经在 RAM 中”和“任务已经运行”之间隔着初始现场建立与 PC 提交。只有最右侧事件改变取指来源。

2.4.3 移交前后的处理器快照

移交前，固件可能有如下状态：

```
PC = 0x000002d8      SP = 0x001ffec0
r0 = load_record_ptr r1 = checksum_so_far
r2 = 0x40000000      r3 = temporary value
```

移交完成后的第一条任务指令开始前，状态必须变为：

```
PC = 0x00100000      SP = 0x001efffc
r0 = 0x00102000      r1 = 4
r2 = 0x00102020      r3 = 0
r4 = 0                r5 = 0
r6 = 0                r7 = 0
```

这两个快照说明“切换”不能只描述为跳转。至少有 PC、SP、参数寄存器和清理后的临时寄存器发生变化。当前固件不需要保存自己的全部现场，因为任务返回后固件进入一个固定返回入口，而不是回到 `0x000002d8` 继续原函数。下一章需要暂停并继续不同任务时，保存现场的要求会更严格。

2.5 把装入器写成可审计的状态机

“固件接收并装入任务”仍然包含多个可能暂停或失败的阶段。若所有局部变量只散落在寄存器中，错误入口无法判断已经接收多少字节，也无法给出稳定诊断。固件因此在自己的 RAM 工作区保存一份装入记录。它不是操作系统任务对象，只代表当前这一笔传输。

2.5.1 字段、写入者和有效期

字段	谁在何时写入	后续读取目的
<code>state</code>	阶段转换代码	限制当前允许的操作
<code>destination_next</code>	每接受一个字节后递增	决定下一笔 RAM 写地址
<code>payload_remaining</code>	头部建立，逐字节递减	判断载荷何时到齐
<code>checksum_so_far</code>	每个已接受字节更新	与头部期望值比较
<code>entry_address</code>	头部验证后计算一次	移交时写入 PC
<code>stack_low/top</code>	布局验证后计算一次	清零栈并设置 SP
<code>error_code</code>	首个失败检测点	报告哪条契约被破坏

装入记录从收到头部开始存在，任务入口提交前仍由固件独占。任务运行后，记录不再描述一个“正在接收”的事务；固定返回入口可以把它改为 `FINISHED`，或直接

重置为 `WAITING_HEADER`。若任务永不返回，记录也不会自行变化。

2.5.2 允许的状态转换

当前状态	输入事件	必须完成的动作	后继状态
<code>WAITING</code>	32 字节头到齐	解码并验证所有范围	<code>RECEIVING</code> 或 <code>REJECTED</code>
<code>RECEIVING</code>	一个载荷字节到达	写 RAM、更新校验和与计数	保持或 <code>VERIFYING</code>
<code>RECEIVING</code>	控制器溢出	记录传输错误	<code>REJECTED</code>
<code>VERIFYING</code>	校验匹配	清零、建栈、建立参数	<code>READY</code>
<code>VERIFYING</code>	校验不匹配	记录内容错误	<code>REJECTED</code>
<code>READY</code>	固件决定运行	设置现场并提交 PC	<code>RUNNING</code>
<code>RUNNING</code>	固定返回入口获得控制	读取并发送结果	<code>FINISHED</code>

状态表排除了若干非法动作：`WAITING` 状态不能消费载荷；`REJECTED` 状态不能跳任务入口；`RUNNING` 状态下固件也没有机会继续修改记录。把这些限制写入转换函数，比在各处检查零散布尔量更容易审计。

2.5.3 头部在串行线上也是字节

以修正后的映像为例，头部前 28 个字节可表示为：

offset	bytes	decoded value
00	4d 54 4b 31	magic "MTK1"
04	20 00 00 00	header_bytes = 0x20
08	00 21 00 00	image_bytes = 0x2100
0c	00 00 10 00	load_address = 0x00100000
10	00 00 00 00	entry_offset = 0
14	00 01 00 00	zero_bytes = 0x100
18	00 40 00 00	stack_bytes = 0x4000
1c		checksum

`load_address` 的四个字节为 `00 00 10 00`。按低有效字节在前组合后才是 `0x00100000`。若固件按相反字节序解码，会得到 `0x00001000`，落入未映射区域。传输协议必须规定字节序，不能依赖发送端和处理器“碰巧相同”。

2.5.4 校验和覆盖什么，不能证明什么

本章采用一个教学化的 32 位逐字节滚动校验：

```
checksum = 0
for each payload byte b:
    checksum = rotate_left(checksum, 5)
    checksum = checksum XOR b
```

发送端对全部载荷计算结果并放入头部，固件按实际收到的次序重新计算。它能够高概率检测常见误码、丢字节和次序变化，但不是密码学签名。知道算法的人可以构造另一个产生相同校验值的载荷；校验通过也不能证明代码可信、没有越界指令或实现了正确算法。

这里有三种不同判断：

1. 格式有效：头部字段可安全解释，范围没有越界；
2. 传输一致：载荷长度和校验符合发送端给出的值；
3. 程序可信：代码来源和行为满足安全策略。

本章只完成前两项。当前机器没有签名验证、权限或沙箱，不能从“校验通过”推出“任务可以任意访问机器而没有风险”。

2.5.5 用失败用例检验头部验证顺序

给装入器一组具体头部，比只读正常伪代码更容易发现边界错误。

2.5.6 长度恰好到达上界

若 `load_address=0x0017ff00`、`image_bytes=0x100`、`zero_bytes=0`，映像终点恰好为开区间上界 `0x00180000`，可以接受。最后一个有效字节地址为 `0x0017ffff`。

若 `image_bytes=0x101`，最后一个字节会落在为栈候选区保留的 `0x00180000`，必须拒绝。用开区间终点计算可以避免“最后一个地址加一”再次溢出。

2.5.7 32 位回绕伪装成小终点

设攻击头部为：

```
load_address = 0xffffffff0
image_bytes  = 0x00000040
```

32 位直接相加得到 `0x00000030`。若装入器只检查“终点小于 RAM 上界”，会错误接受。正确顺序先检查起点是否位于任务区；起点检查已经失败，根本不执行可能回绕的终点判断。

2.5.8 入口位于零填充区

若 `entry_offset=image_bytes+4`，入口落在固件清零的区域。虽然地址属于任务 RAM，那里并不是发送端提供且校验过的指令，所以必须拒绝。入口条件是

$$0 \leq \text{entry_offset} < \text{image_bytes},$$

而不是小于 `image_bytes + zero_bytes`。

2.5.9 入口未对齐

`entry_offset=2` 使 PC 成为 `0x00100002`。该地址位于载荷内部，却不满足教学处理器的 4 字节取指对齐，必须在移交前拒绝。若等到处理器取指才发现，错误仍

可检测，但诊断会从“非法映像入口”退化成“处理器对齐故障”，丢失了协议层原因。

2.5.10 栈与映像不重叠仍可能过小

只检查栈区不重叠还不够。本章至少要预置 4 字节返回地址，因此 `stack_bytes < 4` 必须拒绝。实际约定采用更大的 `MIN_STACK`，为任务的函数调用保留空间。最小值是启动协议的一部分，不由 RAM 自动推断。

复算点。对 `load_address=0x00170000`、`image_bytes=0x10000`，映像终点为 `0x00180000`，仍合法；若再要求 `zero_bytes=1`，则第一字节零区会进入栈候选范围，整个头部必须拒绝。

2.5.11 串行速度如何限制轮询装入器

采用常见的 8N1 串行帧时，一个数据字节在线上占 1 个起始位、8 个数据位和 1 个停止位，合计 10 位。若传输速率为 115200 位每秒，一个字节约每

$$\frac{10}{115200} \text{ s} \approx 86.8 \text{ } \mu\text{s}$$

到达一次。

假设处理器频率为 10MHz，两个字节到达之间约有 868 个处理器周期。固件读取状态、消费字节、写 RAM 和更新校验必须在这段预算内完成，或让发送端等待。若最坏路径需要 950 周期，平均速度看似接近，单槽控制器仍会周期性溢出。

2.5.12 平均成本不足以证明不会溢出

轮询循环平时可能只需 200 周期，但遇到错误报告、长 RAM 等待或其他固件工作时延迟会变大。不会溢出的条件约束的是两次消费之间的最长间隔，而不是平均每字节成本：

$$T_{\text{consume,max}} < T_{\text{arrival}}.$$

如果无法满足，可以选择让发送端依据硬件流量控制暂停、增加接收 FIFO，或以后引入中断和 DMA。每种方案都会新增状态和失败模式；当前样机选择最简单的发送端应答协议。

2.5.13 逐块应答建立背压

外部端每发送 64 字节后等待固件返回一个确认字节。固件只有在这 64 字节全部写入 RAM 且未发生溢出时才确认。若返回拒绝码，外部端从整个映像开始重传。本章不实现分块恢复，因为那需要块编号、重复检测和部分校验等新状态。

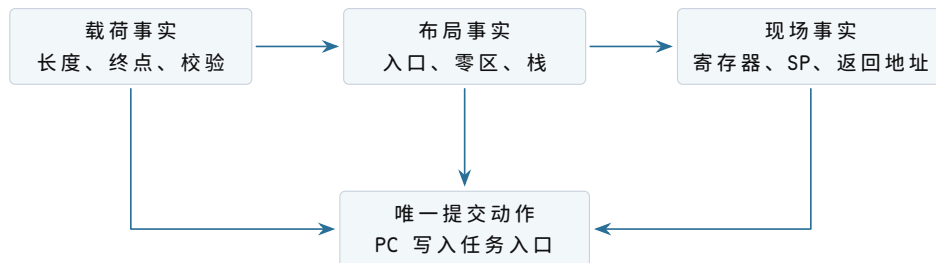
这个协议把发送速率约束反馈给发送端，称为背压。背压不提高控制器内部容量，却阻止外部端无限快地提交数据。即便没有操作系统，异步生产者与有限缓冲之间也已经存在同样的资源问题。

2.5.14 一次成功移交的详细不变量

在提交任务 PC 之前，固件逐项确认：

1. 装入记录状态为 `READY`;
2. `destination_next=load_address+image_bytes`;
3. `payload_remaining=0`;
4. 实际校验值等于头部校验值;
5. 入口位于已校验映像且满足对齐;
6. 零填充区已经写零;
7. 输入数组和结果槽位于任务数据范围;
8. 栈范围与映像、固件工作区均不重叠;
9. `MEM32[0x001efffc]=0x00000300`;
10. 通用寄存器和 `SP` 已按启动契约设置。

这些条件称为移交不变量：只要固件准备改变 `PC`，它们就必须同时成立。若检查分散在不同函数中，最终移交函数仍应以断言或显式返回值确认它们，而不能依赖“前面大概检查过”。



图示：三组事实都成立，才允许执行控制权提交。PC 写入不是又一次“检查”，而是使任务开始取指的不可混淆边界。

2.5.15 用装入记录快照检查一次正常传输

下面选择四个时刻观察装入记录。设载荷长度为 `0x2100`，起点为 `0x00100000`。

观察时刻	下一写入地址	剩余字节	已经成立的事实
头部通过后	<code>0x00100000</code>	<code>0x2100</code>	范围与入口合法，尚无载荷字节
第 1 字节后	<code>0x00100001</code>	<code>0x20ff</code>	起始字节已写入并纳入校验
第 64 字节后	<code>0x00100040</code>	<code>0x20c0</code>	第一块可向发送端确认
最后字节后	<code>0x00102100</code>	<code>0</code>	长度满足，等待最终校验

两个数值始终满足不变量：

$$\text{destination_next} - \text{load_address} = \text{image_bytes} - \text{payload_remaining}.$$

左边是已经写入的地址跨度，右边是已经消费的字节数。若更新时漏掉递增或递减，其中一边会立刻失配。固件可在调试版本中每轮检查该式。

2.5.16 第 37 个字节损坏时哪些字节可见

假设传输没有丢失长度，但第 37 个字节从 `0x11` 变成 `0x10`。装入器会继续收满 `0x2100` 字节，最终校验不匹配，状态转为 `REJECTED`。此时 RAM 中确实有一份长度完整但内容错误的映像。

固件不必在发现校验失败时逐字节回滚，因为任务 PC 尚未提交。它必须保证两点：不把记录转为 `READY`；下一次装入重新覆盖可能使用的范围。若机器要在多个已装入映像之间保留旧版本，直接覆盖就不再足够，需要独立暂存区或双缓冲；本章只有一项任务，因此不引入该结构。

2.5.17 第 37 个字节之后串行溢出

若控制器置 `RX_OVERRUN`，固件知道至少一个字节丢失，却不知道具体有几个。此时 `destination_next` 只表示已经消费的字节数，不能对应发送端位置。固件立即进入 `REJECTED`，丢弃直到下一次明确的传输同步标记，再请求整包重发。

仅把 `payload_remaining` 继续减到零会造成错误对齐：后续字节向前填补缺口，头部长度的仍可能满足，但内容位置全部错位。校验大概率会失败，却要等整包结束才能发现。尽早利用硬件溢出状态可以缩短失败路径。

2.5.18 零填充区为什么不随载荷发送

任务常需要一片初始值全为零的可写区域，例如计数器数组或暂存缓冲。如果发送端把几千个零也逐字节发送，既增加传输时间，又没有增加信息。头部用 `zero_bytes` 表达“映像末尾之后这段范围必须被固件写零”。

这一做法建立了两类来源不同的任务字节：

区域	初值来源	校验与建立规则
映像区	外部载荷	每个字节参与传输校验
零填充区	固件写零	范围由已验证头部决定，不占载荷
栈区	固件初始化	预置返回地址，其余按启动策略清零

零填充必须发生在校验通过之后或之前均可，但在状态进入 `READY` 前必须完成。若清零中途发生复位，启动流程重新开始，不能沿用旧的 `READY` 标记。

2.5.19 为什么不能把整个 RAM 都清零

全清零看似简单，却会覆盖固件工作区，包括当前装入记录和固件栈。即使先避开工作区，清零整片 RAM 也会让启动时间随容量增长。按头部和固定布局只初始化任务真正依赖的区域，使写入范围与所有权一致。

2.5.20 任务栈如何支持一次真实函数调用

求和核心本身是叶函数，但正式任务通常会调用子程序。设任务在写结果前调用位于 `0x00100100` 的格式化例程，调用指令地址为 `0x00100018`，顺序后继为 `0x0010001c`。调用前：

```
PC = 0x00100018
SP = 0x001efffc
MEM32[0x001efffc] = 0x00000300 // outer return
```

`CALL` 先把 `SP` 降到 `0x001efff8`，写入内部返回地址 `0x0010001c`，再进入子程序：

```
PC = 0x00100100
SP = 0x001efff8
MEM32[0x001efff8] = 0x0010001c // return to task
MEM32[0x001efffc] = 0x00000300 // return to firmware
```

子程序的 `RET` 先消费 `0x0010001c`，恢复到任务内部，`SP` 回到 `0x001efffc`。任务最外层 `RET` 再消费 `0x00000300`，返回固件。两个返回地址按后进先出次序工作。



图示：返回地址的顺序来自 `SP` 的移动和 `RAM` 中的布局，不来自处理器对“函数层次”的理解。破坏任一栈字都会改变后续控制流。

2.5.21 谁负责保证栈不越界

当前处理器只检查地址是否位于整个 `RAM`，不知道任务栈下界 `0x001ec000`。若调用过深，`SP` 可以越过下界并覆盖其他任务数据，只要地址仍在 `RAM`，硬件就会接受。固件给出 16 KiB 空间并不能强制任务停在边界内。

可以让编写任务的人证明最大栈深，或在每次函数入口加入软件检查；两者仍依赖任务合作。真正的隔离需要地址访问权限，后续才会从任务相互覆盖的失败中推导。

2.5.22 移交失败时由谁收束状态

在 `PC` 提交之前，固件是唯一执行者，也是装入记录、任务目标区和串行协议的所有者。任何失败都由固件把记录转为 `REJECTED`，发送错误码，并回到等待新头部状态。

`PC` 提交之后，固件不再执行。任务拥有当前寄存器与任务栈，直到 `RET`。此后出现的无限循环、任意 `RAM` 写入和栈破坏，不能再由“装入器检查一次”解决。阶段不同，能采取行动的主体也不同。

阶段	当前执行者	可修改的关键状态	失败收束
接收头部	固件	装入记录、串行槽	拒绝传输
接收载荷	固件	任务 RAM、校验状态	拒绝并重传
建立现场	固件	任务栈、参数寄存器	不提交 PC
任务运行	任务	寄存器及全部可写物理 RAM	合作返回或整机错误入口
固定返回	固件	固件栈、发送状态	报告结果并等待

这张所有权表给出了本章最重要的时间边界：同一片 RAM 在不同阶段由不同代码解释和修改，但硬件尚未实施所有权。当前所有权是分析软件责任的工具，不是安全保证。

2.6 把固件主循环连成完整控制程序

前面已经逐个定义装入记录、串行寄存器和任务现场。现在可以给出不隐藏阶段的固件主循环。伪代码使用函数名表达已经解释过的局部动作，不涉及构建工具或源文件位置。

```

reset_entry:
    initialize firmware SP
    clear serial protocol state
    reset load_record to WAITING

wait_for_transfer:
    header = receive_exactly(32)
    if serial overrun: report SERIAL_OVERRUN; goto wait_for_transfer
    if not validate_header(header):
        report HEADER_INVALID
        goto wait_for_transfer

    initialize load_record from header
    while payload_remaining != 0:
        b = receive_one_byte()
        if serial overrun:
            reject SERIAL_OVERRUN
            goto discard_and_resync
        store b at destination_next
        update checksum and counters
        acknowledge each completed 64-byte block

    load_record.state = VERIFYING
    if checksum_so_far != expected_checksum:
        report CHECKSUM_MISMATCH
        goto wait_for_transfer

```

```

zero requested region
build task stack and arguments
assert handoff invariants
load_record.state = READY
handoff_to_task()          // PC commit occurs here

```

`handoff_to_task` 正常情况下不返回到下一行。它写入任务寄存器和 SP，最后通过受控跳转提交 PC。任务执行最外层 `RET` 时进入另一个固定入口 `firmware_return`，而不是回到调用点。

2.6.1 为什么返回入口不能沿用装入函数的普通栈帧

装入函数的局部变量位于固件栈。移交任务时，SP 已切到任务栈；任务也可以改写所有通用寄存器。若返回入口试图像普通函数那样从旧 SP 取局部变量，它会读错位置。固定入口必须先把 SP 设为固件栈顶，再通过全局固定地址找到装入记录。

```

firmware_return:
    SP = 0x00200000
    load_record.state = FINISHED
    result = read32(0x00102020)
    send_result_frame(result)
    reset load_record to WAITING
    goto wait_for_transfer

```

这里 `0x00200000` 是 RAM 上界，固件实际第一项栈内容写在其下方。固件工作区位于 `0x001f0000--0x001fffff`，任务栈止于 `0x001effff`，两者按软件布局分离。

2.6.2 结果帧为什么还要带状态

只发送四个结果字节，外部端无法区分正常结果、错误码和上一次传输残留。固件发送一个短帧：

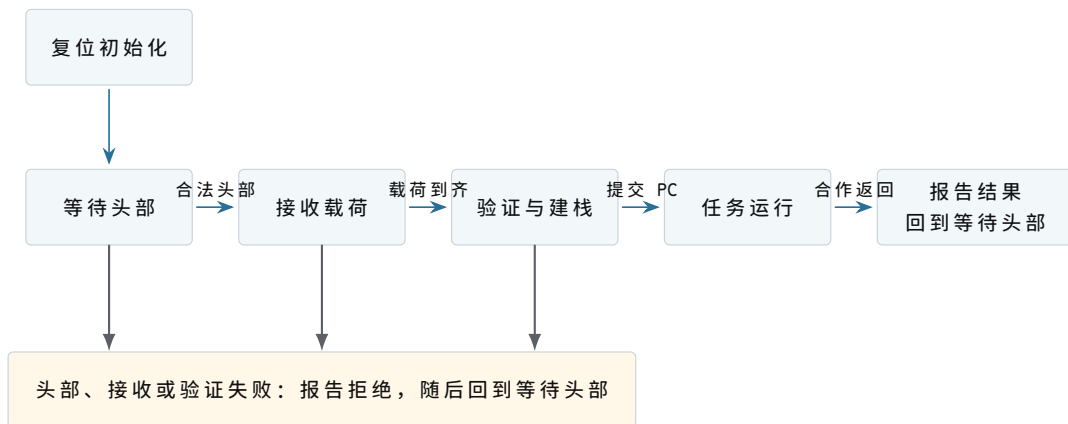
```

result_frame:
    magic      = "RES1"
    status     = OK or an error code
    value      = valid only when status == OK
    checksum   = checksum of preceding fields

```

本次成功帧中的 `value` 为 32 位整数 21，在线上按 `15 00 00 00` 发送。外部端只有在完整接收并验证结果帧后，才把任务标记为成功。固件写 `TXDATA` 只是提交一个待发送字节，不等于外部端已经收到整帧。

2.6.3 从复位到等待下一任务的状态图



图示：上排是一次成功传输从左到右的状态变化，下排统一表示任务运行以前的拒绝路径。进入任务后，唯一正常出边仍是任务主动返回；当前机器没有从运行状态强制回到固件的事件。

2.6.4 状态图中的终止条件

等待头部没有时间上限；机器可以长期停在那里。接收载荷若外部端中途停止，当前协议也会无限等待，因为机器尚无可编程计时器。若希望超时拒绝，必须增加能产生时间事件的硬件和相应软件状态。本章不借用尚未加入的能力。

任务运行同样可能没有终点。状态图不是承诺所有路径最终回到等待，而是准确标出哪些转换需要外部合作。正式模型必须允许停留在 **RECEIVING** 或任务运行状态。

2.6.5 一次完整成功运行中的关键数值

时刻	关键状态	可据此推出的事实
复位释放	<code>PC=0x00000000</code>	下一笔取指来自启动存储
头部通过	<code>state=RECEIVING</code>	地址与长度安全，内容尚未验证
载荷到齐	<code>remaining=0</code>	已写 <code>0x2100</code> 字节
校验通过	<code>state=VERIFYING</code>	传输内容与发送端一致
现场完成	<code>SP=0x001efffc</code>	栈顶含固件返回地址
移交提交	<code>PC=0x00100000</code>	任务开始取指
第四轮后	<code>r4=21, r1=0</code>	循环将退出
结果写回	<code>MEM32[0x00102020]=21</code>	处理器可读取结果
任务返回	<code>PC=0x00000300</code>	固件重新获得控制
结果帧到达	外部校验通过	本次任务对外完成

这十个时刻覆盖了硬件起点、协议接受、内容验证、执行移交、计算效果、控制返回和外部可见。任意相邻两行之间都存在不能省略的动作，因此不能把它们压缩成“开机后运行程序并输出结果”。

2.6.6 本章方案的成本

最小方案追求边界清楚，不追求吞吐最大。处理器轮询串行接口时不能做其他工作；每个任务都要完整传输；任务运行时固件完全停止；结果只存在 RAM；错误通常通过复位收束。这些成本不是待补的一串现代名词，而是当前路径中可以指认的位置。

当前成本	在路径中的原因	只有何时才值得改进
装入时处理器忙等	单槽串行接口只提供轮询状态	输入速度或并行工作成为问题
启动需完整传输	RAM 断电丢失且任务不在 ROM	需要持久保存多个任务
不能同时运行别的任务	只有一份当前处理器现场	第二任务需要取得处理器
无限循环无法收束	没有异步时间事件	必须限制任务运行区间
任意 RAM 写可破坏固件	物理地址没有权限	不可信任务需要隔离

下一章首先处理第三行：在不要求任务重新从入口开始的条件下，让第二项任务获得处理器。其余问题继续保留，避免一次引入尚未由失败要求的结构。

2.7 沿着处理器状态走完求和任务

任务已经获得处理器。现在用具体地址和数值逐条检查执行结果。任务开始时输入内存为：

```
MEM32[0x00102000] = 7
MEM32[0x00102004] = -2
MEM32[0x00102008] = 11
MEM32[0x0010200c] = 5
MEM32[0x00102020] = 0
```

2.7.1 初始化指令

第一笔任务取指从 `0x00100000` 到达 RAM。互连不关心这是任务代码，只因地址匹配而选择 RAM。返回字节译码为 `MOVI r4,0`。指令提交后：

```
PC: 0x00100000 -> 0x00100004
r4: 0x00000000 -> 0x00000000
other architectural state: unchanged
```

虽然 `r4` 的数值碰巧没有变化，写入动作仍由指令语义决定。若固件没有按约定清零 `r4`，这条指令也会建立相同起点。

2.7.2 第一轮：从字节到部分和

在 `0x00100004`, `LOAD r5,[r0+0]` 读取 `0x00102000--03`。事务成功后 `r5=7`, PC 进入 `0x00100008`。`ADD r4,r5` 读取旧值 0 和 7, 产生候选结果 7, 提交后 `r4=7`。

接下来的两条 `ADDI` 分别改变指针和计数：

```
r0: 0x00102000 -> 0x00102004
r1: 4           -> 3
```

`BNEZ r1,0x00100004` 读取 `r1=3`, 条件成立, 因此下一 PC 不是顺序地址 `0x00100018`, 而是循环入口 `0x00100004`。

2.7.3 四轮状态表

每轮在条件分支提交后, 关键状态如下：

轮次	本轮读地址	读出值	部分和 r4	分支后的 r1
1	<code>0x00102000</code>	7	7	3
2	<code>0x00102004</code>	-2	5	2
3	<code>0x00102008</code>	11	16	1
4	<code>0x0010200c</code>	5	21	0

第四轮后, `BNEZ` 不采用分支目标, PC 顺序进入 `0x00100018`。此时 `r0=0x00102010`, 正好指向数组末尾之后的位置。这个地址没有被解引用; 它只表示所有元素已经消费。若循环多执行一轮, 处理器会读取数组外字节, 而当前机器没有边界检查替任务阻止这一错误。

2.7.4 结果写入的可见边界

`STORE r4,[r2+0]` 形成地址 `0x00102020`, 写数据为 `0x00000015`, 宽度为 4。RAM 在事务完成时把四个字节更新为 `15 00 00 00`。在写响应返回前, 任务不能把该存储指令视为完成。

写入完成表示后续处理器读取能够取得新值。它不表示结果跨断电保存, 因为目标是 RAM; 也不表示外部发送端已经看到结果, 因为串行发送尚未发生。可见、传达和持久是三个不同边界。



图示：前三个方框是不同层次的事实。最右侧使用虚线，因为 RAM 和串行发送都不能提供断电持久性。

2.7.5 任务怎样返回固件

执行到 `0x0010001c` 时, 任务发出 `RET`。处理器必须先从 `SP=0x001efffc` 读取 4 字节返回地址。RAM 返回 `0x00000300` 后, 处理器把 SP 增加到 `0x001f0000`, 并把 PC 改为 `0x00000300`。这两个状态更新共同完成返回。

2.7.6 返回前后快照

状态	RET 开始前	RET 完成后
PC	0x0010001c	0x00000300
SP	0x001efffc	0x001f0000
MEM32[0x001efffc]	0x00000300	内容仍在，但已不属于有效栈
MEM32[0x00102020]	21	21

SP 上升不会自动擦除旧返回地址。栈的有效范围由 SP 与约定共同决定，旧字节可以继续留在 RAM 中。后续压栈可能覆盖它。

2.7.7 固件返回入口能依赖什么

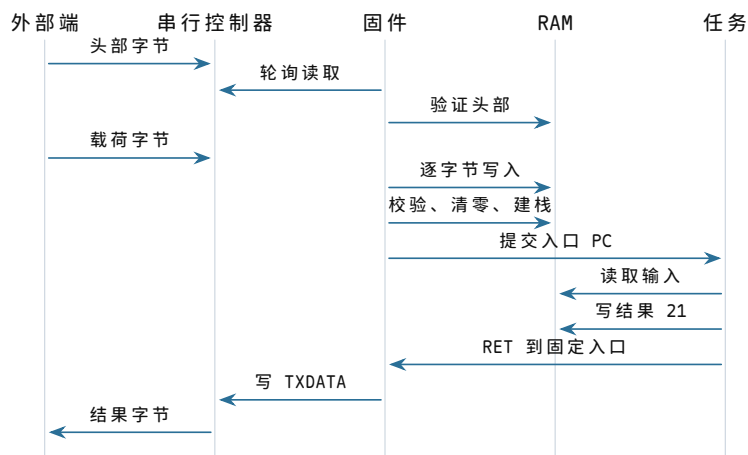
固件不能假定任务保留了自己的临时寄存器，因为本章没有要求任务这样做。固定返回入口只依赖两项事实：PC 已指向 0x00000300，结果位于约定的 RAM 地址。它重新建立自己的 SP，读取结果，等待串行发送端可用，然后按四个字节发送结果或发送文本表示。

```
firmware_return:
    SP = FIRMWARE_STACK_TOP
    result = MEM32[0x00102020]
    for each byte in encode(result):
        repeat until STATUS has TX_READY
            write8(TXDATA, byte)
    enter firmware_wait_loop
```

固件必须先重建 SP，再进行函数调用或压栈。任务返回时的 SP 已在任务栈上界，不能当作固件栈继续使用。两片栈位于同一 RAM，却属于不同执行阶段的软件约定。

2.8 正常路径的端到端时间线

前面分别建立了器件和局部事务，现在可以把它们综合成一次完整运行。下图只使用已经解释过的对象：



图示：时间从上向下。载荷进入 RAM、任务获得 PC、结果进入 RAM、结果到达外部端分别是四个不同事件；前一事件不会自动包含后一事件。

2.8.1 把每个边界写成一句可检验的事实

1. 头部接受：字段合法且后续计算不会越过任务区域；
2. 载荷接收完成：规定数量的字节已经写入 RAM；
3. 映像验证完成：校验通过、零区和栈均已建立；
4. 控制权移交：PC 已提交任务入口，下一次取指来自任务；
5. 计算完成：结果存储事务已得到 OK；
6. 任务返回：PC 和 SP 已按 RET 规则更新；
7. 外部可见：串行发送完成，外部端实际收到结果字节。

如果只说“任务运行成功”，就无法定位失败究竟发生在传输、装入、移交、计算、返回还是报告阶段。正式系统中的请求、完成和持久化边界更复杂，但分析方法从这台小机器开始已经相同：先命名对象状态，再指出哪个事件改变了它。

2.9 错误路径揭示了当前机器缺少什么

正常示例证明最小方案可行，错误路径则说明它的边界。下面每种错误都对应一个具体检测者，不能用“系统会处理”概括。

2.9.1 头部损坏：固件可以在移交前拒绝

若 magic 错误、长度越界或入口未对齐，固件仍掌握 PC，因此可以停止接收、通过串行发送错误码并回到等待头部状态。此时 RAM 中即使残留部分字节，也没有被执行。

错误处理的关键不是把 RAM 恢复成全零，而是不提交任务入口。下一次合法装入会覆盖规定范围；若固件需要避免旧数据泄漏，可以在重新使用前清零，但这属于额外策略。

2.9.2 串行溢出：控制器只能报告已经丢失

若发送端速度超过固件轮询速度，新字节会覆盖单槽接收寄存器，控制器随即置溢出位 `RX_OVERRUN`。固件无法由该位推断丢失字节的内容和位置，只能拒绝整次传输并要求外部端重发。校验和可以检测内容不一致，却不能凭空恢复缺失字节。

增加接收 FIFO、流量控制或 DMA 都能改变吞吐边界，但当前问题只需要一个最小可靠方案，所以采用“检测溢出并重传”的规则。后续若设备速率成为主要矛盾，再引入更复杂硬件。

2.9.3 未映射地址：互连报告错误，任务没有恢复机制

若任务错误地读取 `0x20000000`，互连返回 `UNMAPPED`。教学处理器把失败地址、访问方向和当时 PC 写入固定固件错误槽，然后跳到固件错误入口。这使机器至少不会把任意电平当作成功数据。

本章尚未为不同任务建立独立错误现场，也没有把错误转换成可返回给应用的信号。故障后的最小处理是报告并复位。错误可检测不等于错误可恢复。

2.9.4 任务写坏固件工作区：软件约定无法强制隔离

任务执行：

```
STORE r4, [0x001f0000]
```

时，地址合法地落在 RAM 范围。互连和 RAM 都会完成写入，因为它们不知道该范围按约定属于固件。任务随后返回时，固件的装入记录或栈可能已经损坏。这不是地址互连故障，而是当前机器缺少访问权限边界。

2.9.5 栈指针损坏：返回地址的存在也不足以保证返回

若任务把 SP 改成 `0x00102000` 后执行 `RET`，处理器会把第一个输入整数 `0x00000007` 当作返回 PC。该地址未按指令对齐或落在无效区域，机器随后进入错误入口。固件预置返回地址只能保证合作任务有一条返回路径，不能保证任意代码使用它。

2.9.6 无限循环：没有外部事件就无法取回处理器

任务可能执行：

```
repeat:
  ADDI r4, 1
  BNEZ r4, repeat
```

只要指令和 RAM 访问合法，处理器就会继续执行任务。串行控制器即使收到新命令，当前代码也没有读取它。时钟边沿只能推动指令前进，并不会把 PC 自动改回固件。机器此时既没有可编程时钟事件，也没有异步入口，固件无法重新获得控制。

这一失败是后续演进的关键证据。“任务最终会执行 RET”是合作约定，不是机器保证。若希望运行多个任务或限制单个任务占用处理器的时间，必须先回答怎样保存正在执行的现场，再回答怎样在任务不合作时进入受控代码。

2.9.7 这台机器的软件模型究竟是什么

到本章末，程序员面对的不是一堆孤立器件，而是一组边界清楚的契约。

2.9.8 地址契约

一个 32 位物理地址经过互连选择启动存储、RAM、串行控制器或错误响应。读写宽度和方向是事务的一部分。地址只标识位置，不表达对象身份、权限或所有权。

2.9.9 执行契约

PC 指出下一条指令，寄存器与 RAM 保存继续计算所需的状态。普通指令顺序推进，分支、调用和返回按明确规则改变 PC。只有已经提交的状态才属于下一条指令的输入。

2.9.10 启动契约

复位先进入固件。固件验证任务映像，建立内存布局、参数和栈，最后提交任务入口 PC。任务执行 RET 才会沿预置栈内容回到固定固件入口。

2.9.11 设备契约

串行控制器把异步到达的线信号变成带状态位的字节槽。固件通过轮询消费字节，并负责在溢出或校验失败时拒绝整次传输。控制器不理解任务格式。

2.10 为什么这仍然不是操作系统

固件已经做了不少软件工作：它初始化器件、接收字节、检查范围、装入任务、建立初始现场并报告结果。但“存在启动软件”不等于“已经有操作系统”。当前方案只服务于一条预先约定的运行路径，没有建立长期存在的任务身份和资源管理规则。

当前已经具备	依靠什么实现	当前仍未具备
确定的复位入口	复位状态与启动存储	多个任务的身份和生命周期
任务字节装入 RAM	串行协议与固件装入记录	文件、目录和持久命名
一次明确的控制权移交	初始寄存器、栈和 PC 提交	暂停后恢复任意任务现场
合作式返回	预置返回地址与 RET	强制收回处理器
物理地址访问	地址互连	地址隔离和访问权限
错误检测与复位	互连错误和固件入口	面向任务的独立错误恢复

表的左列是本章已经逐步建立并走读过的能力，右列则是具体失败留下的空缺。后续章节不会直接罗列现代操作系统模块，而会从这些空缺中一次选择一个进行演进。

2.10.1 本章复盘：从一个要求到一台可运行机器

最初的要求只有“计算四个整数之和”。为了让这句话在真实机器上成立，我们依次得到以下因果链：

1. 计算必须跨动作保留事实，因此需要寄存器、PC、SP 和可写状态；
2. 第一条指令必须跨断电存在，因此需要启动存储；
3. 运行数据必须可变，因此需要 RAM；
4. 多个器件不能同时响应同一请求，因此需要地址互连和明确事务；
5. 上电状态不可靠，因此需要时钟边界、复位状态和固定入口；
6. 外部任务不能凭空进入 RAM，因此需要串行控制器和传输协议；
7. 任意字节不能直接执行，因此需要范围、校验和入口验证；
8. 任务不能继承固件偶然状态，因此需要初始寄存器、独立栈和移交顺序；
9. 任务完成后必须有控制流去向，因此需要预置返回地址和固定固件入口。

这条链不是整机启动的口号，而是每一步都有对象、字段和状态变化的运行记录。求和任务的最终结果为 21，RAM 中的结果字节为 15 00 00 00，外部端只有在串行发送完成后才能看到它。

2.11 为什么四轮结束时一定得到 21

逐条 trace 能证明本次输入的实际运行结果，还可以从循环状态中提炼一个每轮都成立的关系。设原数组为 a_0, a_1, a_2, a_3 ，总元素数为 $n = 4$ ，已经处理的元素数为 k 。每次进入 LOAD 前，程序应满足：

$$\begin{aligned} r0 &= \text{array_base} + 4k, \\ r1 &= n - k, \\ r4 &= \sum_{i=0}^{k-1} a_i. \end{aligned}$$

这个关系称为循环不变量，因为初始化建立它，每一轮保持它，退出条件再利用它推出结果。

2.11.1 初始化为什么建立不变量

第一轮之前 $k = 0$ 。固件令 $r0=\text{array_base}$ 、 $r1=4$ ，第一条任务指令令 $r4=0$ 。空前缀的和定义为 0，因此三个等式全部成立。这里能看出任务代码和启动约定共同建立初态：若固件传错 $r0$ 或 $r1$ ，单靠 $\text{MOVI } r4, 0$ 不能修复。

2.11.2 一轮怎样保持不变量

假设进入第 k 轮时关系成立。LOAD 从 $\text{array_base} + 4k$ 读取 a_k ，ADD 把它加入 $r4$ ；ADDI $r0, 4$ 令指针对应 $k+1$ ，ADDI $r1, -1$ 令剩余数变成 $n - (k+1)$ 。于是下一次进入循环时：

$$r4 = \sum_{i=0}^k a_i,$$

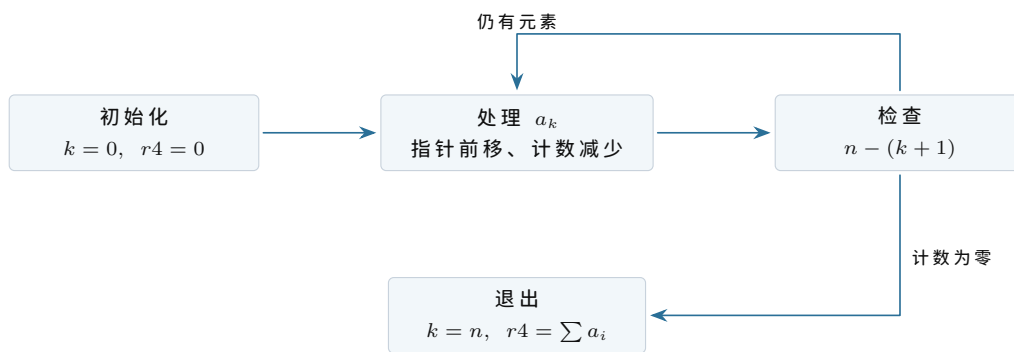
正好是把 k 替换为 $k+1$ 后的不变量。

2.11.3 退出条件怎样推出最终结果

分支不采用循环目标时 $r1=0$ 。由 $r1 = n - k$ 得 $k = n = 4$ ，因此

$$r4 = a_0 + a_1 + a_2 + a_3 = 7 - 2 + 11 + 5 = 21.$$

STORE 把该值写到结果槽。这个推导还揭示零长度问题：当前代码在第一次分支检查前就执行装入，只有 $n > 0$ 时初始化到第一次循环入口的路径才满足预期使用条件。



图示：机器 trace 给出每步具体值，不变量说明这些值为何对任意满足启动契约的非空数组都沿同一关系推进。

2.11.4 装入一项任务需要多少次数据移动

载荷长度 $0x2100$ 等于 8448 字节，也恰好是 132 个 64 字节块。对每个字节，串行控制器先把外部位收进 $RXDATA$ ；处理器读取一次 $RXDATA$ ；随后处理器向 RAM 写入一次。仅计算有效数据移动，就有 8448 笔设备读和 8448 笔 RAM 写。

状态轮询次数不固定。如果每个字节到达时固件恰好检查一次，需要至少 8448 笔 $STATUS$ 读；若固件检查十次才等到一个字节，状态读会增加到约 84480 笔。轮询的成本来自“没有数据时仍要主动读取状态”。

动作	本次最少次数	次数由什么决定
读取 $RXDATA$	8448	载荷字节数
写入任务 RAM	8448	载荷字节数
读取 $STATUS$	不少于 8448	到达间隔与轮询频率
发送块确认	132 次确认帧	64 字节分块规则

零填充写入	256 字节	zero_bytes=0x100
建立返回地址	1 笔 4 字节写	启动栈约定

这个计数没有把取指事务算入，因为固件执行每条轮询和搬运指令还要取指。即使不建立精确周期模型，也能看出装入成本至少与载荷长度线性增长。DMA 可以减少逐字节处理器搬运，但要引入描述符、缓冲所有权和完成事件；当前规模没有要求它。

2.11.5 确认帧不是免费的

每 64 字节等待一次确认会限制连续突发长度，也增加往返时间。若确认帧占 8 个串行字节，132 次确认会额外发送 1056 字节。把块增大到 256 字节可减少确认次数，却会让出错后重传更多数据，并要求控制器或协议容纳更长未确认区间。

因此块大小不是任意常数。它在状态开销、错误恢复粒度和缓冲需求之间取舍。本章固定为 64 字节，只为使单槽轮询方案具有明确背压点。

2.11.6 三个快照足以区分“装入”“运行”和“返回”

2.11.7 快照一：映像已经在 RAM，但固件仍运行

```
load_record.state = VERIFYING
PC = 0x00000280
SP = 0x001ffec0
MEM[0x00100000..0x001020ff] = received image
task registers = not established
```

这时读取任务区域能看到完整字节，却不能说任务已运行。PC 仍在启动存储，寄存器和栈也仍属于固件阶段。

2.11.8 快照二：第一条任务指令尚未提交

```
load_record.state = RUNNING
PC = 0x00100000
SP = 0x001efffc
r0 = 0x00102000
r1 = 4
r2 = 0x00102020
```

PC 已指向任务入口，下一笔取指将访问 RAM。第一条 `MOVI` 尚未提交，因此这个快照是控制权移交后的初始任务现场。

2.11.9 快照三：任务返回入口刚获得控制

```
PC = 0x00000300
SP = 0x001f0000
r4 = 21
```

```
MEM32[0x00102020] = 21
firmware SP = not restored yet
```

处理器已经进入固件地址，但返回入口的第一条初始化指令尚未执行。SP 仍表示任务栈的空栈位置；固件必须先建立自己的 SP，之后才能安全调用发送例程。

快照	PC 所属范围	SP 所属范围	当前可执行者
映像已到齐	启动存储	固件工作区	固件装入器
任务初始现场	任务 RAM	任务栈	求和任务
返回入口刚进入	启动存储	任务栈上界	固件返回入口

第三行尤其说明 PC 和 SP 不必在同一条指令边界同时“属于同一软件”。受控入口的第一项职责常常就是从被打断或返回方的栈切换到入口代码自己的栈。当前固定返回协议很简单，但已经展示了这个状态过渡。

2.11.10 错误码怎样对应到最早能够判断的层次

错误码	最早检测者	能证明的事实
SERIAL_OVERRUN	串行控制器/固件	至少一个外部字节未被消费
HEADER_INVALID	固件头部验证	字段不能形成安全布局
CHECKSUM_MISMATCH	固件载荷验证	实收载荷不同于发送端声明
INVALID_OPCODE	处理器译码	PC 指向的四字节不是合法指令
UNMAPPED	地址互连	一笔合法格式请求没有目标
ALIGNMENT	处理器或互连	地址违反当前宽度对齐规则

错误码不能相互替代。校验失败不能说明是哪条指令错误；无效操作码也不能证明串行传输损坏，因为发送端可能原本就提供了错误代码。诊断应报告最早被可靠观察到的契约破坏，同时保留 PC、地址和装入阶段等上下文。

经过这些计数、快照和不变量检查，本章不再依赖“任务应该可以运行”的直觉。机器的正确起点、数据移动量、执行结果和失败位置都能由具体状态复算。

2.11.11 同一个“完成”对三个观察者含义不同

求和任务、固件和外部发送端处在不同位置，能够观察的事件也不同。任务在存储指令收到成功响应后知道结果已经进入 RAM；固件只有在任务返回后才开始读取结果；外部端则要等结果帧通过串行线到达并完成校验。

观察者	首次能够确认的事件	此时仍不能推出
求和任务	STORE 得到 OK	固件已经运行、外部端已经收到

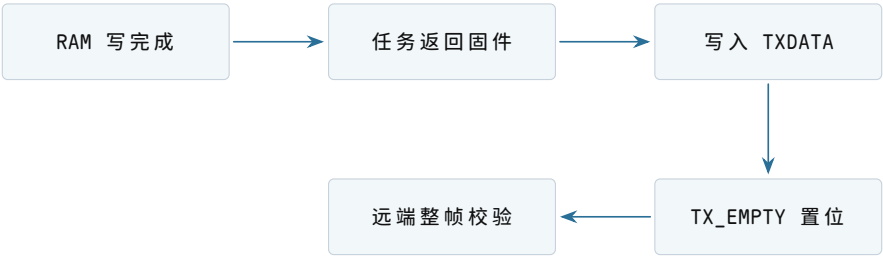
固件返回入口	重新读取结果槽得到 21	发送线已经传完结果帧
外部发送端	完整结果帧到达且校验通过	结果断电后仍会保留

若固件在写入 TXDATA 后立即复位串行控制器，最后一个字节可能还在移位寄存器中，外部端收不到完整帧。因而“处理器完成设备寄存器写入”和“设备完成物理发送”也不是同一时刻。最小接口可以增加 TX_EMPTY 状态，表示发送槽和移位寄存器都为空；固件在进入等待或复位前检查它。

硬件模型。发送侧除 tx_ready 外保存移位寄存器占用状态，最后一位离开引脚后置 tx_empty=1。

软件模型。STATUS 的 bit3 暴露 TX_EMPTY。写 TXDATA 后该位清零，整字节发送结束后重新置位。

抽象。软件等待 TX_EMPTY=1，可以依赖此前提提交的字节已经离开控制器；这仍不保证远端正确采样或通过帧校验，端到端确认必须由远端协议提供。



图示：五个事件顺序发生，分别关闭计算、控制流、设备提交、物理发送和端到端接收边界。

2.11.12 重新运行任务与整机复位有什么不同

固件收到下一份合法头部后，可以覆盖任务区、重建栈并再次提交入口，而不必让电源复位。这种重新运行保留启动存储内容和固件初始化后的设备配置，只重建与本次任务相关的状态。

整机复位则把 PC 送回 0x00000000，清除处理器控制状态、串行协议状态和互连未完成事务。RAM 字节可能仍在，但不再被信任。两条路径的共同点是：在任务入口提交前都必须重新验证映像并建立现场。

状态	固件重新运行任务	整机复位
启动存储	不变	不变
固件设备配置	可以沿用	重新初始化
RAM 旧任务字节	按新映像覆盖或清零	可能仍在，但必须视为未验证
处理器 PC/SP	由固件直接建立新任务现场	先回复位入口，再执行固件

未完成串行事务	由协议明确丢弃和重同步	控制器复位后丢弃
---------	-------------	----------

“重启程序”在当前裸机上不是硬件指令，而是一段固件流程。硬件复位提供确定的最外层恢复手段，固件重新运行则是较小范围的软件动作。

2.11.13 自修改写入说明保护不能由布局表代替

设错误任务把结果地址 `r2` 改成 `0x00100004`，即循环中 `LOAD` 指令的位置。最终 `STORE` 会把整数 21 的字节 `15 00 00 00` 写入代码。任务随后立即执行位于 `0x0010001c` 的 `RET`，所以本次仍可能正常回到固件。

固件若不重新装入就再次从 `0x00100000` 运行任务，第二条指令字节已经不再是 `20 05 00 00`。操作码 `0x15` 未定义，处理器进入 `INVALID_OPCODE` 错误入口。第一次运行的结果正确，第二次才暴露代码破坏。

时刻	MEM[0x00100004..07]	后果
装入完成	20 05 00 00	译码为数组装入
错误存储后	15 00 00 00	当前 PC 已越过此处，暂未失败
再次从入口执行	同上	第二条取指得到无效操作码

地址布局告诉任务“哪里应当是代码”，RAM 和互连却没有只读规则。要阻止这种写入，机器需要一种把地址范围与允许操作绑定的强制机制。此处只记录失败，不提前展开地址保护；它会在后续机器真正需要隔离时成为新的演进问题。

2.11.14 本章事实之间的最终依赖关系



图示：每个方框都依赖前一个方框已经建立的状态。缺少任意一环，都不能从“字节存在”直接推出“任务完成”。

至此，最小机器的器件、连接、软件接口、启动协议、执行 `trace` 和错误边界均已落到具体状态。后续章节可以把这台机器作为已知起点，而不必再次把“处理器能够执行一项任务”当作未经证明的前提。

2.11.15 从可观察现象反推连接错误

具体机器模型还应支持诊断。若上电后没有得到结果，不能只说“机器没跑起来”，而要利用 PC、事务和状态寄存器把故障范围逐步缩小。

2.11.16 复位后连第一笔请求都没有

若示波观察不到 `req_valid`，先检查时钟是否产生边沿、复位是否释放以及 PC 是否得到规定值。此时还没有理由检查任务映像，因为固件第一条取指尚未发生。

2.11.17 请求地址正确，但始终没有响应

观察到地址 `0x00000000` 和读请求，却没有 `resp_valid`，故障位于互连选择、启动存储连接或目标握手。若互连错误地把该地址送往 RAM，RAM 中的随机字节可能返回成功，随后表现为无效操作码。区分“无响应”和“错误目标响应”需要同时观察目标选择信号。

2.11.18 固件运行，但接收状态永远为零

若 PC 已在固件轮询循环，`STATUS` 读也持续成功，问题不在处理器取指和 RAM。应检查外部发送端是否产生串行位、控制器采样时序是否匹配以及接收移位寄存器是否能置 `RX_READY`。软件看到的零值只是控制器没有宣布完整字节，不足以判断线路哪一端失效。

2.11.19 任务结果正确，外部端却没有收到

先读 `MEM32[0x00102020]`。若值为 21，计算和 RAM 写入已经完成；再看 PC 是否到达固件返回入口，区分“任务未返回”和“返回后发送失败”；最后检查 `TX_READY/TX_EMPTY` 与远端帧校验。诊断顺序沿完成边界向外推进。

最后确认成功的事实	下一项观察	尚不应检查的远端事项
时钟与复位正常	第一笔取指请求	任务求和结果
固件取指成功	串行状态与接收字节	任务返回地址
映像校验成功	初始现场与入口 PC	外部结果帧
结果槽为 21	返回入口 PC	传输校验
<code>TX_EMPTY=1</code>	远端接收与帧校验	RAM 计算过程

这种从已知成功边界向下一边界推进的方法，与后续操作系统中的系统调用、设备请求和文件持久化诊断具有相同结构：先确定事实在哪一层成立，再检查下一层所需的新状态。

2.11.20 最小机器的最终接线清单

到本章末再列清单，作用是复核已经推导的连接，而不是用名词代替正文：

连接	承载的信息	接错后的直接后果
时钟到处理器、互连和控制器	共同状态更新边沿	握手双方不能在同一边界观察状态
复位到处理器与控制状态	PC 起点、协议初态	旧事务或任意 PC 被当作有效

处理器请求到互连	地址、读写、宽度、写数据	无法取指或访问错误目标
目标响应经互连回处理器	状态与读数据	指令永久等待或读取错误字节
互连到启动存储	固件读请求	复位后没有可靠指令
互连到 RAM	任务取指和数据读写	任务无法装入或保存运行状态
互连到串行控制器	状态、收发字节	外部任务无法进入或结果无法报告
串行控制器到外部端	按时间传送的位	软件寄存器与外部信息断开

每一行都能回到前文的一次具体事务。清单中没有中断控制器、计时器、地址转换器、磁盘或操作系统；这些结构只有在后续问题确实需要时才会加入，并重新说明硬件模型、软件接口和提供的抽象。

本章得到的机器。固定启动程序已经能够接收、验证和装入一项外部程序，并在程序合作返回后等待下一次传输。但 RAM 中只有一套当前程序布局和一份活动处理器状态。若希望保留 A 的中间结果，同时让 B 获得处理器，就必须回答暂停现场保存在哪里以及怎样恢复。

第二项任务为什么需要常驻协调代码

继承的机器。固件能装入一项任务，为它设置入口、栈和初始寄存器；任务完成时主动跳回固定入口。所有地址仍是物理地址，任务仍可访问整台机器。

3.1 直接从 A 跳到 B 会丢掉什么

设 A 正在累加一组数据。下一条指令地址保存在指令位置寄存器中，当前循环计数和部分和位于通用寄存器，函数调用关系位于 A 的栈。此时若把指令位置改成 B 的入口，处理器会立即开始取 B 的指令；但它仍带着 A 的栈顶与寄存器值。

给 B 设置自己的栈和寄存器可以让 B 正确开始，却会覆盖处理器中唯一的一份 A 状态。以后再把指令位置改回 A，并不能恢复 A，因为“不知道回到 A 的哪条指令”只是丢失状态的一部分。

必须保留的状态	丢失后的表现	为什么不能从代码映像重建
继续指令位置	A 从错误位置执行或重新开始	它取决于暂停发生的时刻
栈指针	返回地址与局部变量无法找到	栈深度随已经完成的调用变化
通用寄存器	循环变量、地址和中间结果改变	它们是运行结果，不是初始映像
状态标志	后续条件分支得到不同结果	标志来自暂停前最后一次运算
约定的扩展状态	浮点或向量计算被 B 覆盖	同样属于运行中的中间结果

这组状态可以称为任务的处理器现场。名称出现在问题之后：我们需要一份数据，能够让暂停的任务在未来像没有被中断一样继续，才有必要给这份数据命名。

3.2 现场保存在哪里

不能把 A 的现场继续留在处理器寄存器里，因为 B 就要使用这些寄存器。也不能把所有现场都压到同一栈上，再任意切换栈而不记位置。最小安排是为每项任务分配一块现场记录和一段独立栈：

```
task_state A:
    status
```

```
resume_instruction
stack_pointer
general_registers
arithmetic_flags
stack_low, stack_high

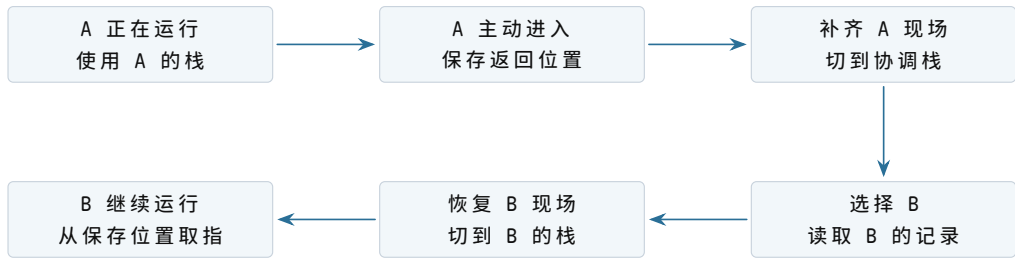
task_state B:
    same fields, different storage
```

`status` 至少区分“尚未开始”“可以继续”“正在运行”和“已经完成”。它不是为了美化结构，而是避免协调代码把一份只有初始入口、尚无暂停现场的记录当作可恢复记录。

3.3 谁有资格保存现场

若 A 自己把寄存器依次写入记录，会遇到一个细节：执行保存代码本身也需要寄存器，而且“保存后的继续地址”应当指向 A 交还处理器之后的位置。最小方案规定一个调用约定：A 像调用函数一样进入协调代码，调用约定先把一部分状态压到 A 的栈，协调代码再补存其余状态并记录当前栈指针。

协调代码必须常驻在不会被任务映像覆盖的内存中。它还需要一个只在选择任务和恢复现场时使用的协调栈，否则恢复 A 的栈之后再运行复杂选择代码，会误把协调者的局部数据写进 A。



图示：切换不是把 A 的代码换成 B 的代码，而是先把处理器唯一的活动现场写回 A 的记录，再把 B 的记录装入处理器。

3.4 第一次运行和恢复运行为何不同

B 第一次获得处理器时，没有“暂停位置”。它的记录由装入者人工构造成一种可恢复形态：继续位置设为 B 的入口，栈指针设为 B 的初始栈顶，其余寄存器按启动约定给值。以后 B 主动交还处理器，记录才被真实运行状态覆盖。

B 的阶段	记录中的继续位置	栈中已经存在的内容
尚未第一次运行	任务入口	人工准备的初始参数或空栈
运行后主动交还	交还入口之后的继续点	B 已经形成的调用链与局部数据

已经完成	不再作为可恢复位置使用	可等待回收或检查结果
------	-------------	------------

统一的恢复代码只读取记录，不必在最后一步区分“第一次”和“后来”。区别已经在记录如何构造、状态字段是否允许恢复中表达。

3.5 最小选择规则怎样形成

当前只有 A 和 B。协调代码可使用一个当前编号：A 交还时若 B 可继续，就选择 B；B 交还时再选择 A。这个规则尚不需要复杂队列，但仍要处理任务完成：

```
choose_next:
  inspect the other task
  if it is ready:
    return that task
  if current task only yielded and remains ready:
    return current task
  otherwise:
    wait for an external event
```

这里第一次出现“选择下一项可运行任务”的职责。由于选择只发生在任务主动进入协调代码后，它仍是合作式安排：协调者能决定交还之后运行谁，却不能决定任务何时交还。

3.6 一次切换的状态账本

时刻	处理器活动现场	A 的内存记录	B 的内存记录
t_0	A 的状态	旧值，不可用于恢复 当前 A	B 的初始或暂停状态
t_1	A 正在执行保存入口	部分字段已更新	不变
t_2	协调代码使用协调栈	A 的完整暂停状态	不变
t_3	正在装入 B 的状态	完整且稳定	被读取，尚未运行
t_4	B 的活动现场	A 可恢复	B 记录成为旧值

任一时刻，正在运行任务的最新寄存器在处理器中，其内存记录只是上一次保存值；暂停任务的最新现场则必须完整位于内存记录和它自己的栈中。说“每项任务都有一份现场”时，需要同时说明活动现场究竟位于处理器还是内存。

3.7 这段常驻代码已经是什么

它不是完整操作系统，却已经具有三个此前不存在的职责：保存当前任务、选择下一任务、恢复所选任务。由于任务运行期间协调代码仍保留在内存中，可以把它

称为常驻协调者。以后若加入更多任务，选择规则和记录集合会扩展，但本章不提前设计最终结构。

章末出现的问题。A 可以约定每处理一批数据就主动交还处理器；B 却可能进入无限循环，或者因为错误永远不调用交还入口。此时协调代码虽然常驻，却没有任何机会执行。下一章只处理“机器怎样在任务不合作时重新得到控制”。

任务不交还处理器时怎么办

继承的机器。常驻协调者能保存、选择和恢复 A/B；每项任务有独立现场记录与栈。切换入口只能由正在运行的任务主动调用。

4.1 常驻并不等于有机会执行

B 执行下面的指令序列：

```
repeat_forever:
  compute
  jump repeat_forever
```

处理器会不断从 B 的地址取指。协调代码虽然存在于另一段内存，但“存在”不会自动改变指令位置。外部观察者也不能靠在内存里写一个标志解决问题，因为 B 没有读取该标志。

要在 B 不执行交还指令时重新获得控制，机器必须提供一种与当前指令流独立的事件：事件到达后，处理器先保存足以继续 B 的状态，再把指令位置改成预先登记的入口。

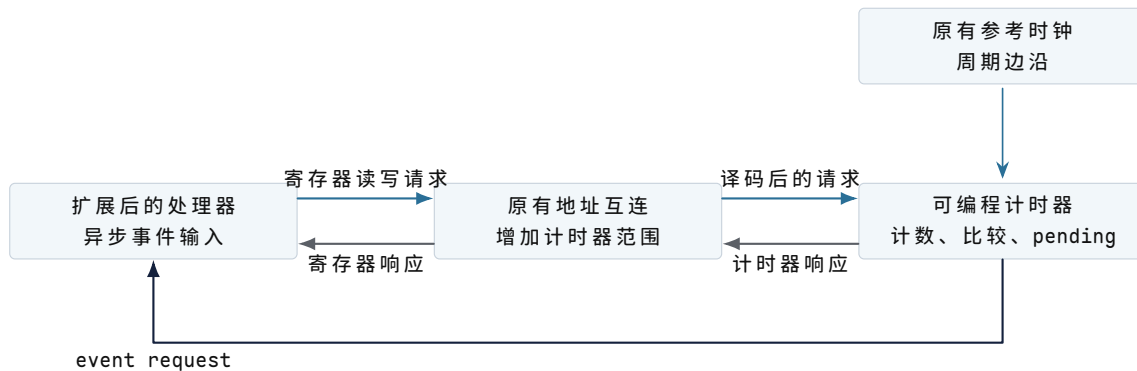
4.2 为什么选择可编程时钟事件

设备完成也能产生外部事件，但机器空闲时未必有设备活动，不能用它保证处理器一定被收回。一个可编程计时器可以在设定时间后发出事件，与 B 是否读内存、是否调用函数无关。

4.2.1 新器件接到现有机器的哪里

第一章的时钟源只能推动电路状态变化，软件不能要求它在未来某个计数值通知处理器。现在增加的计时器是一项独立器件：它接收同一参考时钟，通过地址互连暴露配置寄存器，并用一根事件请求线连接处理器的异步事件输入。

本样机只有一个事件源，所以计时器请求线直接连接处理器，不增加中断控制器。以后事件源增多，才会出现“多根请求线如何编号、屏蔽和路由”的新问题。



图示：总线方向用于配置和查询，事件请求线用于计时器主动通知处理器。软件写过寄存器并不等于事件已经发生；只有计数条件满足后，计时器才置 pending 并拉起请求线。

4.2.2 计时器的硬件模型

计时器内部保存四类最小状态：

内部状态	状态变化	对外作用
counter	每个参考时钟边沿递增	提供单调增加的比较基准
deadline	处理器通过写事务设置	指定下一次到期计数值
enabled	处理器启用或停止计时比较	决定是否检查到期条件
pending	到期时置 1，确认后清 0	为 1 时保持事件请求有效

每个时钟边沿后，若 `enabled=1` 且 `counter >= deadline`，硬件就设置 `pending=1`。请求线保持有效，直到软件确认事件。采用“保持”而不是一个极短脉冲，可以避免处理器暂时关闭事件接收时永久丢失通知。

这个模型没有自动周期。协调代码处理一次到期事件后，写入新的 `deadline`，才形成重复时间片。这样可以先看清“本次到期”和“安排下次到期”是两个动作。

4.2.3 计时器的软件模型

软件不直接访问内部连线，而是通过地址互连看到一组内存映射寄存器：

物理地址与名称	访问方式	软件可见语义
0x40001000 COUNT	只读	返回当前 counter 快照
0x40001008 DEADLINE	读写	设置下一次到期比较值
0x40001010 CONTROL	读写	bit 0 控制 enabled
0x40001014 STATUS/A CK	读/写	读 bit 0 得 pending；写 1 清除 pending

让计时器在 Q 个计数单位后产生一次事件，软件按以下顺序配置：

```
now = read64(COUNT)
```

```
write64(DEADLINE, now + Q)
write32(STATUS_ACK, 1)      // clear an earlier pending event
write32(CONTROL, ENABLE)
```

若先启用、后写 deadline，旧 deadline 可能立即满足并产生一次非预期事件。寄存器顺序因此是软件模型的一部分，不只是代码风格。

4.2.4 计时器提供的抽象

上层软件得到“在单调计数达到 deadline 后形成一个保持到确认的事件”。它没有得到以下保证：

- deadline 不是日期，也不自动等于毫秒；
- 到期表示请求已经提出，不表示处理器恰好在该计数值执行入口；
- 事件被屏蔽时，请求可以延迟交付；
- 计时器不会保存任务现场，也不会选择下一项任务。

因此，计时器抽象适合保证协调代码最终重新获得一次入口机会；任务选择仍属于软件。

4.2.5 处理器异步入口也需要明确三层模型

计时器有了请求线，原处理器仍只会顺序取指。机器还要扩展处理器的事件入口能力。

硬件模型。处理器在一条指令完成后检查事件输入。若事件允许且请求有效，处理器保存下一条指令位置、原栈指针和状态标志，切到固定入口栈顶，关闭同类事件嵌套，再把指令位置改为固定事件入口。事件返回指令执行相反的受控恢复。

软件模型。固件把事件入口约定为 `0x00008000`，把入口栈顶约定为 `0x001f0000`。硬件建立的最小帧按顺序包含：

```
event_frame:
  resume_instruction
  interrupted_stack_pointer
  saved_flags
```

入口代码能读取这份帧，并通过确认寄存器清除计时器 pending。普通通用寄存器尚未由硬件保存，必须由入口软件补齐。

抽象。软件得到一个可在普通指令流之外发生的受控入口，以及一份足以返回被打断位置的最小继续状态。该抽象不等于完整上下文切换：它不保存全部寄存器，也不决定事件到达后继续原任务还是恢复另一个任务。

为支持这条路径，硬件至少要明确：

硬件能力	必须提供的事实	软件随后才能做的事
计时器	到期时产生一个可识别事件	规定最长连续运行区间

事件入口表	事件编号对应固定入口地址	让协调代码得到控制
最小现场保存	保存被打断继续位置与必要标志	以后从原指令流继续
事件屏蔽状态	防止入口最早阶段被同类事件反复覆盖	完成一次一致的现场建立
事件返回指令	从规定帧恢复状态与执行级别	回到所选任务

硬件不必替软件决定运行 A 还是 B。它只保证事件发生时能离开 B，并把控制交到一个软件已知入口。选择仍由协调代码完成。

4.3 硬件保存的状态为什么通常不够

事件到达的第一目标是安全进入入口，而不是一次完成所有任务切换。处理器通常只自动保存继续位置、状态标志以及进入和返回所必需的少量字段。通用寄存器若要保持不变，入口软件仍要尽早保存。



图示：时钟事件只强制打开入口机会。完整保存、任务选择和恢复仍是软件协议。

入口不能先使用尚未保存的通用寄存器计算，因为那会覆盖 B 的值。最早几条指令只能使用硬件约定允许破坏的状态，或者把寄存器按固定顺序压入已知安全的栈。

4.4 事件入口使用哪一份栈

B 的栈可能已接近边界，甚至因为错误把栈指针改成无效值。若硬件入口无条件继续使用 B 的栈，保存第一项状态就可能失败。机器需要一个不由普通任务控制的入口栈位置；单核样机可让硬件或最早入口从固定协调状态取得它。

栈	保存的调用关系	所有权边界
A 的任务栈	A 的函数返回地址与局部数据	A 运行或暂停时保留
B 的任务栈	B 的函数返回地址与局部数据	B 运行或暂停时保留

协调入口栈	事件最早保存与选择代码局部状态	普通任务不应改写
-------	-----------------	----------

入口栈解决的是“如何安全开始保存”，不是自动隔离。由于当前机器仍允许 B 写任意物理地址，B 依然能够故意或错误地覆盖协调入口栈；这个缺陷留到章末。

4.5 怎样把事件帧并入 B 的任务记录

事件帧记录 B 被打断的位置，软件补存通用寄存器。协调者可以让 B 的任务记录指向这份完整帧，而不必把每个字段再复制一次。关键是记录必须说明帧位于哪一段栈、布局采用哪个入口约定。

```
timer_entry:
    hardware has saved resume position and flags
    save remaining registers in fixed order
    current_task.saved_stack = current stack pointer
    current_task.status = ready
    switch to coordinator stack
    next = choose_ready_task()
    restore next.saved_stack
    return_from_event
```

主动交还与时钟打断可以最终汇入相同的“已形成完整现场”边界，但两者最早入口不同。主动交还遵守普通调用约定；时钟可以发生在任意指令，必须按异步入口约定保存所有可能被后续代码破坏的状态。

4.6 第一次出现可运行集合

只有两项任务时也需要区分“暂停但能继续”和“已经完成”。协调者不应把完成的任务重新装入处理器。可以维护一个很小的可运行集合：

任务状态	是否参与选择	状态改变的入口
ready	是	被时钟打断或主动交还后
running	否，当前已占用处理器	协调者恢复该任务时
finished	否	任务进入完成入口时
not started	可先构造初始现场再加入	装入者完成代码、数据和栈准备时

选择规则可以是 A、B 轮流。每次恢复前重新设置计时器，给所选任务一个有限运行区间。时间区间结束并不表示任务完成，只表示协调者获得一次重新选择的机会。

4.7 “被唤醒”和“正在运行”尚未出现

本章任务只做计算，没有等待设备或条件；ready 已足够表达“具有运行资格”。以后加入等待后，某项任务即使没有完成也可能暂时不应进入选择集合，届时才需要额外状态和唤醒协议。现在提前画完整生命周期只会让尚无问题来源的概念进入样机。

4.8 一次强制切换的逐时刻状态

时刻	控制流位置	B 的最新现场	A 的最新现场
t_0	B 的循环	处理器中	A 的内存记录与栈
t_1	硬件事件入口	最小字段在事件帧， 其余仍在寄存器	不变
t_2	软件入口完成保存	B 的完整帧与栈	不变
t_3	协调者选择 A	稳定，可供以后恢复	正在被读取
t_4	事件返回到 A	B 的内存现场	处理器中

事件返回的目标不必是刚才被打断的 B。硬件要求返回帧格式正确；软件可以在返回前换成 A 的已保存帧。这个能力使时钟入口成为抢占切换点。

4.9 本章新增结构的准确边界

这一阶段只新增：

- 可编程计时器及其事件入口；
- 不依赖普通任务栈的最早入口状态；
- 能容纳异步打断的完整任务现场格式；
- ready/running/finished 三类选择状态；
- 每次恢复前重新设置时间预算的轮流选择规则。

它没有新增文件、用户态、系统调用、虚拟地址、权限或设备服务。所谓“最小抢占调度”在此只表示：协调者能在任务未主动交还时获得入口，并从可运行集合选择另一项任务。

章末出现的问题。 B 已经不能永久独占处理器，却仍能执行一条写指令覆盖 A 的栈、A 的代码、协调者记录或时钟入口。时间上的轮流使用已经建立，空间上的互不破坏仍未建立。下一次演进应从这次真实覆盖失败出发，再决定硬件与软件需要增加什么；本样稿在此停止。

样稿停在这里

此时机器已经能自动装入程序，保留两项程序的运行现场，并依靠时钟事件收回处理器。然而程序仍共享同一组物理地址，任何程序都能覆盖协调代码或另一项程序。这是下一次演进应处理的问题；本样稿暂不提前给出答案。